RTL Design Sherpa

RTL AMBA AXI4 Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXI4 (Advanced eXtensible Interface) Modules

Location: `rtl/amba/axi4/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXI4 subsystem is a complete implementation of the ARM AMBA AXI4 protocol: masters, slaves, monitors, data width converters, and interconnect components. AXI4 is the high-performance, high-frequency protocol in the AMBA family, designed for multi-master, multi-slave systems with out-of-order transaction support. This page is the map — what exists, where it lives, and how the pieces fit together.

1.1.1 Key Features

- **Full AXI4 Protocol Support:** Complete implementation of read and write channels
- **Burst Transactions:** Support for fixed, incrementing, and wrapping bursts

- **Out-of-Order Transparent:** `AxID`/`RID`/`BID` are carried end-to-end, so out-of-order slaves are supported; the buffer modules themselves are ID-agnostic pass-throughs and do not reorder or track transactions
- **Quality of Service (QoS):** Priority-based arbitration support
- **Monitoring Infrastructure:** Comprehensive verification and debug capabilities
- **Data Width Conversion:** Automatic upsizing and downsizing
- **Clock Gating Variants:** Power-optimized versions with dynamic gating

1.1.2 Module Categories

1.1.2.1 Core Master/Slave Modules

Module	Description	Documentation	Status
axi4_master_rd	AXI4 read master with buffered channels	axi4_master_rd.md	Documented
axi4_master_wr	AXI4 write master with buffered channels	axi4_master_wr.md	Documented
axi4_slave_rd	AXI4 read slave with buffered channels	axi4_slave_rd.md	Documented
axi4_slave_wr	AXI4 write slave with buffered channels	axi4_slave_wr.md	Documented

1.1.2.2 Monitor Modules (Verification)

Module	Description	Documentation	Status
axi4_master_rd_mon	Read master with integrated monitoring	axi4_master_rd_mon.md	Documented
axi4_master_wr_mon	Write master with integrated monitoring	axi4_master_wr_mon.md	Documented
axi4_slave_rd_mon	Read slave with integrated monitoring	axi4_slave_rd_mon.md	Documented
axi4_slave_wr_mon	Write slave with integrated monitoring	axi4_slave_wr_mon.md	Documented

1.1.2.3 Data Width Converters

Module	Description	Documentation	Status
axi4_dwidth_converter	Bidirectional data width conversion	axi4_dwidth_converter.md	Planned - no RTL
axi4_dwidth_converter_rd	Read-only data width conversion	converters MAS 2.6	Not in rtl/amba — converters component
axi4_dwidth_converter_wr	Write-only data width conversion	converters MAS 2.5	Not in rtl/amba — converters component

The converters live in `projects/components/converters/rtl/`, not `rtl/amba/axi4/`.

1.1.2.4 Clock-Gated Variants

All clock-gated variants documented in: axi4_clock_gating_guide.md

Module	Base Module	Status
axi4_master_rd_cg	axi4_master_rd	Documented
axi4_master_wr_cg	axi4_master_wr	Documented
axi4_slave_rd_cg	axi4_slave_rd	Documented
axi4_slave_wr_cg	axi4_slave_wr	Documented
axi4_master_rd_mon_cg	axi4_master_rd_mon	Documented
axi4_master_wr_mon_cg	axi4_master_wr_mon	Documented
axi4_slave_rd_mon_cg	axi4_slave_rd_mon	Documented
axi4_slave_wr_mon_cg	axi4_slave_wr_mon	Documented

1.2 Functional Description

1.2.1 Channel Architecture

AXI4 uses five independent channels for read and write transactions:

Write Transaction: 1. **AW (Write Address)** - Master → Slave: Address and control 2. **W (Write Data)** - Master → Slave: Data and strobes 3. **B (Write Response)** - Slave → Master: Completion status

Read Transaction: 1. **AR (Read Address)** - Master → Slave: Address and control 2. **R (Read Data)** - Slave → Master: Data and response

1.2.2 Key Signals

Address Channels (AR/AW): - `ARID/AWID` - Transaction ID for reordering - `ARADDR/AWADDR` - Byte address - `ARLEN/AWLEN` - Burst length (1-256 beats) - `ARSIZE/AWSIZE` - Transfer size (1, 2, 4, 8, 16, ... 128 bytes) - `ARBURST/AWBURST` - Burst type (FIXED, INCR, WRAP) - `ARCACHE/AWCACHE` - Cache attributes - `ARPROT/AWPROT` - Protection attributes - `ARQOS/AWQOS` - Quality of Service

Data Channels (R/W): - `RID` - Read data transaction ID - `RDATA/WDATA` - Transfer data - `RRESP/BRESP` - Response (OKAY, EXOKAY, SLVERR, DECERR) - `RLAST/WLAST` - Last transfer in burst - `WSTRB` - Write strobes (byte lane enables)

AXI4 vs AXI3 note: AXI4 removed `WID`. Write data beats are matched to their address by ordering alone, so write bursts from a single master cannot be interleaved. None of the modules here implement or expect `WID`.

Exclusive access: `EXOKAY` appears in the `RRESP`/`BRESP` encodings because it is part of the AXI4 protocol, but no module in this subsystem implements an exclusive monitor. `AxLOCK` and any `EXOKAY` response are carried through untouched; exclusive-access semantics must be provided by the endpoint slave.

1.3 Timing

1.3.1 Throughput

Configuration	Address CH	Data CH	Notes
Single transaction	1 txn/cycle	1 beat/cycle	When buffers not stalled
Burst (len=16)	1 txn/16 cycles	1 beat/cycle	Sustained
Multiple outstanding	Up to MAX_TRANS	1 beat/cycle	Out-of-

Configuration	Address CH	Data CH	Notes
			order

1.3.2 Latency

Path	Cycles	Notes
AR → R (single)	2-3	Minimum read latency
AW → B (single)	2-3	Minimum write latency
Buffer overhead	+1 per channel	Skid buffer latency

1.3.3 Resource Usage

Module Type	LUTs	FFs	BRAM	Notes
Master/Slave (32-bit)	~500	~330	0	Per direction (rd/wr). FFs are the SKID payloads: AR 70 b x 2 + R 44 b x 4 + control
Monitor (+mon)	thousands	>**10,000**	0	At the default `MAX_TRANSACTIONS = 16`. See below – this is NOT a small addition
Clock-gated (+cg)	not measured	**+5**	0	Counted from the RTL: `r_wakeup` (1 FF, `amba_clock_gate_ctrl`) + `r_idle_counter` (`IDLE_CNTR_W`

Module Type	LUTs	FFs	BRAM	Notes
				IDTH, default 4, clock_gate_ctrl). Scales as 1 + CG_IDLE_COUNT_WIDTH. The ICG is a latch/BUFGCE, not a fabric flop

The monitor row deserves the emphasis. It is a structural floor, counted from the RTL rather than estimated, so synthesis can only add to it:

Contributor	FFs	Where
Transaction table	4,560	bus_transaction_t is 285 bits x MAX_TRANSACTIONS = 16
Reporter's local copy	4,560	r_trans_table_local – a second full snapshot
Timeout timers	768	3 phases x 16 slots x 16 bits
Threshold latency	512	16 slots x 32 bits
Floor	**>10,000**	plus the interrupt FIFO and the CAM compare/priority-encoder LUTs

This table previously said "+800 FFs", which is roughly **13x low**. Budget a monitored interface accordingly: four monitors is tens of thousands of flops, not a few thousand. MAX_TRANSACTIONS is the knob – the cost is linear in it, and NUM_BANKS exists because 40 slots did not close timing where 16 did.

1.4 Usage Example

1.4.1 Basic Read Master

```
axi4_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_rd_master (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Frontend interface
    .fub_axi_arid     (cpu_arid),
    .fub_axi_araddr   (cpu_araddr),
    .fub_axi_arlen    (cpu_arlen),
    // ... rest of AR/R signals

    // Master interface
    .m_axi_arid       (m_axi_arid),
    .m_axi_araddr     (m_axi_araddr),
    // ... rest of AR/R signals

    .busy             (rd_busy)
);
```

1.4.2 Basic Write Slave

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_wr_slave (
    .aclk             (axi_clk),
    .aresetn          (axi_resetn),

    // Slave interface (from interconnect)
    .s_axi_awid       (s_axi_awid),
    .s_axi_awaddr     (s_axi_awaddr),
    // ... rest of AW/W/B signals

    // Backend interface (to memory/logic)
    .fub_axi_awid     (mem_awid),
```

```
    .fub_axi_awaddr   (mem_awaddr),
    // ... rest of AW/W/B signals

    .busy             (wr_busy)
);
```

1.4.3 Monitor Integration

```
axi4_master_rd_mon #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(16),
    .ENABLE_FILTERING(1)
) u_rd_mon (
    .aclk                (axi_clk),
    .aresetn             (axi_resetn),

    // AXI interfaces
    .fub_axi_*(frontend_axi_*),
    .m_axi_*(master_axi_*),

    // Monitor configuration
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd1000),

    // Filtering configuration
    .cfg_axi_pkt_mask       (16'b1111_1111_0000_0011),
    .cfg_axi_error_mask     (16'h0000),
    // ... rest of mask signals

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (rd_busy),
    .active_transactions    (rd_active),
```

```
    .error_count            (rd_errors)
);
```

1.5 Design Notes

1.5.1 Design Patterns

1.5.1.1 Pattern 1: Buffered Master/Slave

All core modules use `gaxi_skid_buffer` for elastic buffering: - Decouples frontend and backend timing - Configurable depth per channel - Minimal latency overhead (1 cycle) - Full backpressure handling

1.5.1.2 Pattern 2: Monitor Integration

Monitor modules combine functional core with verification: - Non-invasive monitoring (tap signals) - 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing) - 128-bit standardized monitor bus + 64-bit side-band timestamp (`monitor_common_pkg::monitor_packet_t` / `monbus_timestamp_t`; field layout is documented in the monitor bus group reference) - Real-time error detection

1.5.1.3 Pattern 3: Clock Gating

Clock-gated variants (`*_cg`) add power management: - Dynamic clock gating based on busy signal - Test mode support for scan insertion - Minimal area overhead

1.5.2 ID Width Selection

Choose ID width based on system requirements: - **4 bits (16 IDs):** Single master systems, simple slaves - **8 bits (256 IDs):** Multi-master systems, reordering support - **12+ bits:** Large systems, extensive out-of-order capability

1.5.3 Buffer Depth Guidelines

`SKID_DEPTH_*` is an entry count, not a log2 exponent: `SKID_DEPTH_AR = 2` gives a 2-entry buffer, not 4. The underlying `gaxi_skid_buffer` allocates one register slot per entry and tracks occupancy with a 4-bit counter; legal values are 2..8 inclusive (any integer – odd depths are legal). Values above 8 fail elaboration (the guard errors rather than overflowing); place a `gaxi_fifo_sync` stage ahead of the module when deeper elasticity is required.

Address Channels (AR/AW): - Default: 2 entries - sufficient for most cases - Increase for high-latency address decode or frequent backpressure

Data Channels (R/W): - Default: 4 entries - accommodates moderate bursts - Increase for large bursts (AWLEN/ARLEN > 8) - Use 8 entries for variable latency backends

Response Channel (B): - Default: 2 entries - responses are single-beat - Increase for many concurrent outstanding writes

1.5.4 Burst Optimization

Optimize burst parameters for efficiency:

```
// Cache line burst (64 bytes, 8x 64-bit)
.ARLEN    = 8'd7,          // 8 beats
.ARSIZE   = 3'b011,        // 8 bytes per beat
.ARBURST  = 2'b01          // INCR
```

1.6 Related Modules

1.6.1 RTL Design Sherpa Documentation

- **rtl-amba Overview** - Complete AMBA subsystem architecture
- **APB Modules** - Advanced Peripheral Bus
- **AXIL4 Modules** - AXI4-Lite (simplified protocol)
- **AXIS4 Modules** - AXI4-Stream (streaming data)
- **GAXI Modules** - Generic AXI utilities (buffers, FIFOs)

1.6.2 Configuration and Integration

- **AXI Monitor Configuration Guide** - Monitor setup strategies
- **Monitor Packet Specification** - 128-bit packet format (+ 64-bit side-band timestamp)

1.6.3 Source Code

- RTL: `rtl/amba/axi4/`
- Tests: `val/amba/test_axi4*.py`
- Framework: `bin/TBClasses/components/axi4/`
- Shared Infrastructure: `rtl/amba/shared/` (monitor base, transaction manager)

1.7 Testing

All AXI4 modules are verified using CocoTB-based testbenches:

```
# Run all AXI4 tests
pytest val/amba/test_axi4*.py -v
```

```
# Run specific module tests
pytest val/amba/test_axi4_master_rd.py -v
pytest val/amba/test_axi4_slave_wr.py -v
pytest val/amba/test_axi4_master_rd_mon.py -v

# Run data width converter tests
pytest val/amba/test_axi4_dwidth_converter*.py -v

# Run with waveforms
pytest val/amba/test_axi4_master_rd.py --vcd=waves.vcd -v
```

1.8 References

1.8.1 Protocol Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- ARM IHI 0022D: AMBA AXI and ACE Protocol Specification (AXI3/AXI4/AXI5)

Last Updated: 2025-10-20

1.9 Navigation

- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

2 AXI4 Clock-Gated Variants Guide

Location: `rtl/amba/axi4/*_cg.sv` **Status:** Production Ready

2.1 Overview

Every AXI4 module in this subsystem ships with a clock-gated (_cg) variant that adds power management through dynamic clock gating. The idea is simple: when the interfaces go idle for a configurable stretch, the clock stops, and the dynamic power goes with it. When traffic shows up again, the clock restarts on its own.

2.1.1 Available Clock-Gated Modules

Module	Base Module	Description
`axi4_master_rd_cg`	axi4_master_rd	Clock-gated read master
`axi4_master_wr_cg`	axi4_master_wr	Clock-gated write master
`axi4_slave_rd_cg`	axi4_slave_rd	Clock-gated read slave
`axi4_slave_wr_cg`	axi4_slave_wr	Clock-gated write slave
`axi4_master_rd_mon_cg`	axi4_master_rd_mon	Clock-gated read master with monitoring
`axi4_master_wr_mon_cg`	axi4_master_wr_mon	Clock-gated write master with monitoring
`axi4_slave_rd_mon_cg`	axi4_slave_rd_mon	Clock-gated read slave with monitoring
`axi4_slave_wr_mon_cg`	axi4_slave_wr_mon	Clock-gated write slave with monitoring

2.1.2 Key Features

- **Dynamic Clock Gating:** Automatic clock disable during idle periods
- **Configurable Idle Threshold:** Programmable idle count before gating
- **Functional Equivalence** for the plain _cg wrappers (minus the un-exported `busy`); the _mon_cg wrappers forward every base parameter except `ACTIVE_TRANS_THRESHOLD` (harmless – its inner default is derived from the forwarded `MAX_TRANSACTIONS`) and tie off `debug_block_ready` – see their pages
- **Status Monitoring:** Real-time gating and idle status outputs
- **Test Mode Support:** Bypass capability for scan testing
- **Low Ungating Overhead:** One register stage on the wake-up path; the first usable gated-clock edge arrives 2 cycles after activity is seen

---

2.2 Parameters

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of idle counter

Parameter	Type	Default	Description
			(max idle = 2^N-1 cycles)

All other parameters are identical to the base module.

2.3 Ports

2.3.1 Clock Gating Configuration Inputs

Port	Direction	Width	Description
`cfg_cg_enable`	Input	1	Enable clock gating (0=disabled, 1=enabled)
`cfg_cg_idle_count`	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating; the clock gates `cfg_cg_idle_count` + 1 cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count` + 2 cycles after the last bus activity on AXI4 (one extra for the `r_wakeup` flop)

2.3.2 Clock Gating Status Outputs

Port	Direction	Width	Description
`cg_gating`	Output	1	Clock currently gated (1=gated, 0=running)
`cg_idle`	Output	1	All internal buffers empty (1=idle, 0=active)

The `_cg` wrappers expose gating state only. They do not implement a cumulative gated-cycle counter; accumulate `cg_gating` externally if a power metric is needed.

All other ports are identical to the base module, with two exceptions: the plain `_cg` wrappers do NOT re-export the base `busy` output (it is consumed internally as a wake term), and the `_mon_cg` wrappers tie off `debug_block_ready` rather than forwarding it. Connect either of those and elaboration fails – use the base module when you need them.

2.4 Functional Description

2.4.1 Gating Conditions (All Must Be True)

1. **Interface Idle:** No valid/ready handshakes on any AXI channel
2. **Idle Duration:** Idle for `cfg_cg_idle_count` + 1 consecutive cycles measured from the internal wakeup deassertion, which is `cfg_cg_idle_count` + 2 cycles from the last bus activity
3. **Gating Enabled:** `cfg_cg_enable = 1`

2.4.2 Ungating Conditions (Any Triggers Ungating)

1. Any `*valid` signal asserted on any channel
2. `cfg_cg_enable` deasserted (opens the clock immediately). NOTE: `cfg_cg_idle_count` is NOT a wake trigger – while gated, changing it has no effect until bus activity wakes the block (the counter only reloads on wakeup or !enable)

Ungating Latency: activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The AXI4 `_cg` wrappers drive `user_valid`/`axi_valid` combinationally, so there is exactly **1 register stage** — `r_wakeup` inside `amba_clock_gate_ctrl` — and the first gated-clock rising edge available to the block arrives **2 cycles** after activity asserts.

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> IDLE : activity = 0<br/>(no valid signals)
    IDLE --> RUNNING : wakeup (activity) or !cfg_cg_enable
    IDLE --> GATED : count >= threshold<br/>&& cg_enable
    GATED --> RUNNING : activity detected<br/>(any valid signal)

    state RUNNING {
        note right of RUNNING : Clock running normally
    }

    state IDLE {
        note right of IDLE : Counting idle cycles
    }

    state GATED {
        note right of GATED : Clock stopped
    }
```

2.5 Timing

2.5.1 Ungating Latency

One register stage on the wake-up path; the first usable gated-clock edge arrives 2 cycles after activity asserts. `clock_gate_ctrl` adds no flop of its own — `w_gate_enable` is combinational from `wakeup` into the ICG enable, and its header comment "Wakeup: 1 clock from wakeup assertion to clock restoration" describes the resulting clock edge rather than a register stage.

```
Cycle N:   Interface idle, clock gated. ARVALID asserted; user_valid =
1
           combinationally, ARREADY forced low so no handshake occurs.
Cycle N+1: r_wakeup = 1 -> ICG enable released combinationally,
cg_gating = 0.
Cycle N+2: First usable gated-clock edge; transaction proceeds
normally.
```

2.5.2 Throughput Impact

- Two cycles of added latency on the first transfer out of a gated period
- No penalty on subsequent transfers while the interface stays active
- Functionally equivalent to the non-gated module (AXI handshakes are not dropped, only delayed by the wake-up cycles)

2.5.3 Sustained Throughput

Identical to base module: - Gating only occurs during idle - Active transactions unaffected

---

2.6 Usage Example

2.6.1 Basic Clock Gating (Master Read)

```
axi4_master_rd_cg #(
    // Base parameters
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),

    // Clock gating parameters
    .CG_IDLE_COUNT_WIDTH(4)   // Up to 15 idle cycles
) u_rd_master_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),
```

```
    // Clock gating configuration
    .cfg_cg_enable      (1'b1),      // Enable gating
    .cfg_cg_idle_count  (4'd5),      // Gate after 5 idle cycles

    // AXI signals (identical to base module)
    .fub_axi_arid       (cpu_arid),
    .fub_axi_araddr     (cpu_araddr),
    // ... rest of AR/R signals ...

    .m_axi_arid         (mem_arid),
    .m_axi_araddr       (mem_araddr),
    // ... rest of AR/R signals ...

    // Clock gating status
    .cg_gating          (rd_clk_gated),
    .cg_idle            (rd_idle)
);

// Monitor power savings
always_ff @(posedge axi_clk) begin
    if (rd_clk_gated) begin
        $display("Read master clock gated - saving power");
    end
end
```

2.6.2 Dynamic Configuration (Runtime Adjustment)

```
// Power management controller
logic [3:0] idle_threshold;

always_comb begin
    case (power_mode)
        POWER_HIGH_PERF:  idle_threshold = 4'd15;  // Conservative
gating
        POWER_BALANCED:   idle_threshold = 4'd5;   // Moderate gating
        POWER_LOW:        idle_threshold = 4'd1;   // Aggressive
gating
        default:          idle_threshold = 4'd5;
    endcase
end

axi4_master_wr_cg #(
    // ... parameters ...
) u_wr_master_cg (
    // ... ports ...
```

```
    // Dynamic clock gating control
    .cfg_cg_enable      (power_mgmt_enable),
    .cfg_cg_idle_count  (idle_threshold),

    .cg_gating          (wr_clk_gated),
    .cg_idle            (wr_idle)
);
```

2.6.3 System-Level Power Monitoring

```
// Aggregate power metrics from all interfaces
wire master_rd_gated, master_wr_gated;
wire slave_rd_gated, slave_wr_gated;

// The _cg wrappers do not count gated cycles. Accumulate each
cg_gating
// output locally (see "Measuring Power Savings" below) to obtain
these.
logic [31:0] master_rd_gated_cycles;
logic [31:0] master_wr_gated_cycles;
logic [31:0] slave_rd_gated_cycles;
logic [31:0] slave_wr_gated_cycles;

// Total gated cycles (power savings metric)
wire [31:0] total_gated_cycles = master_rd_gated_cycles +
                                 master_wr_gated_cycles +
                                 slave_rd_gated_cycles +
                                 slave_wr_gated_cycles;

// Instantaneous gating status
wire any_gating = master_rd_gated | master_wr_gated |
                  slave_rd_gated | slave_wr_gated;

// Power savings estimate (assuming 50% power reduction when gated)
wire [31:0] power_savings_percent = (total_gated_cycles * 50) /
total_cycles;
```

2.7 Design Notes

2.7.1 Idle Count Selection

Aggressive Gating (High Idle Time):

```
.cfg_cg_idle_count(4'd1)   // Gate after 1 idle cycle
// Use when: Burst traffic, low duty cycle, max power savings
```

Moderate Gating (Balanced):

```
.cfg_cg_idle_count(4'd5)   // Gate after 5 idle cycles
// Use when: Mixed traffic, balanced power/performance
```

Conservative Gating (Low Latency):

```
.cfg_cg_idle_count(4'd10)  // Gate after 10 idle cycles
// Use when: High throughput, latency-sensitive, minimal power
overhead
```

Disable Gating:

```
.cfg_cg_enable(1'b0)       // Clock gating disabled
// Use when: 100% utilization, ultra-low latency, debug/test
```

2.7.2 When to Use Clock Gating

Recommended for: - Burst traffic patterns (idle periods between bursts) - Low-duty-cycle interfaces (< 50% utilization) - Power-constrained systems (battery, thermal limits) - Variable workloads (idle states common)

Not recommended for: - 100% utilization scenarios (no idle time = no power benefit) - Ultra-low-latency requirements (gating overhead unacceptable) - Continuous streaming (no natural idle points)

2.7.3 Traffic Pattern Analysis

Burst Traffic (Good for Clock Gating):

```
Time:    |----BURST----|idle|----BURST----|idle|----BURST----|
Gating:  |             |GG|               |GG|               |
Savings: ~30-40% depending on burst duty cycle
```

Continuous Traffic (Poor for Clock Gating):

```
Time:    |----DATA----DATA----DATA----DATA----DATA----|
Gating:  |                                             |
Savings: ~0% (never idle long enough to gate)
```

2.7.4 Power Savings Estimates

Illustrative, not measured. No power characterization has been run on this library; `axi4_master_rd_cg.md` says so explicitly. These rows also disagree with the per-module tables at comparable duty cycles (this table: 30 % duty -> 25-40 %; `axi4_master_rd_mon_cg.md`: 25 % utilization -> 45-55 %), which is itself evidence that neither is a measurement. Use them for the trend, never in a power budget.

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
Burst	30%	1	35-40%

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
(aggressive)			
Burst (moderate)	30%	5	30-35%
Burst (conservative)	30%	10	25-30%
Mixed workload	50%	5	20-25%
Low activity	10%	1	40-45%
High activity	80%	5	5-10%
Continuous	100%	any	0%

Note: Actual savings depend on: - Process technology (clock tree power) - Traffic patterns (idle duration distribution) - Configuration (idle count threshold) - Logic behind clock gate (amount of logic gated)

2.7.5 Measuring Power Savings

```
// Calculate power savings percentage
logic [31:0] total_cycles;
logic [31:0] gated_cycles;

always_ff @(posedge axi_clk or negedge aresetn) begin
    if (!aresetn) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (cg_gating) begin
            gated_cycles <= gated_cycles + 1;
        end
    end
end

// Power savings = (gated_cycles / total_cycles) × 100%
assign power_savings_pct = (gated_cycles * 100) / total_cycles;
```

2.7.6 Test Mode Support

For scan testing, disable clock gating:

```
.cfg_cg_enable(~scan_mode)  // Disable during DFT scan
```

2.7.7 Simulation Considerations

Waveform Analysis: - `cg_gating=1`: Clock gated (idle) - `cg_gating=0`: Clock running (active)

Power Estimation:

```
initial begin
    $monitor("Time %0t: Gating=%b, Idle=%b",
             $time, cg_gating, cg_idle);
end
```

2.7.8 Synthesis Considerations

Clock Gating Cell: - Uses standard cell library clock gate (GTECH_AND, ICG, etc.) - Synthesis tool automatically inserts appropriate cell - No vendor-specific primitives required

Timing: - Clock gating logic adds minimal delay - The ICG enable is combinational from `r_wakeup`, so the ungating path must close in a single cycle - Gating path is non-critical

---

2.8 Related Modules

2.8.1 Base Module Documentation

- **axi4_master_rd** - Base read master
- **axi4_master_wr** - Base write master
- **axi4_slave_rd** - Base read slave
- **axi4_slave_wr** - Base write slave
- **axi4_master_rd_mon** - Base master read monitor
- **axi4_master_wr_mon** - Base master write monitor
- **axi4_slave_rd_mon** - Base slave read monitor
- **axi4_slave_wr_mon** - Base slave write monitor

2.8.2 Architecture

- **rtl-amba Overview** - Complete AMBA subsystem
- **AXI4 Index** - AXI4 module index

Last Updated: 2025-10-20

2.9 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

3 axi4_dwidth_converter

Module: `axi4_dwidth_converter.sv` **Location:** Not implemented **Status:** Planned - no RTL in this repository

The read-only and write-only converters DO exist, but they belong to the converters component, not to `rtl/amba/axi4`, and they are documented in its micro-architecture spec: `axi4_dwidth_converter_rd` and `axi4_dwidth_converter_wr`. This book carried a second copy of both until 2026-08-31; a module's docs live with the component that owns the module.

3.1 Overview

> **Implementation status:** There is no `axi4_dwidth_converter.sv` in the repository. This page describes a planned bidirectional converter and its intended parameterization (`AW_FIFO_DEPTH`, `W_FIFO_DEPTH`, `B_FIFO_DEPTH`, `AR_FIFO_DEPTH`, `R_FIFO_DEPTH`); none of those parameters exist in RTL today. The shipping converters are the read-only and write-only variants in `projects/components/converters/rtl/`, which use `SKID_DEPTH_*` skid buffers rather than channel FIFOs:
>
> - axi4_dwidth_converter_rd
> - axi4_dwidth_converter_wr
>
> Instantiate a `_rd` and a `_wr` converter side by side to obtain full-duplex width conversion.

Read the blockquote above before you read anything else on this page — this module is a plan, not silicon. With that said, the design is worth documenting: the AXI4 Data Width Converter provides bidirectional data width conversion between AXI4 interfaces of different data widths while maintaining full protocol compliance. A single parameterized module handles both upsizing (narrow→wide) and downsizing (wide→narrow), automatically recalculating burst parameters and managing data packing/unpacking.

3.1.1 Key Features

- **Bidirectional Conversion:** Single module supports both upsize and downsize
- **Full AXI4 Protocol:** All 5 channels (AW, W, B, AR, R) with complete signal support
- **Burst Preservation:** Maintains burst semantics across width conversion
- **Auto Burst Recalculation:** Adjusts AWLEN/ARLEN and AWSIZE/ARSIZE automatically
- **Error Propagation:** Correctly forwards SLVERR/DECERR responses
- **Elastic Buffering:** Configurable FIFO depths for each channel
- **Status Outputs:** Busy signal and pending transaction counters
- **Parameter Validation:** Compile-time checks for valid configurations

---

3.2 Parameters

3.2.1 Width Configuration

Parameter	Type	Default	Range	Description
`S_AXI_DATA_WIDTH`	int	32	8-1024	Slave interface data width (power of 2)
`M_AXI_DATA_WIDTH`	int	128	8-1024	Master interface data width (power of 2)
`AXI_ID_WIDTH`	int	8	1-16	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	12-64	Address bus width
`AXI_USER_WIDTH`	int	1	0-1024	User signal width

3.2.2 Buffer Depths

Parameter	Type	Default	Description
`AW_FIFO_DEPTH`	int	4	Write address FIFO depth (power of 2)
`W_FIFO_DEPTH`	int	8	Write data FIFO depth (power of 2)

Parameter	Type	Default	Description
B_FIFO_DEPTH	int	4	Write response FIFO depth (power of 2)
AR_FIFO_DEPTH	int	4	Read address FIFO depth (power of 2)
R_FIFO_DEPTH	int	8	Read data FIFO depth (power of 2)

3.2.3 Calculated Parameters (Auto)

Parameter	Calculation	Description
S_STRB_WIDTH	S_AXI_DATA_WIDTH / 8	Slave write strobe width
M_STRB_WIDTH	M_AXI_DATA_WIDTH / 8	Master write strobe width
WIDTH_RATIO	MAX / MIN	Data width ratio (2-16)
UPSIZE	S < M	1 if upsizing, 0 if downsizing
DOWNSIZE	S > M	1 if downsizing, 0 if upsizing
PTR_WIDTH	$clog2(WIDTH_RATIO)	Beat pointer width

3.3 Ports

3.3.1 AXI4 Slave Interface

Write Channels: - AW channel: `s_axi_awid, s_axi_awaddr, s_axi_awlen, s_axi_awsize, s_axi_awburst, s_axi_awlock, s_axi_awcache, s_axi_awprot, s_axi_awqos, s_axi_awregion, s_axi_awuser, s_axi_awvalid, s_axi_awready` - W channel: `s_axi_wdata[S_AXI_DATA_WIDTH], s_axi_wstrb[S_STRB_WIDTH], s_axi_wlast, s_axi_wuser, s_axi_wvalid, s_axi_wready` - B channel: `s_axi_bid, s_axi_bresp, s_axi_buser, s_axi_bvalid, s_axi_bready`

Read Channels: - AR channel: `s_axi_arid, s_axi_araddr, s_axi_arlen, s_axi_arsize, s_axi_arburst, s_axi_arlock, s_axi_arcache, s_axi_arprot, s_axi_arqos, s_axi_arregion, s_axi_aruser, s_axi_arvalid, s_axi_arready` - R channel: `s_axi_rid, s_axi_rdata[S_AXI_DATA_WIDTH], s_axi_rresp, s_axi_rlast, s_axi_ruser, s_axi_rvalid, s_axi_rready`

3.3.2 AXI4 Master Interface

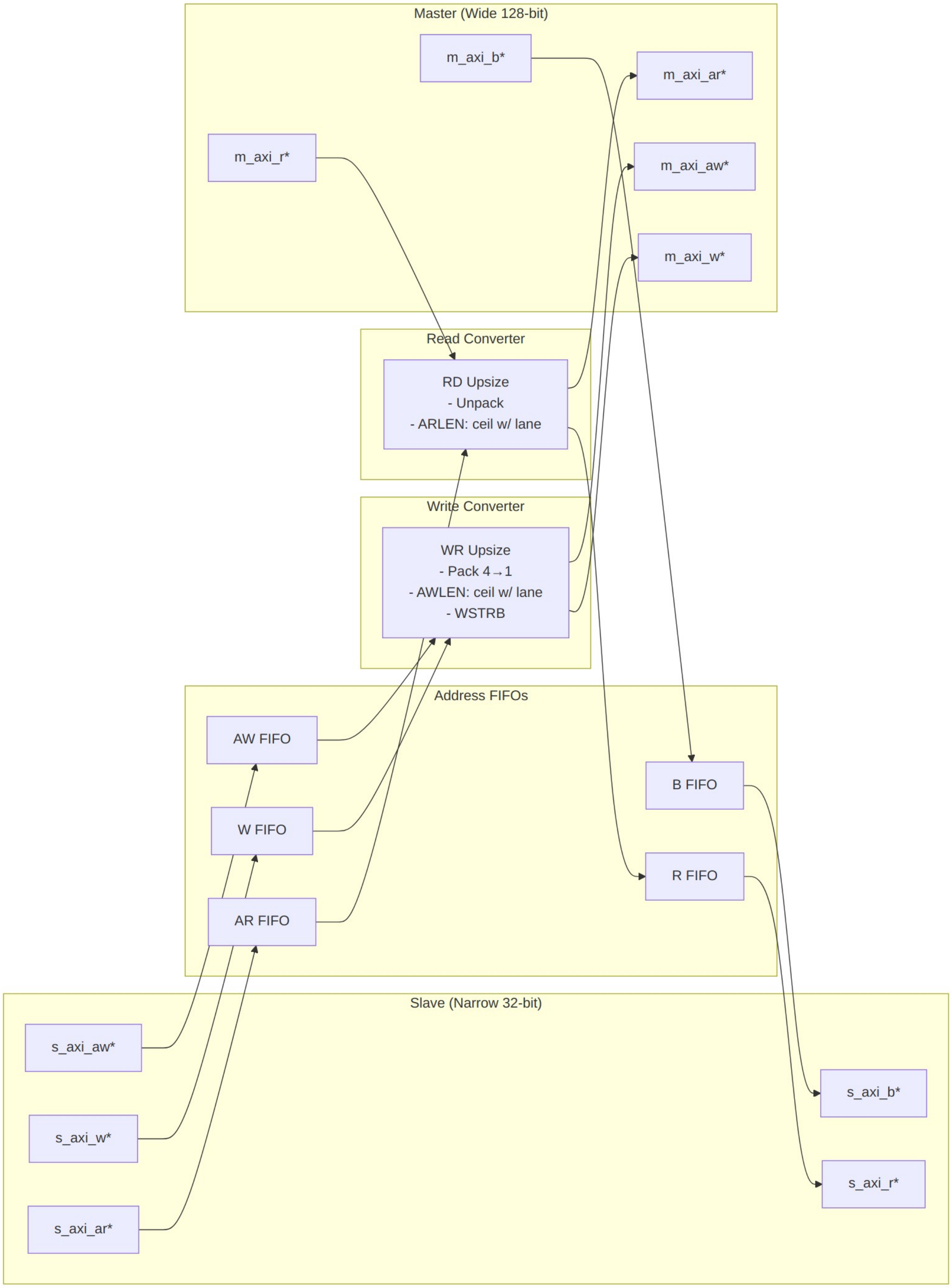
Write Channels: - AW channel: `m_axi_awid, m_axi_awaddr, m_axi_awlen, m_axi_awsize, m_axi_awburst, m_axi_awlock, m_axi_awcache, m_axi_awprot, m_axi_awqos, m_axi_awregion, m_axi_awuser, m_axi_awvalid, m_axi_awready` - W channel: `m_axi_wdata[M_AXI_DATA_WIDTH], m_axi_wstrb[M_STRB_WIDTH], m_axi_wlast, m_axi_wuser, m_axi_wvalid, m_axi_wready` - B channel: `m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid, m_axi_bready`

Read Channels: - AR channel: `m_axi_arid, m_axi_araddr, m_axi_arlen, m_axi_arsize, m_axi_arburst, m_axi_arlock, m_axi_arcache, m_axi_arprot, m_axi_arqos, m_axi_arregion, m_axi_aruser, m_axi_arvalid, m_axi_arready` - R channel: `m_axi_rid, m_axi_rdata[M_AXI_DATA_WIDTH], m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid, m_axi_rready`

3.3.3 Status/Debug Outputs

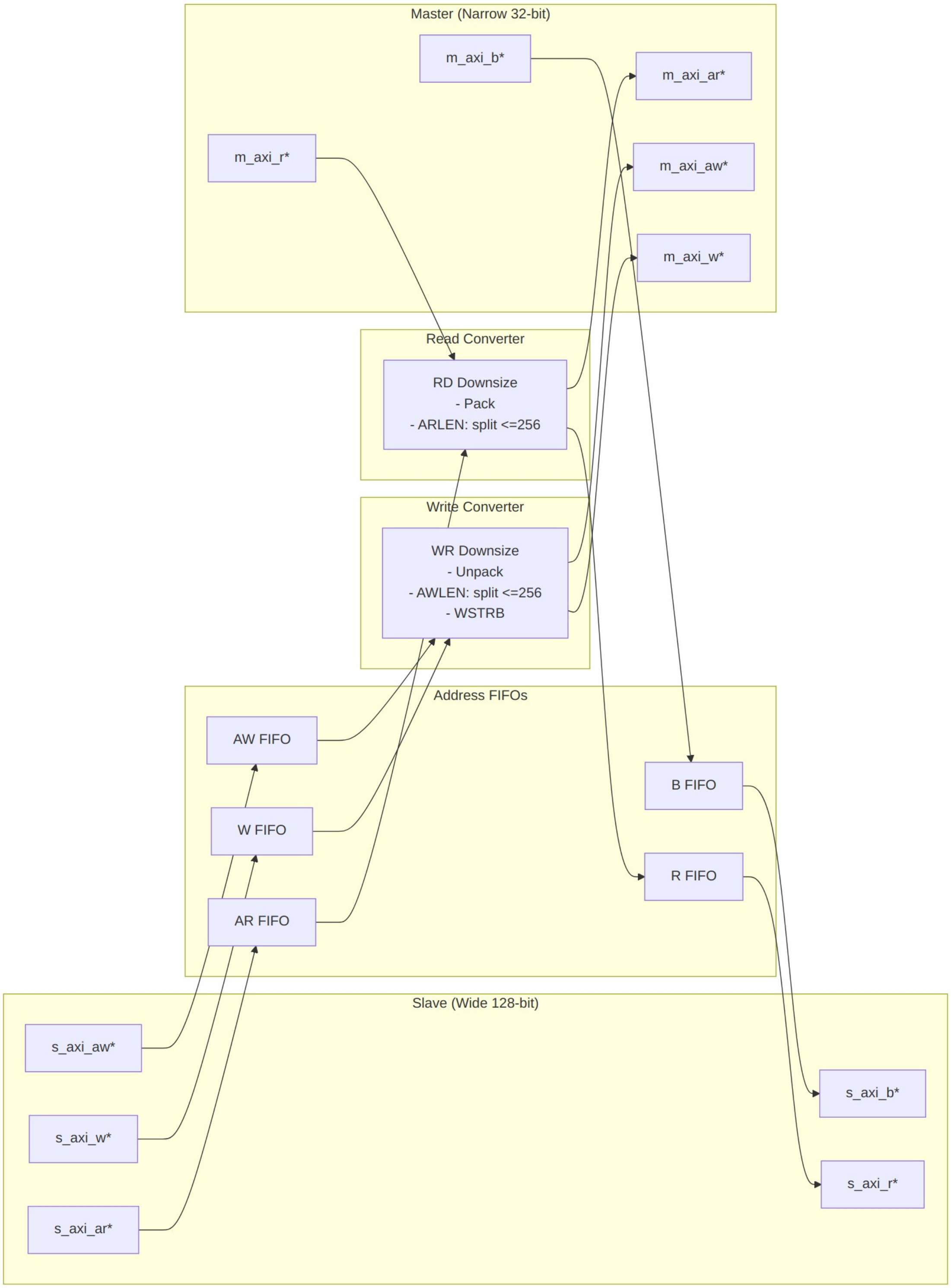
Port	Direction	Width	Description
`busy`	Output	1	Indicates active conversions in progress
`wr_transactions_pending`	Output	16	Number of pending write transactions
`rd_transactions_pending`	Output	16	Number of pending read transactions

3.4 Functional Description



Upsize Mode (Narrow → Wide)

WIDTH_RATIO = **4** (128/32): Burst 16 beats @ 32-bit → 4 beats @ 128-bit



Downsize Mode (Wide → Narrow)

WIDTH_RATIO = 4 (128/32): Burst 4 beats @ 128-bit → 16 beats @ 32-bit

3.4.1 Conversion Mechanics

3.4.1.1 Upsize Conversion (Narrow → Wide)

Write Path:

```
Example: 32-bit → 128-bit (WIDTH_RATIO = 4)

Slave Write:                Master Write:
AWLEN = 15 (16 beats)      AWLEN = 3 (4 beats)
AWSIZE = 3'b010 (4 bytes)  AWSIZE = 3'b100 (16 bytes)
AWADDR = 0x1000            AWADDR = 0x1000 (aligned)

W beats (32-bit):          W beats (128-bit):
Beat 0: wdata[31:0]
Beat 1: wdata[31:0]
Beat 2: wdata[31:0]    Beat 0: {beat3, beat2, beat1, beat0}
Beat 3: wdata[31:0]
Beat 4: wdata[31:0]
Beat 5: wdata[31:0]
Beat 6: wdata[31:0]    Beat 1: {beat7, beat6, beat5, beat4}
Beat 7: wdata[31:0]
... (16 beats total)       ... (4 beats total)
```

Read Path:

```
Slave Read:                Master Read:
ARLEN = 15 (16 beats)      ARLEN = 3 (4 beats)
ARSIZE = 3'b010 (4 bytes)  ARSIZE = 3'b100 (16 bytes)
ARADDR = 0x2000            ARADDR = 0x2000 (aligned)

R beats (128-bit):         R beats (32-bit):
Beat 0: {127:0}  Beat 0: rdata[31:0]   (bits [31:0])
                           Beat 1: rdata[31:0]   (bits [63:32])
                           Beat 2: rdata[31:0]   (bits [95:64])
                           Beat 3: rdata[31:0]   (bits [127:96])
Beat 1: {127:0}  Beat 4-7: ...
... (4 beats total)        ... (16 beats total)
```

3.4.1.2 Downsize Conversion (Wide → Narrow)

Write Path:

```
Example: 128-bit → 32-bit (WIDTH_RATIO = 4)

Slave Write:               Master Write:
AWLEN = 3 (4 beats)        AWLEN = 15 (16 beats)
```

```
AWSIZE = 3'b100 (16 bytes) AWSIZE = 3'b010 (4 bytes)
AWADDR = 0x3000            AWADDR = 0x3000 (aligned)

W beats (128-bit):         W beats (32-bit):
Beat 0: wdata[127:0]  Beat 0: [31:0]
                           Beat 1: [63:32]
                           Beat 2: [95:64]
                           Beat 3: [127:96]
Beat 1: wdata[127:0]  Beat 4-7: ...
... (4 beats total)        ... (16 beats total)
```

Read Path:

```
Slave Read:                Master Read:
ARLEN = 3 (4 beats)        ARLEN = 15 (16 beats)
ARSIZE = 3'b100 (16 bytes) ARSIZE = 3'b010 (4 bytes)
ARADDR = 0x4000            ARADDR = 0x4000 (aligned)

R beats (32-bit):          R beats (128-bit):
Beat 0: rdata[31:0]
Beat 1: rdata[31:0]
Beat 2: rdata[31:0]    Beat 0: {beat3, beat2, beat1, beat0}
Beat 3: rdata[31:0]
... (16 beats total)       ... (4 beats total)
```

3.4.1.3 WSTRB Handling

Upsize: Packs narrow WSTRB beats into wide WSTRB

```
32-bit WSTRB:    4'b1111, 4'b1111, 4'b1111, 4'b1111
                    ↓        ↓        ↓        ↓
128-bit WSTRB:   16'b1111_1111_1111_1111
```

Downsize: Unpacks wide WSTRB into narrow WSTRB beats

```
128-bit WSTRB:   16'b1111_0000_1111_0000
                    ↓
32-bit WSTRB:    4'b0000, 4'b1111, 4'b0000, 4'b1111
```

3.5 Timing Characteristics

3.5.1 Performance Characteristics

Throughput (Upsize): - Accumulation latency: WIDTH_RATIO-1 cycles per wide beat - Example (WIDTH_RATIO=4): 3-cycle latency to accumulate first wide beat

Throughput (Downsize): - Unpacking latency: 1 cycle per narrow beat - Full wide beat can be unpacked in consecutive cycles

Backpressure Handling: - FIFOs provide elastic buffering - Prevents deadlock during width mismatch scenarios - Slave can stall independently of master

---

3.6 Usage Examples

3.6.1 Basic Upsize Converter (32-bit → 128-bit)

```
axi4_dwidth_converter #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (32),     // Narrow slave
    .M_AXI_DATA_WIDTH   (128),    // Wide master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_USER_WIDTH     (1),

    // Buffer depths
    .AW_FIFO_DEPTH      (4),
    .W_FIFO_DEPTH       (16),     // Deeper for burst accumulation
    .B_FIFO_DEPTH       (4),
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (16)      // Deeper for burst unpacking
) u_upsize_conv (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (narrow) interface
    .s_axi_awid         (narrow_awid),
    .s_axi_awaddr       (narrow_awaddr),
    .s_axi_awlen        (narrow_awlen),     // e.g., 15 (16 beats)
    .s_axi_awsize       (narrow_awsize),    // e.g., 3'b010 (4 bytes)
    .s_axi_awvalid      (narrow_awvalid),
    .s_axi_awready      (narrow_awready),
    // ... rest of slave AW/W/B/AR/R signals

    // Master (wide) interface
    .m_axi_awid         (wide_awid),
    .m_axi_awaddr       (wide_awaddr),
    .m_axi_awlen        (wide_awlen),       // e.g., 3 (4 beats)
    .m_axi_awsize       (wide_awsize),      // e.g., 3'b100 (16 bytes)
    .m_axi_awvalid      (wide_awvalid),
    .m_axi_awready      (wide_awready),
    // ... rest of master AW/W/B/AR/R signals
```

```
    // Status
    .busy                   (conv_busy),
    .wr_transactions_pending(wr_pend),
    .rd_transactions_pending(rd_pend)
);

// Typical upsize scenario: CPU (32-bit) → Memory controller (128-bit)
```

3.6.2 Basic Downsize Converter (128-bit → 32-bit)

```
axi4_dwidth_converter #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (128),    // Wide slave
    .M_AXI_DATA_WIDTH   (32),     // Narrow master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),

    // Buffer depths
    .AW_FIFO_DEPTH      (4),
    .W_FIFO_DEPTH       (8),      // Moderate depth for unpacking
    .B_FIFO_DEPTH       (4),
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (32)      // Deeper for burst packing
) u_downsize_conv (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (wide) interface
    .s_axi_awid         (wide_awid),
    .s_axi_awaddr       (wide_awaddr),
    .s_axi_awlen        (wide_awlen),       // e.g., 3 (4 beats)
    .s_axi_awsize       (wide_awsize),      // e.g., 3'b100 (16 bytes)
    // ... rest of slave signals

    // Master (narrow) interface
    .m_axi_awid         (narrow_awid),
    .m_axi_awaddr       (narrow_awaddr),
    .m_axi_awlen        (narrow_awlen),     // e.g., 15 (16 beats)
    .m_axi_awsize       (narrow_awsize),    // e.g., 3'b010 (4 bytes)
    // ... rest of master signals

    // Status
    .busy                   (conv_busy),
    .wr_transactions_pending(wr_pend),
    .rd_transactions_pending(rd_pend)
);
```

```
// Typical downsize scenario: Interconnect (128-bit) → Peripheral (32-bit)
```

3.7 Design Notes

3.7.1 WIDTH_RATIO Constraints

Supported Ratios: 2, 4, 8, 16 - WIDTH_RATIO = 2: e.g., 32→64, 64→128, 128→256 - WIDTH_RATIO = 4: e.g., 32→128, 64→256 - WIDTH_RATIO = 8: e.g., 32→256, 64→512 - WIDTH_RATIO = 16: e.g., 32→512, 64→1024

Why Limited to 16? - Larger ratios require excessive buffering (512+ beats) - Address alignment becomes complex - Performance degrades significantly - Rare in real designs (use multi-stage conversion instead)

3.7.2 Address Alignment

Critical: Slave addresses must be aligned to master data width in upsize mode

```
// Upsize 32→128 (WIDTH_RATIO=4, M_STRB_WIDTH=16)
// Slave addresses must be 16-byte aligned (bottom 4 bits = 0)

VALID:   AWADDR = 0x1000 (aligned to 16 bytes)
VALID:   AWADDR = 0x2000
INVALID: AWADDR = 0x1004 (not 16-byte aligned)
INVALID: AWADDR = 0x2008 (not 16-byte aligned)
```

Downsize: No alignment restrictions (master can access any byte)

3.7.3 Burst Length Calculation

Upsize: the start address matters. A burst that begins part-way into a wide word fills fewer narrow lanes in its first beat, so the lane offset is part of the count – omit it and an unaligned burst is short by one beat.

```
lane          = AWADDR[clog2(M_STRB_WIDTH)-1 : 0] >> clog2(S_STRB_WIDTH)
Master AWLEN  = ceil((lane + Slave AWLEN + WIDTH_RATIO) / WIDTH_RATIO) - 1
Master AWSIZE = $clog2(M_STRB_WIDTH)          // the full wide size, not a delta

Example: 32->128 (WIDTH_RATIO=4), aligned
Slave:  AWLEN=15, AWSIZE=2 (4 bytes)  -> 16 beats x 4 bytes = 64 bytes
Master: AWLEN=3,  AWSIZE=4 (16 bytes) -> 4 beats x 16 bytes = 64 bytes
```

```
Same burst starting at lane 2 (AWADDR = 0x8):
Master: AWLEN=4  -- five wide beats, the first and last partial
```

`axi4_dwidth_converter_wr.sv` computes this on a 10-bit intermediate so that `lane + 255 + WIDTH_RATIO` cannot wrap before the divide.

Downsize: there is no single-burst formula, because there cannot be one. One wide beat becomes `WIDTH_RATIO` narrow beats, so a full 256-beat slave burst would need up to `256 * WIDTH_RATIO` narrow beats – not expressible in an 8-bit AWLEN and not legal AXI4.

```
Master AWSIZE = $clog2(M_STRB_WIDTH)
Master AWLEN  = one burst per <= 256 narrow beats, issued back to back
```

The converter therefore **splits** one slave burst into as many master bursts of <= 256 beats as it takes, recording each in a split queue so the W framing and the B fold stay consistent. WRAP never reaches this path: AXI4 caps WRAP at 16 beats, and `16 * WIDTH_RATIO <= 256` for every supported ratio.

Ordering constraint on the split fold. The split records live in a single FIFO (`splitq_*`), and the B fold pops one entry per downstream response, forwarding a B to the slave only on the record marked final. That is correct while downstream B responses arrive in the order the split AWs were issued – guaranteed by AXI4 within one ID, since the pieces of a split burst all carry the AWID of the burst they came from. It is NOT guaranteed across IDs: AXI4 permits a downstream to return B responses for different IDs in any order, and a single FIFO cannot tell them apart, so an interleaved response would be folded against the wrong record. Drive this converter from a single ID, or from a master that does not interleave write responses, until the fold is made ID-aware (tracked as CONV-001).

> The naive `(Slave AWLEN + 1) * WIDTH_RATIO - 1` this section used to give is the bug the split logic replaced: computed into the 8-bit field it wrapped, turning 511 into 255 (half the burst silently lost), and at 4:1 a 128-beat slave burst asked for 515 beats and got 3.

3.7.4 Buffer Depth Guidelines

Address Channels (AW/AR): - Default: 4 entries (sufficient for most cases) - Increase if high address command rate

Data Channels (W/R): - **Upsize:** Deeper buffers needed to accumulate narrow beats - W_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - **Downsize:** Moderate buffers for unpacking wide beats - W_FIFO_DEPTH ≥ max_burst_length (wide side) - R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length (for packing)

Example (32→128, max burst=16 narrow beats):

```
.W_FIFO_DEPTH  (16),  // Accumulate 16 narrow beats → 4 wide beats
.R_FIFO_DEPTH  (16)   // Unpack 4 wide beats → 16 narrow beats
```

3.8 Related Modules

3.8.1 Specialized Converters

- **axi4_dwidth_converter_rd** - Read-only data width conversion
- **axi4_dwidth_converter_wr** - Write-only data width conversion

3.8.2 Core Modules

- **axi4_master_rd** - AXI4 read master
- **axi4_master_wr** - AXI4 write master
- **axi4_slave_rd** - AXI4 read slave
- **axi4_slave_wr** - AXI4 write slave

3.8.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **gaxi_fifo_sync** - Synchronous FIFOs

3.9 References

3.9.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Chapter 11: Data Width Conversion

3.9.2 Source Code

- RTL: none – this page describes a PLANNED combined converter; the real modules are `projects/components/converters/rtl/axi4_dwidth_converter_{rd,wr}.sv`
- Tests: `projects/components/converters/dv/tests/test_axi4_dwidth_converter_wr.py`
- Framework: `bin/TBClasses/components/axi4/`

3.9.3 Documentation

- Architecture: rtl-amba Overview
- AXI4 Index: README.md

Last Updated: 2025-10-20

3.10 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

4 axi4_master_rd

An AXI4 read master module that provides high-performance read transaction processing with dual skid buffer architecture for optimal throughput and pipeline efficiency in AXI4 memory-mapped systems.

4.1 Overview

The `axi4_master_rd` module implements a complete AXI4 read master interface with integrated address and data channel buffering using GAXI skid buffers. It acts as a bridge between upstream AXI4 read requestors and downstream AXI4 memory interfaces, providing transaction buffering, pipeline optimization, and flow control to maximize memory subsystem performance. It's a complete, high-performance read-master solution: advanced buffering, full protocol compliance, and the system integration hooks (busy status for clock gating, full sideband support) you'd expect.

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width

Parameter	Type	Default	Description
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)

4.3 Ports

4.3.1 Module Declaration

```
module axi4_master_rd #(
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Short and calculated params
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int SW       = AXI_WSTRB_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int ARSize   = IW+AW+8+3+2+1+4+3+4+4+UW,
    parameter int RSize    = IW+DW+2+1+UW
) (
    // Global Clock and Reset
    input  logic                       aclk,
    input  logic                       aresetn,

    // Slave AXI Interface (Input Side) - FUB
    // Read address channel (AR)
    input  logic [IW-1:0]              fub_axi_arid,
    input  logic [AW-1:0]              fub_axi_araddr,
    input  logic [7:0]                 fub_axi_arlen,
    input  logic [2:0]                 fub_axi_arsize,
    input  logic [1:0]                 fub_axi_arburst,
    input  logic                       fub_axi_arlock,
    input  logic [3:0]                 fub_axi_arcache,
    input  logic [2:0]                 fub_axi_arprot,
    input  logic [3:0]                 fub_axi_arqos,
    input  logic [3:0]                 fub_axi_arregion,
    input  logic [UW-1:0]              fub_axi_aruser,
    input  logic                       fub_axi_arvalid,
    output logic                       fub_axi_arready,
```

```
    // Read data channel (R)
    output logic [IW-1:0]                  fub_axi_rid,
    output logic [DW-1:0]                  fub_axi_rdata,
    output logic [1:0]                     fub_axi_rresp,
    output logic                           fub_axi_rlast,
    output logic [UW-1:0]                  fub_axi_ruser,
    output logic                           fub_axi_rvalid,
    input  logic                           fub_axi_rready,

    // Master AXI Interface (Output Side)
    // Read address channel (AR)
    output logic [IW-1:0]                  m_axi_arid,
    output logic [AW-1:0]                  m_axi_araddr,
    output logic [7:0]                     m_axi_arlen,
    output logic [2:0]                     m_axi_arsize,
    output logic [1:0]                     m_axi_arburst,
    output logic                           m_axi_arlock,
    output logic [3:0]                     m_axi_arcache,
    output logic [2:0]                     m_axi_arprot,
    output logic [3:0]                     m_axi_arqos,
    output logic [3:0]                     m_axi_arregion,
    output logic [UW-1:0]                  m_axi_aruser,
    output logic                           m_axi_arvalid,
    input  logic                           m_axi_arready,

    // Read data channel (R)
    input  logic [IW-1:0]                  m_axi_rid,
    input  logic [DW-1:0]                  m_axi_rdata,
    input  logic [1:0]                     m_axi_rresp,
    input  logic                           m_axi_rlast,
    input  logic [UW-1:0]                  m_axi_ruser,
    input  logic                           m_axi_rvalid,
    output logic                           m_axi_rready,

    // Status outputs for clock gating
    output logic                           busy
);
```

4.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

4.3.3 Slave AXI Interface (Input Side - FUB)

4.3.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Input	Read address ID
fub_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
fub_axi_arlen	8	Input	Burst length, AXI-encoded: beats - 1 (`0x00` = 1 beat, `0xFF` = 256)
fub_axi_arsize	3	Input	Transfer size (bytes per beat)
fub_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Input	Lock type (atomic access)
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_arregion	4	Input	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_arvalid	1	Input	Read address valid
fub_axi_arready	1	Output	Read address ready

4.3.3.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WID	Output	Read data ID

Port	Width	Direction	Description
	TH		
fub_axi_rdata	AXI_DATA_WIDTH	Output	Read data
fub_axi_rresp	2	Output	Read response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_rlast	1	Output	Last read transfer in burst
fub_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

4.3.4 Master AXI Interface (Output Side)

4.3.4.1 Read Address Channel (AR)

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length, AXI-encoded: beats - 1 (`0x00` = 1 beat, `0xFF` = 256)
m_axi_arsize	3	Output	Transfer size (bytes per beat)
m_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)
m_axi_arlock	1	Output	Lock type (atomic access)
m_axi_arcache	4	Output	Cache attributes

Port	Width	Direction	Description
m_axi_arprot	3	Output	Protection attributes
m_axi_arqos	4	Output	Quality of Service
m_axi_arregion	4	Output	Region identifier
m_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready

4.3.4.2 Read Data Channel (R)

Port	Width	Direction	Description
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	2	Input	Read response (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	1	Input	Last read transfer in burst
m_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
m_axi_rvalid	1	Input	Read data valid
m_axi_rready	1	Output	Read data ready

4.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module

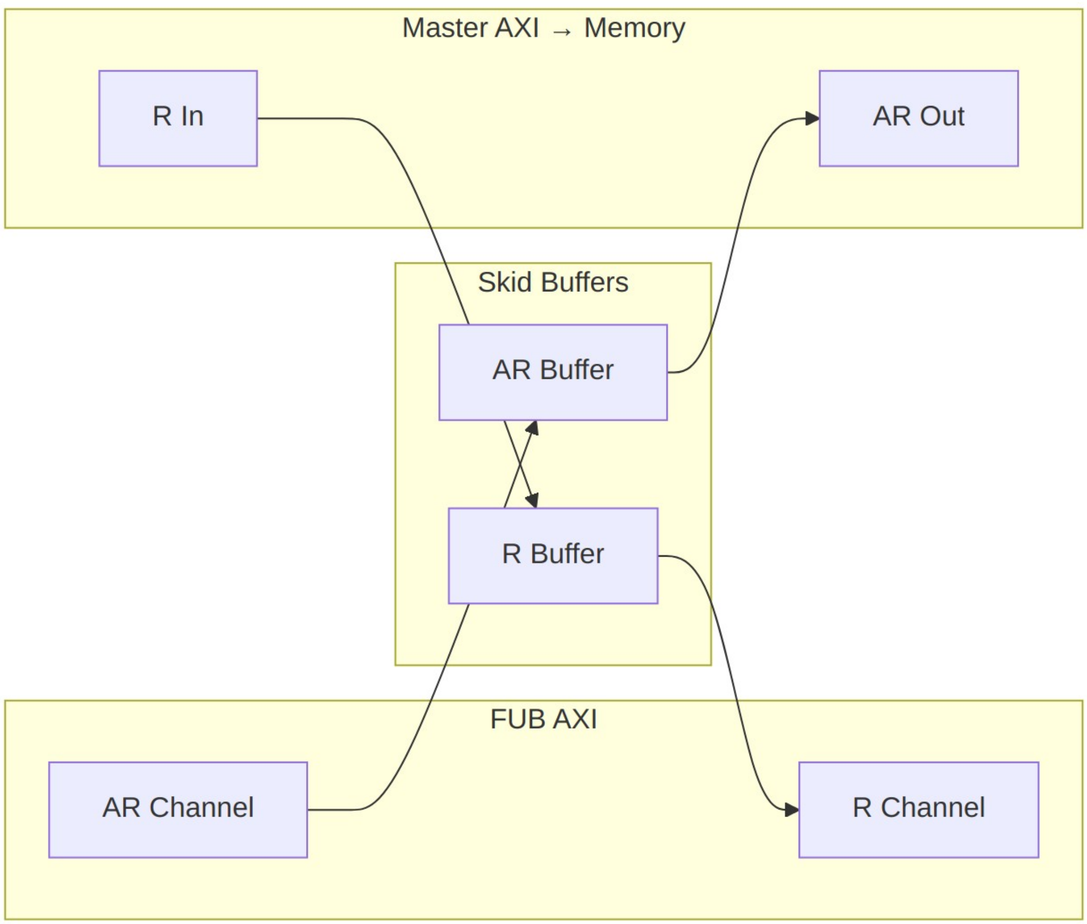
Port	Width	Direction	Description
			activity indicator for clock gating

4.4 Functional Description

4.4.1 Dual Buffer Design

The module employs a dual skid buffer architecture:

1. **AR Channel Buffer**: Buffers read address transactions
2. **R Channel Buffer**: Buffers read data responses



Data Flow

4.4.2 Key Features

- **Pipeline Optimization**: Dual buffering eliminates pipeline stalls
- **AXI4 Compliance**: Full AXI4 protocol support including all signals
- **Flow Control**: Independent buffering for address and data channels
- **Clock Gating Support**: Busy signal for power optimization
- **Burst Support**: Full burst transaction handling (1-256 beats)
- **ID Preservation**: Transaction ID forwarding (no matching – IDs pass through unchecked, as the ID handling section states)
- **Error Handling**: Proper error response propagation

4.4.3 Address Channel Processing

1. **Address Capture**: Incoming address transactions buffered in AR skid buffer
2. **Address Forward**: Buffered addresses forwarded to master interface
3. **Flow Control**: Ready/valid handshaking manages buffer flow

4.4.4 Data Channel Processing

1. **Data Reception**: Read data from master interface buffered in R skid buffer
2. **Data Forward**: Buffered data forwarded to FUB interface
3. **ID Pass-Through**: `RID` is carried through the R skid buffer as ordinary payload bits. The module does not maintain a transaction table and performs no ID matching, reordering, or outstanding-transaction tracking; ordering is whatever the downstream slave produces.

4.4.5 Busy Signal Generation

The busy output indicates module activity:

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
                fub_axi_arvalid || m_axi_rvalid;
```

4.5 Timing Characteristics

4.5.1 Buffer Latency

Buffer	Default Depth	Latency	Description
AR Channel	2 entries (SKID_DEPTH_AR=2)	1 cycle	Address buffering
R Channel	4 entries	1 cycle	Data buffering

Buffer	Default Depth	Latency	Description
	(SKID_DEPTH_R=4)		

4.5.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	300-600 MHz	Technology dependent
Address Throughput	1 transaction/cycle	When buffers not full
Data Throughput	1 beat/cycle	When buffers not empty
Pipeline Efficiency	95-99%	With appropriate buffer sizing

4.5.3 Transaction Latency

Transaction Type	Latency	Description
Single Read	2-3 cycles	Address → Data (minimum)
Burst Read	2+N cycles	Address + N data beats
Back-to-back	1 cycle/txn	Sustained rate with buffering

4.6 Usage Examples

4.6.1 Basic AXI4 Read Master Configuration

```
axi4_master_rd #(
    .SKID_DEPTH_AR(2),        // 2-entry address buffer
    .SKID_DEPTH_R(4),         // 4-entry data buffer
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi4_rd_master (
    .aclk             (axi_clk),
    .aresetn          (axi_resetn),

    // Slave interface (from CPU/DMA)
    .fub_axi_arid       (cpu_axi_arid),
```

```
    .fub_axi_araddr    (cpu_axi_araddr),
    .fub_axi_arlen     (cpu_axi_arlen),
    .fub_axi_arsize    (cpu_axi_arsize),
    .fub_axi_arburst   (cpu_axi_arburst),
    .fub_axi_arlock    (cpu_axi_arlock),
    .fub_axi_arcache   (cpu_axi_arcache),
    .fub_axi_arprot    (cpu_axi_arprot),
    .fub_axi_arqos     (cpu_axi_arqos),
    .fub_axi_arregion  (cpu_axi_arregion),
    .fub_axi_aruser    (cpu_axi_aruser),
    .fub_axi_arvalid   (cpu_axi_arvalid),
    .fub_axi_arready   (cpu_axi_arready),

    .fub_axi_rid       (cpu_axi_rid),
    .fub_axi_rdata     (cpu_axi_rdata),
    .fub_axi_rresp     (cpu_axi_rresp),
    .fub_axi_rlast     (cpu_axi_rlast),
    .fub_axi_ruser     (cpu_axi_ruser),
    .fub_axi_rvalid    (cpu_axi_rvalid),
    .fub_axi_rready    (cpu_axi_rready),

    // Master interface (to memory)
    .m_axi_arid        (mem_axi_arid),
    .m_axi_araddr      (mem_axi_araddr),
    .m_axi_arlen       (mem_axi_arlen),
    .m_axi_arsize      (mem_axi_arsize),
    .m_axi_arburst     (mem_axi_arburst),
    .m_axi_arlock      (mem_axi_arlock),
    .m_axi_arcache     (mem_axi_arcache),
    .m_axi_arprot      (mem_axi_arprot),
    .m_axi_arqos       (mem_axi_arqos),
    .m_axi_arregion    (mem_axi_arregion),
    .m_axi_aruser      (mem_axi_aruser),
    .m_axi_arvalid     (mem_axi_arvalid),
    .m_axi_arready     (mem_axi_arready),

    .m_axi_rid         (mem_axi_rid),
    .m_axi_rdata       (mem_axi_rdata),
    .m_axi_rresp       (mem_axi_rresp),
    .m_axi_rlast       (mem_axi_rlast),
    .m_axi_ruser       (mem_axi_ruser),
    .m_axi_rvalid      (mem_axi_rvalid),
    .m_axi_rready      (mem_axi_rready),

    // Status for clock gating
    .busy              (rd_master_busy)
);
```

4.6.2 High-Performance Memory Interface

```
// High-performance configuration for DDR interface
axi4_master_rd #(
    .SKID_DEPTH_AR(4),        // 4-entry address buffer
    .SKID_DEPTH_R(8),         // 8-entry data buffer
    .AXI_ID_WIDTH(4),         // Reduced ID width for efficiency
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(128),     // Wide data path
    .AXI_USER_WIDTH(1)
) u_ddr_rd_master (
    .aclk            (ddr_clk),
    .aresetn         (ddr_resetn),

    // Connect to multiple read requestors via AXI interconnect
    .fub_axi_*(cpu_cluster_axi_*),

    // Connect to DDR controller
    .m_axi_*(ddr_ctrl_axi_*),

    .busy            (ddr_rd_busy)
);
```

4.6.3 Multi-Master System Integration

```
module axi_memory_system (
    input logic axi_clk,
    input logic axi_resetn,

    // CPU interface
    axi4_if.slave   cpu_axi,
    // DMA interface
    axi4_if.slave   dma_axi,
    // Memory interface
    axi4_if.master  mem_axi
);

    // Read masters for independent buffering
    axi4_master_rd #(
        .SKID_DEPTH_AR(2),
        .SKID_DEPTH_R(4),
        .AXI_ID_WIDTH(4),
        .AXI_DATA_WIDTH(64)
    ) u_cpu_rd_master (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .fub_axi_*(cpu_axi.*),
        .m_axi_*(cpu_rd_axi.*),
```

```
        .busy(cpu_rd_busy)
    );

    axi4_master_rd #(
        .SKID_DEPTH_AR(4),
        .SKID_DEPTH_R(6),
        .AXI_ID_WIDTH(4),
        .AXI_DATA_WIDTH(64)
    ) u_dma_rd_master (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .fub_axi_*(dma_axi.*),
        .m_axi_*(dma_rd_axi.*),
        .busy(dma_rd_busy)
    );

    // AXI interconnect for arbitration
    axi4_interconnect #(
        .NUM_MASTERS(2),
        .NUM_SLAVES(1)
    ) u_interconnect (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .s_axi({cpu_rd_axi, dma_rd_axi}),
        .m_axi(mem_axi)
    );

endmodule
```

4.6.4 Clock Gating Integration

```
// Clock gated version for power optimization
axi4_master_rd_cg #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_DATA_WIDTH(32)
) u_rd_master_cg (
    .aclk            (axi_clk),
    .aresetn         (axi_resetn),

    // Standard AXI interfaces
    /* ... fub_axi_* and m_axi_* ports ... */

    // Clock gating control (runtime inputs; scan bypass =
cfg_cg_enable=0)
    .cfg_cg_enable    (rd_enable && !scan_mode),
    .cfg_cg_idle_count(4'd8),
    .cg_gating        (rd_cg_gating),
```

```
    .cg_idle          (rd_cg_idle)
);
// NOTE: the _cg wrapper does not re-export the base module's busy output;
// use cg_idle for system-level power management instead.
always_ff @(posedge axi_clk) begin
    rd_power_gate <= rd_cg_idle && idle_counter > IDLE_THRESHOLD;
end
```

4.7 Design Notes

4.7.1 Buffer Depth Selection

Choose buffer depths based on system characteristics:

`SKID_DEPTH_*` is an entry count, not a log2 exponent. The underlying `gaxi_skid_buffer` stores one register slot per entry and tracks occupancy in a 4-bit counter, so legal values are 2..8 inclusive (any integer – odd depths are legal). Values above 8 fail elaboration (the guard errors rather than overflowing); for deeper elasticity use a `gaxi_fifo_sync` stage ahead of the module instead.

System Type	SKID_DEPTH_AR	SKID_DEPTH_R	Rationale
Low Latency CPU	2	2	Minimize latency
High Throughput DMA	4	8	Maximize bandwidth
DDR4 Interface	4	8	Handle refresh cycles
PCIe Interface	4	6	Variable latency tolerance

4.7.2 Burst Optimization

```
// Optimize for burst efficiency
localparam BURST_LENGTH = 16; // Optimize for cache line size

// Generate efficient burst addresses
always_comb begin
    axi_araddr = base_addr & ~(BURST_LENGTH * DATA_BYTES - 1); // Align
    axi_arlen  = BURST_LENGTH - 1;                             // Length
```

```
    axi_arsize = $clog2(DATA_BYTES);                          // Size
    axi_arburst = 2'b01;                                      // INCR
end
```

4.7.3 Quality of Service (QoS)

```
// QoS-aware read master configuration
always_comb begin
    case (requestor_type)
        CPU_CRITICAL:  axi_arqos = 4'hF;    // Highest priority
        CPU_NORMAL:    axi_arqos = 4'h8;    // Medium priority
        DMA_BULK:      axi_arqos = 4'h4;    // Lower priority
        BACKGROUND:    axi_arqos = 4'h1;    // Lowest priority
        default:       axi_arqos = 4'h0;
    endcase
end
```

4.7.4 Transaction Monitoring

```
// Performance monitoring integration
logic [31:0] read_transaction_count;
logic [31:0] read_burst_total;
logic [31:0] read_latency_accumulator;

always_ff @(posedge axi_clk) begin
    // Count transactions
    if (fub_axi_arvalid && fub_axi_arready) begin
        read_transaction_count <= read_transaction_count + 1;
        read_burst_total <= read_burst_total + fub_axi_arlen + 1;
    end

    // Measure latency (simplified)
    if (fub_axi_rvalid && fub_axi_rready && fub_axi_rlast) begin
        read_latency_accumulator <= read_latency_accumulator +
measured_latency;
    end
end

// Average burst length and latency calculations
assign avg_burst_length = read_burst_total / read_transaction_count;
assign avg_latency = read_latency_accumulator /
read_transaction_count;
```

4.7.5 Error Handling and Recovery

```
// Enhanced error handling
logic error_detected;
logic [7:0] error_count;
```

```
always_ff @(posedge axi_clk) begin
    error_detected <= fub_axi_rvalid && fub_axi_rready &&
                     (fub_axi_rresp == 2'b10 || fub_axi_rresp ==
2'b11);

    if (error_detected) begin
        error_count <= error_count + 1;
        // Log error details for debugging
        error_log <= {fub_axi_rid, fub_axi_rresp, timestamp};
    end
end
```

4.7.6 Synthesis Considerations

Area Optimization: - Reduce buffer depths for area-constrained designs - Use smaller ID widths when possible - Share buffers across multiple masters if timing allows

Timing Optimization: - Register all interface signals for timing closure - Use appropriate buffer depths to break critical paths - Consider pipeline stages for very high-frequency operation

Power Optimization: - Use clock gating variant (`axi4_master_rd_cg`) when available - Implement QoS-based power scaling - Size buffers appropriately to minimize switching activity

4.8 Related Modules

- **axi4_master_rd_cg**: Clock-gated version for power optimization
- **axi4_master_wr**: Complementary AXI4 write master
- **gaxi_skid_buffer**: Underlying buffer infrastructure
- **axi4_interconnect**: AXI4 interconnect for multi-master systems
- **axi_monitor_base**: AXI4 protocol monitoring and analysis

4.9 Testing

4.9.1 Protocol Compliance

- Verify AXI4 handshaking on all channels
- Check burst handling and RLAST generation
- Validate ID preservation through the pipeline

4.9.2 Buffer Verification

- Test buffer overflow/underflow protection
- Verify data integrity through buffers
- Check flow control under various load conditions

4.9.3 Performance Verification

- Measure sustained throughput with various patterns
- Verify buffer efficiency and utilization
- Check latency under different loading conditions

4.10 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 axi4_master_rd_cg

Module: `axi4_master_rd_cg.sv` **Base Module:** axi4_master_rd **Location:** `rtl/amba/axi4/` **Status:** Production Ready

---

5.1 Overview

This is the **clock-gated variant** of axi4_master_rd — same datapath, with an activity-based clock gate wrapped around it for power optimization.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ Clock-Gated Variants Guide

What you get over the base module:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable at runtime:** `cfg_cg_enable` / `cfg_cg_idle_count` inputs
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate

5.2 Parameters

In addition to the axi4_master_rd parameters (note: the base module's `busy` output is NOT re-exported by this wrapper – it is consumed internally as a wake term):

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`SKID_DEPTH_AR`	2	Skid-buffer depth on the AR channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_R`	4	Skid-buffer depth on the R channel. Legal range 2..8 inclusive; odd depths are legal.

The gating controls are RUNTIME INPUTS, not parameters: `cfg_cg_enable` (gate master-enable) and `cfg_cg_idle_count` (idle cycles before gating). Status outputs are `cg_gating` and `cg_idle`. There are no per-domain gates – one `amba_clock_gate_ctrl` gates the whole module. (This page once documented an ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* parameter interface that never existed; the clock-gating guide in this book was always the accurate reference.)

5.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`
`ARSize`	`IW+AW+8+3+2+1+4+3+4+4+UW`
`RSize`	`IW+DW+2+1+UW`

5.3 Ports

Port	Dir	Width	Description
aclk	In	1	
aresetn	In	1	
cfg_cg_enable	In	1	
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_axi_arid	In	[IW-1:0]	
fub_axi_araddr	In	[AW-1:0]	
fub_axi_arlen	In	[7:0]	
fub_axi_arsize	In	[2:0]	
fub_axi_arburst	In	[1:0]	
fub_axi_arlock	In	1	
fub_axi_arcache	In	[3:0]	
fub_axi_arprot	In	[2:0]	
fub_axi_arqos	In	[3:0]	
fub_axi_arregion	In	[3:0]	
fub_axi_aruser	In	[UW-1:0]	
fub_axi_arvalid	In	1	
fub_axi_arready	Out	1	
fub_axi_rid	Out	[IW-1:0]	
fub_axi_rdata	Out	[DW-1:0]	
fub_axi_rresp	Out	[1:0]	
fub_axi_rlast	Out	1	
fub_axi_ruser	Out	[UW-1:0]	
fub_axi_rvalid	Out	1	

Port	Dir	Width	Description
fub_axi_rready	In	1	
m_axi_arid	Out	[IW-1:0]	
m_axi_araddr	Out	[AW-1:0]	
m_axi_arlen	Out	[7:0]	
m_axi_arsize	Out	[2:0]	
m_axi_arburst	Out	[1:0]	
m_axi_arlock	Out	1	
m_axi_arcache	Out	[3:0]	
m_axi_arprot	Out	[2:0]	
m_axi_arqos	Out	[3:0]	
m_axi_arregion	Out	[3:0]	
m_axi_aruser	Out	[UW-1:0]	
m_axi_arvalid	Out	1	
m_axi_arready	In	1	
m_axi_rid	In	[IW-1:0]	
m_axi_rdata	In	[DW-1:0]	
m_axi_rresp	In	[1:0]	
m_axi_rlast	In	1	
m_axi_ruser	In	[UW-1:0]	
m_axi_rvalid	In	1	
m_axi_rready	Out	1	
cg_gating	Out	1	Active gating indicator
cg_idle	Out	1	All buffers empty indicator

5.4 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

5.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

5.6 Usage Examples

```
axi4_master_rd_cg #(
    // Base module parameters (see axi4_master_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .CG_IDLE_COUNT_WIDTH(4)
```

```
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(),           // status: clock currently gated
    .cg_idle(),             // status: interfaces quiet (~r_wakeup)
    // ... all other ports same as axi4_master_rd (except busy)
);
```

5.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

5.8 Related Modules

- **Base Module Functionality:** axi4_master_rd.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

5.9 Testing

`val/amba/test_axi4_master_rd_cg.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_rd_cg.py -v
```

5.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

6 axi4_master_rd_mon

Module: `axi4_master_rd_mon.sv` **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

6.1 Overview

The AXI4 Master Read Monitor combines a functional AXI4 master read interface with comprehensive transaction monitoring and filtering. This is the module you drop into a verification environment when you want real-time protocol checking, error detection, performance metrics, and configurable packet filtering without giving up the datapath — the traffic flows through it, and it watches on the way by.

6.1.1 Key Features

- **Integrated Monitoring:** Combines `axi4_master_rd` with `axi_monitor_filtered`
- **2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)**
- **Error Detection:** SLVERR, DECERR, orphaned read data, timeouts (protocol-violation events are write-monitor only – see Error Detection Events)
- **Timeout Monitoring:** Configurable timeout detection for stuck transactions

- **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
- **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
- **Configuration Validation:** Detects conflicting configuration settings
- **Clock Gating Support:** Busy signal for power management

6.2 Parameters

6.2.1 AXI4 Master Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	2	AR channel skid buffer depth
`SKID_DEPTH_R`	int	4	R channel skid buffer depth
`AXI_ID_WIDTH`	int	8	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	Address bus width
`AXI_DATA_WIDTH`	int	32	Data bus width
`AXI_USER_WIDTH`	int	1	User signal width

6.2.2 Monitor Parameters

Parameter	Type	Default	Description
`UNIT_ID`	logic [7:0]	8'h01	8-bit unit identifier in monitor packets
`AGENT_ID`	logic [15:0]	16'h000A	16-bit agent identifier in monitor packets
`MAX_TRANSACTIONS`	int	16	Maximum concurrent outstanding transactions
`ACLK_MHZ`	int	100	Clock frequency in MHz – keeps the 1 us tick

Parameter	Type	Default	Description
			exact off-100MHz
`CFI_MIN_FREQ_MHZ` / `CFI_MAX_FREQ_MHZ`	int	= ACLK_MHZ	Bounds for the freq-invariant counter LUT (`cfg_freq_sel` indexes within them)
`USE_WDATA_ORDER_Q`	bit	0	Write-data ordering queue
`NUM_BANKS`	int	1	Banked transaction tables. >**1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise
`ID_FILTER_ENABLE`	bit	0	Synthesises the per-instance ID-slice filter
`ID_MATCH_BASE`	int	0	First ID this instance owns
`ID_MATCH_COUNT`	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
`ACTIVE_TRANS_THRESHOLD`	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when `cfg_threshold_enable=1`. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
`ENABLE_FILTERING`	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing

Parameter	Type	Default	Description
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ADDR_RANGE_IS_ERROR	logic [N_ADDR_RANGES-1:0]	'0	Per-range flavor: bit i = 0 -> DEBUG range (hit -> AddrMatch), 1 -> ERROR range (allowlist miss -> Error/ADDR_RANGE). Default all-0 (feature inert).

6.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a `generate`-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection

Parameter	Type	Default	Effect when 0
			cone
`ENABLE_TIMEOUT_LOGIC`	bit	1	Drop the timeout cone **and** the `axi_monitor_timeout` instance
`ENABLE_COMPL_LOGIC`	bit	1	Drop the completion cone
`ENABLE_THRESHOLD_LOGIC`	bit	1	Drop the threshold cone
`ENABLE_PERF_LOGIC`	bit	1	Gates only the reporter's legacy perf-packet cone and the two lifetime counters – the window FSM and meters are unconditional
`ENABLE_DEBUG_LOGIC`	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (the reporter's legacy perf cone is built; `ENABLE_PERF_LOGIC` gates THAT cone only, never the measurement window) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

6.2.4 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

6.3 Ports

6.3.1 AXI4 Interfaces

Frontend Interface (fub_axi_*): - AR channel inputs: `arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser, arvalid` - AR channel output: `arready` - R channel outputs: `rid, rdata, rresp, rlast, ruser, rvalid` - R channel input: `rready`

Master Interface (m_axi_*): - AR channel outputs: `arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser, arvalid` - AR channel input: `arready` - R channel inputs: `rid, rdata, rresp, rlast, ruser, rvalid` - R channel output: `rready`

6.3.2 Monitor Configuration

Basic Configuration:

Port	Direction	Width	Description
`cfg_monitor_enable`	Input	1	Master runtime gate. 0 = monitor inert: no allocation, transaction CAM held clear, and the upstream ready is never stalled (a disabled monitor cannot block the datapath). 1 = normal operation
`cam_clear`	Input	1	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
`cfg_error_enable`	Input	1	Enable error packet generation
`cfg_timeout_enable`	Input	1	Enable timeout detection
`cfg_perf_enable`	Input	1	Enable performance metrics
`cfg_compl_enable`	Input	1	Enable transaction-completion packets

Port	Direction	Width	Description
`cfg_threshold_enable`	Input	1	Enable threshold-crossed packets
`cfg_debug_enable`	Input	1	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
`cfg_timeout_cycles`	Input	16	Unified coarse timeout control, a MICROSECOND count passed through at FULL 16-bit width: 1..65535 us per phase; 0 = 16'hFFFF (~65 ms, effectively never). One value drives all three phase counts (addr/data/resp). The old 4-bit squash that saturated >= 16 at 15 us is retired
`cfg_latency_threshold`	Input	32	Latency threshold for alerts
`cfg_freq_sel`	Input	4	`counter_freq_invariant` LUT index scaling the 1 us timer tick – the microsecond timeouts above are measured in these ticks

The inner monitor's `cfg_debug_level` (tied to 0) and `cfg_debug_mask` (0) are fixed inside the wrapper; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`). `cfg_debug_enable` is the only debug-cone control exposed here.

Filtering Configuration (levels 1 and 3 drop packets; level 2 is reserved):

Level 1 - Packet Type Masks:

Port	Direction	Width	Description
`cfg_axi_pkt_mask`	Input	16	Drop mask for entire packet types
`cfg_axi_err_select`	Input	16	Error routing select (future use)

Level 2 & 3 - Individual Event Masks:

Port	Direction	Width	Description
`cfg_axi_error_mask`	Input	16	Mask specific error events
`cfg_axi_timeout_mask`	Input	16	Mask specific timeout events
`cfg_axi_compl_mask`	Input	16	Mask specific completion events
`cfg_axi_thresh_mask`	Input	16	Mask specific threshold events
`cfg_axi_perf_mask`	Input	16	Mask specific performance events
`cfg_axi_addr_mask`	Input	16	Mask specific address match events
`cfg_axi_debug_mask`	Input	16	Mask specific debug events

6.3.3 Monitor Bus Output

Port	Direction	Width	Description
`monbus_valid`	Output	1	Monitor packet valid
`monbus_ready`	Input	1	Downstream ready to accept packet
`monbus_packet`	Output	128	`monitor_packet_t` (see format below)
`monbus_timestamp`	Output	64	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_bloc	Output	1	Observability tap for the

Port	Direction	Width	Description
k_ready			block_ready gating net (drives nothing internally; leave unconnected if unused)
`i_mon_time`	Input	64	Free-running counter from `monbus_group_core`, sampled at packet emission

6.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions (for clock gating)
`active_transactions`	Output	8	Current number of outstanding transactions
`error_count`	Output	16	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`transaction_count`	Output	32	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`cfg_conflict_error`	Output	1	Configuration conflict detected

6.3.5 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

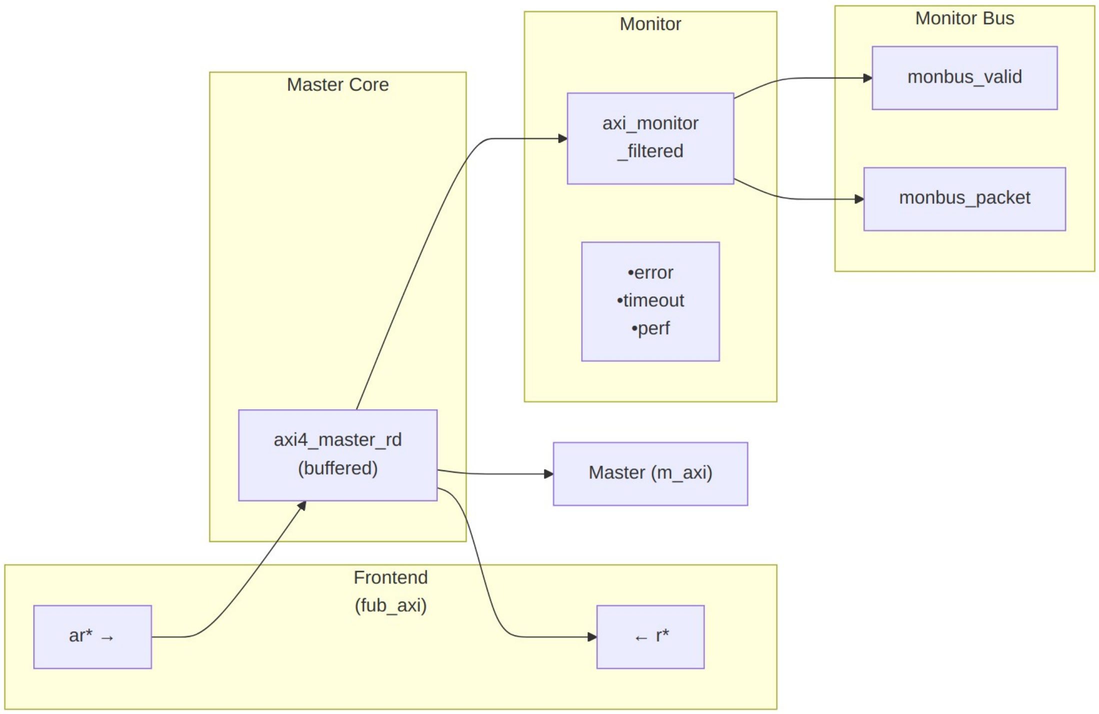
The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
`cfg_id_filter_enable`	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
`cfg_id_match_base`	`ID_WIDTH`	First ID owned
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

6.4 Functional Description



Module Architecture

The module instantiates two sub-modules: 1. **axi4_master_rd** - Core AXI4 functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)

6.4.1 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the plain `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axi_arready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axi_arready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (in-RTL formal property `ap_disabled_never_stalls`).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `monitor_common_pkg`), and `block_ready` re-asserts at occupancy `< MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

6.4.2 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper exposes **`ADDR_RANGE_IS_ERROR`** (per-range) as a parameter, so ranges can be assigned to either flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

6.4.3 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`. The **measurement-window state machine** and its counters are unconditional `always_ff` blocks in `axi_monitor_base` – outside every generate – so they are present and running in EVERY build, including `ENABLE_PERF_LOGIC = 0`. That parameter reaches one consumer: the `g_perf` generate in `axi_monitor_reporter` holding the legacy perf-packet cone and the two 16-bit lifetime counters behind `perf_completed_count` / `perf_error_count`. Setting it to 0 saves that cone and nothing else. `USE_MONITOR = 0` is what ties the perfmon outputs off. The wrapper instantiates plus a bank of R-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

6.4.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

6.4.3.2 Utilization Counters (R Data Channel)

Every cycle inside the window is classified by the R channel's `rvalid` / `rready` into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	rvalid && rready	productive beat transferred
`perf_bp_cycles`	32	rvalid && !rready	back-pressure (data offered, sink not ready)
`perf_starv_cycles`	32	!rvalid && rready	starvation (sink ready, no data)
`perf_idle_cycles`	32	!rvalid && !rready	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); the one-count skew is negligible for long windows.

6.4.3.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	R data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)

Output	Width	Meaning
`perf_byte_count`	64	bytes transferred = beats × (1 << ARSIZE), using the ARSIZE captured at the most recent AR address phase (upper bound: counts full-width beats; an unaligned start address means the first beat carries fewer useful bytes)
`perf_burst_count`	32	AR address-phase handshakes

The integrator computes average burst length as `perf_beat_count / perf_burst_count`.

6.4.3.4 Performance Monitoring Ports

Port	Direction	Width	Description
`cfg_perf_enable`	Input	1	Enable performance-metric packet generation
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end** event source select
`cfg_start_trigger`	Input	1	Pulse: open the measurement window
`cfg_end_trigger`	Input	1	Pulse: close the measurement window
`cfg_window_force_close`	Input	1	Software override: force the window closed
`window_active`	Output	1	High while a measurement window is open
`window_cycles`	Output	32	Cycles elapsed in the current window
`perf_prod_cycles`	Output	32	rvalid && rready cycles

Port	Direction	Width	Description
perf_bp_cycles	Output	32	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	Output	32	!rvalid && rready cycles (starvation)
perf_idle_cycles	Output	32	!rvalid && !rready cycles
perf_beat_count	Output	32	R data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AR address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

6.4.4 Monitor Packet Format

The 128-bit `monbus_packet` (paired with the 64-bit `monbus_timestamp` side-band signal) follows the standardized AMBA monitor bus format:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, DECERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished)
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits — AXI ID or channel index)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits — full address, latency, etc.)
```

6.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

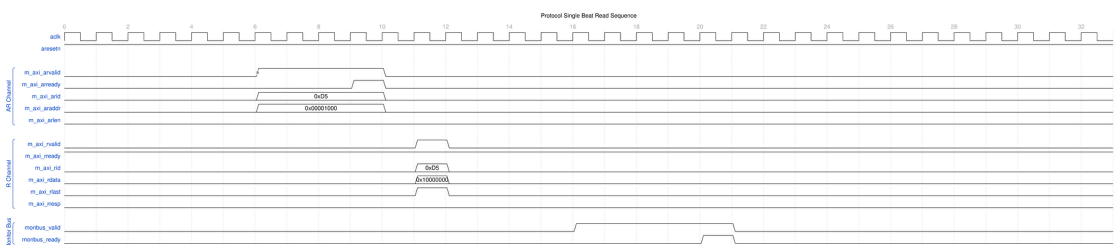
No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

6.6 Waveforms

The following waveforms show AXI4 master read monitor behavior across various scenarios:

6.6.1 Scenario 1: Single-Beat Read Transaction

Complete AXI4 read transaction with AR handshake and R response:



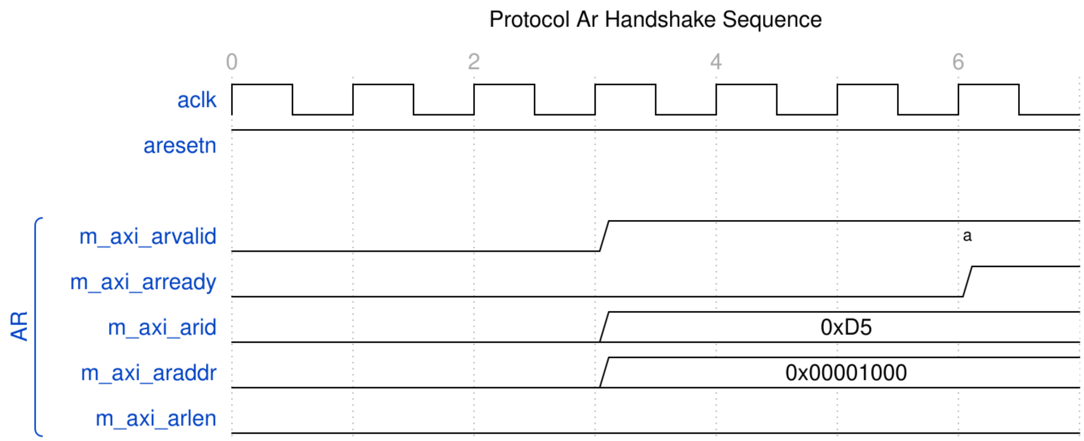
Single Beat Read

WaveJSON: single_beat_read_001.json

Key Observations: - AR channel handshake (ARVALID/ARREADY) - R channel response (RVALID/RREADY/RLAST) - Monitor bus packet generation - Transaction tracking and completion

6.6.2 Scenario 2: AR Channel Handshake Detail

Detailed view of address read channel handshake:



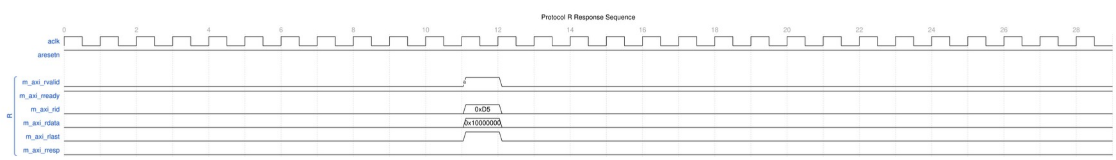
AR Handshake

WaveJSON: ar_handshake_001.json

Key Observations: - ARVALID assertion timing - ARREADY response from slave - Address and control signal stability - Handshake protocol compliance

6.6.3 Scenario 3: R Response Channel Detail

Read data response channel behavior:



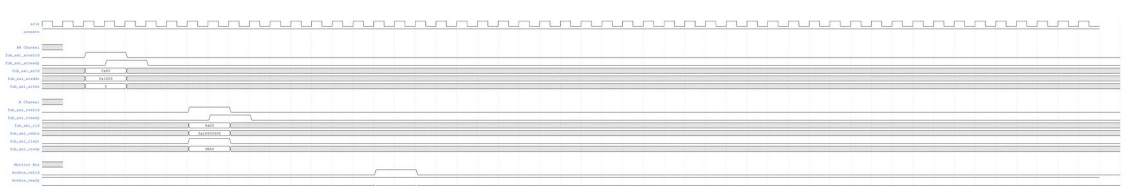
R Response

WaveJSON: r_response_001.json

Key Observations: - RVALID/RREADY handshake - RLAST assertion for transaction completion - RRESP status (OKAY/SLVERR/DECERR) - Read data capture

6.6.4 Scenario 4: Complete AXI4 Read Transaction

End-to-end AXI4 read transaction flow:



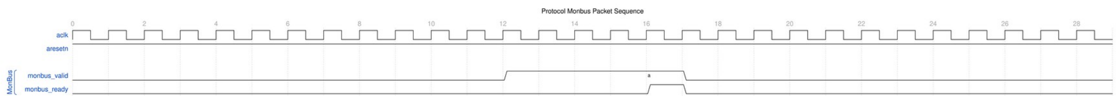
AXI4 Read Transaction

WaveJSON: axi4_read_transaction_001.json

Key Observations: - Complete AR → R channel flow - Transaction ID tracking - Burst length and size handling - Monitor packet correlation

6.6.5 Scenario 5: Monitor Bus Packet Generation

Monitor bus output packet format and timing:



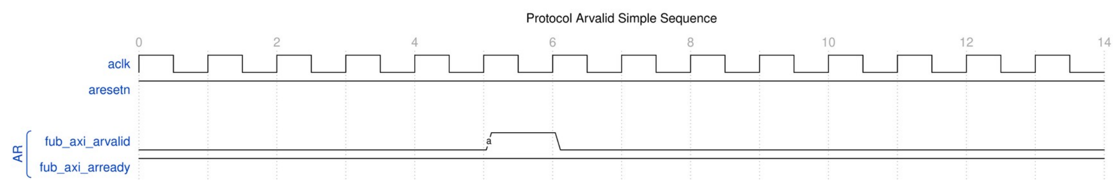
MonBus Packet

WaveJSON: monbus_packet_001.json

Key Observations: - Monitor bus valid/ready handshake - 128-bit packet format (plus 64-bit side-band timestamp) - Packet type encoding - Event data payload

6.6.6 Scenario 6: Simple ARVALID Transaction

Simplified view of ARVALID-based transaction:



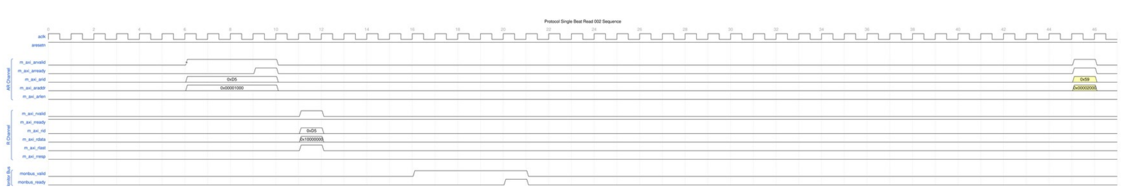
ARVALID Simple

WaveJSON: arvalid_simple_001.json

Key Observations: - Minimal handshake sequence - Single-beat read operation - Fast transaction completion

6.6.7 Scenario 7: Alternative Single-Beat Read

Variant single-beat read with different timing:



Single Beat Read Alt

WaveJSON: single_beat_read_002_001.json

Key Observations: - Different backpressure pattern - Ready signal de-assertion effects - Transaction latency variation

---

6.7 Usage Examples

6.7.1 Configuration Strategies

6.7.1.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch error responses and orphans, and track completion

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),      // Errors
.cfg_timeout_enable     (1'b1),      // Timeouts
.cfg_perf_enable        (1'b0),      // Disable (reduces traffic)

// Filtering - pass error and timeout, drop others
.cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR, TIMEOUT
(set bit = drop)
.cfg_axi_err_select     (16'h0000),
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask   (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask      (16'hFFFF),  // Drop performance
.cfg_axi_debug_mask     (16'hFFFF),  // Drop debug

// Timeouts
.cfg_timeout_cycles     (16'd10),    // 10 microseconds per phase
(full 16-bit range)
.cfg_freq_sel         (4'd0),    // counter_freq_invariant LUT index;
scales the 1 us tick
.cfg_latency_threshold  (32'd500)
```

6.7.1.2 Strategy 2: Performance Analysis

Goal: Collect performance metrics, suppress completions

Two suppression options exist. Runtime-disabling completions (`cfg_compl_enable=0`) is safe — terminal entries auto-retire without emitting packets or bumping counters (see axi_monitor_reporter) — but the `transaction_count` output stops advancing. The mask approach below (`cfg_axi_compl_mask`) keeps marking and counting while dropping the packets downstream in `axi_monitor_filtered`.

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
```

```
.cfg_error_enable       (1'b1),      // Still catch errors
.cfg_timeout_enable     (1'b0),      // Disable timeouts
.cfg_perf_enable        (1'b1),      // Enable performance

// Filtering - pass error and performance only
.cfg_axi_pkt_mask       (16'hFFEE),  // Drop all but ERROR, PERF (set
bit = drop)
.cfg_axi_err_select     (16'h0000),
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_perf_mask      (16'h0000),  // Pass all performance
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_timeout_mask   (16'hFFFF),  // Drop timeouts
.cfg_axi_debug_mask     (16'hFFFF),  // Drop debug
```

6.7.1.3 Strategy 3: Debug Mode

Goal: Maximum visibility, expect low traffic

```
// Enable everything
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),
.cfg_timeout_enable     (1'b1),
.cfg_perf_enable        (1'b1),

// Filtering - pass all packets
.cfg_axi_pkt_mask       (16'h0000),  // Pass all packet types
.cfg_axi_err_select     (16'h0000),
.cfg_axi_error_mask     (16'h0000),
.cfg_axi_timeout_mask   (16'h0000),
.cfg_axi_compl_mask     (16'h0000),  // Pass completions
.cfg_axi_thresh_mask    (16'h0000),
.cfg_axi_perf_mask      (16'h0000),
.cfg_axi_addr_mask      (16'h0000),
.cfg_axi_debug_mask     (16'h0000)
```

WARNING: Avoid enabling all packet types in high-throughput scenarios — the monitor bus sustains at most one packet per two cycles and will congest. (Since the auto-retire fix in `95c9490a`, congestion or runtime-disabling classes can drop packets but can no longer leak table slots or wedge the bus.)

6.7.2 Basic Integration

```
// Instantiate AXI4 master read monitor
axi4_master_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
```

```
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (1),
    .AGENT_ID           (10),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_master_rd_mon (
    .aclk               (axi_aclk),
    .aresetn            (axi_aresetn),

    // Frontend interface
    .fub_axi_arid       (read_arid),
    .fub_axi_araddr     (read_araddr),
    .fub_axi_arlen      (read_arlen),
    .fub_axi_arsize     (read_arsize),
    .fub_axi_arburst    (read_arburst),
    .fub_axi_arlock     (1'b0),
    .fub_axi_arcache    (4'b0010),
    .fub_axi_arprot     (3'b000),
    .fub_axi_arqos      (4'b0000),
    .fub_axi_arregion   (4'b0000),
    .fub_axi_aruser     (1'b0),
    .fub_axi_arvalid    (read_arvalid),
    .fub_axi_arready    (read_arready),

    .fub_axi_rid        (read_rid),
    .fub_axi_rdata      (read_rdata),
    .fub_axi_rresp      (read_rresp),
    .fub_axi_rlast      (read_rlast),
    .fub_axi_ruser      (read_ruser),
    .fub_axi_rvalid     (read_rvalid),
    .fub_axi_rready     (read_rready),

    // Master interface (to interconnect)
    .m_axi_arid         (m_axi_arid),
    .m_axi_araddr       (m_axi_araddr),
    .m_axi_arlen        (m_axi_arlen),
    .m_axi_arsize       (m_axi_arsize),
    .m_axi_arburst      (m_axi_arburst),
    .m_axi_arlock       (m_axi_arlock),
    .m_axi_arcache      (m_axi_arcache),
    .m_axi_arprot       (m_axi_arprot),
    .m_axi_arqos        (m_axi_arqos),
    .m_axi_arregion     (m_axi_arregion),
    .m_axi_aruser       (m_axi_aruser),
    .m_axi_arvalid      (m_axi_arvalid),
    .m_axi_arready      (m_axi_arready),
```

```
    .m_axi_rid          (m_axi_rid),
    .m_axi_rdata        (m_axi_rdata),
    .m_axi_rresp        (m_axi_rresp),
    .m_axi_rlast        (m_axi_rlast),
    .m_axi_ruser        (m_axi_ruser),
    .m_axi_rvalid       (m_axi_rvalid),
    .m_axi_rready       (m_axi_rready),

    // Monitor configuration (Strategy 1 - Functional)
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd10),    // 10 microseconds per phase
(full 16-bit range)
    .cfg_latency_threshold  (32'd500),

    .cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR,
TIMEOUT
    .cfg_axi_err_select     (16'h0000),
    .cfg_axi_error_mask     (16'h0000),
    .cfg_axi_timeout_mask   (16'h0000),
    .cfg_axi_compl_mask     (16'hFFFF),
    .cfg_axi_thresh_mask    (16'hFFFF),
    .cfg_axi_perf_mask      (16'hFFFF),
    .cfg_axi_addr_mask      (16'hFFFF),
    .cfg_axi_debug_mask     (16'hFFFF),

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (rd_busy),
    .active_transactions    (rd_active),
    .error_count            (rd_errors),
    .transaction_count      (rd_count),
    .cfg_conflict_error     (cfg_conflict),

    // ... 21 further inputs elided for brevity. Verilator escalates
    // PINMISSING to an error in this repo, so a real instantiation
must
    // connect them all. Two are load-bearing rather than merely tied
off:
    //   .i_mon_time  -- the free-running timestamp broadcast; leave
it
```

```
    //                  floating and every packet is stamped with X
    //   .cam_clear   -- synchronous clear of the transaction CAM
    // The rest are tie-offs: cfg_compl_enable, cfg_threshold_enable,
    // cfg_debug_enable, cfg_freq_sel, the addr-check group
    // (cfg_addr_check_enable / cfg_addr_range_enable / _low / _high),
    // the filter group (cfg_addr_filter_* , cfg_id_*) and the perfmon
    // window controls (cfg_start_event_sel / cfg_end_event_sel /
    // cfg_start_trigger / cfg_end_trigger / cfg_window_force_close).
    .i_mon_time             (mon_time),
    .cam_clear              (1'b0)
);

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH(128),
    .DEPTH(256)
) u_mon_fifo (
    .axi_aclk      (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    .wr_valid    (mon_valid),
    .wr_data     (mon_packet),
    .wr_ready    (mon_ready),
    .rd_valid    (fifo_valid),
    .rd_data     (fifo_data),
    .rd_ready    (consumer_ready)
);
```

6.8 Design Notes

6.8.1 Filtering Hierarchy

The 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)

1. **Level 1 (cfg_axi_pkt_mask):** Coarse filtering by packet type
   - Bit per packet type (ERROR, TIMEOUT, COMPL, etc.)
   - 1 = drop, 0 = pass to next level
2. **Level 2 (cfg_axi_err_select):** Future error routing
   - Reserved for directing errors to alternate paths
3. **Level 3 (cfg_axi_*_mask):** Fine-grained event filtering
   - Individual event masks within each packet type
   - Allows selecting specific events while dropping others

6.8.2 Configuration Conflicts

`cfg_conflict_error` asserts when a packet type dropped by `cfg_axi_pkt_mask` is simultaneously selected for error routing by `cfg_axi_err_select`:

```
assign cfg_conflict_error = |(cfg_axi_pkt_mask & cfg_axi_err_select);
```

6.8.3 Performance Considerations

Monitor Bus Bandwidth: - The reporter sustains at most 1 packet per 2 cycles - Completion packets: 1 per transaction - Performance packets: periodic count rollups (completed-count and error-count), not per-transaction - Error packets: Variable (0-N per transaction)

Recommended Packet Budget: - Functional mode: 1-2 packets per transaction (COMPL + occasional ERROR/TIMEOUT) - Performance mode: ERROR packets plus periodic PERF rollups - Debug mode: several packets per transaction (all types)

6.8.4 Transaction Table

Monitors up to `MAX_TRANSACTIONS` concurrent transactions: - Tracks ARID, address, latency, status - Generates packets on completion, timeout, or error - Proper cleanup via event_reported feedback (fixed in v1.1)

6.9 Related Modules

6.9.1 Companion Monitors

- **axi4_master_wr_mon** - AXI4 master write with monitoring
- **axi4_slave_rd_mon** - AXI4 slave read with monitoring
- **axi4_slave_wr_mon** - AXI4 slave write with monitoring

6.9.2 Base Modules

- axi4_master_rd - Functional AXI4 master read (without monitoring)
- **axi_monitor_filtered** - Monitoring engine with filtering (monitor/)

6.9.3 Used Components

- gaxi_skid_buffer - Elastic buffering
- **axi_monitor_base** - Core monitoring logic (monitor/)
- **axi_monitor_trans_mgr** - Transaction tracking (monitor/)

6.10 Testing

`val/amba/test_axi4_master_rd_mon.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_rd_mon.py -v
```

6.11 References

6.11.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: monitor_package_spec.md

6.11.2 Source Code

- RTL: `rtl/amba/axi4/axi4_master_rd_mon.sv`
- Tests: `val/amba/test_axi4_master_rd_mon.py`
- Framework: `bin/TBClasses/components/axi4/`

6.11.3 Documentation

- Configuration Guide: AXI Monitor Base
- Architecture: rtl-amba Overview
- AXI4 Index: axi4/README.md

Last Updated: 2026-07-18

6.12 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

7 axi4_master_rd_mon_cg

Module: `axi4_master_rd_mon_cg.sv` **Base Module:** axi4_master_rd_mon **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

7.1 Overview

The `axi4_master_rd_mon_cg` module is a clock-gated variant of axi4_master_rd_mon that adds comprehensive power optimization capabilities through activity-based clock gating.

7.1.1 Key Differences from Base Module

- **Activity-Based Clock Gating:** one gate for the whole module, woken by bus valids, core busy, and a pending monitor packet
- **Runtime Control:** `cfg_cg_enable` / `cfg_cg_idle_count` (no per-domain policies)
- **Gating Status:** `cg_gating` / `cg_idle` outputs (no built-in power statistics)
- **Functional differences from base:** `ACTIVE_TRANS_THRESHOLD` not forwarded (harmless – its inner default is derived from the forwarded `MAX_TRANSACTIONS`), `debug_block_ready` tied off, and gated-clock caveats for windows/triggers (see the WARNING below) – use the base module when those matter

All other functionality is identical to the base module. See axi4_master_rd_mon.md for complete functional specification.

7.1.2 When to Use the Clock-Gated Variant

Use `axi4_master_rd_mon_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use base module (`axi4_master_rd_mon`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

---

7.2 Parameters

7.2.1 Clock Gating Parameters

In addition to all parameters from axi4_master_rd_mon, this module adds:

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`ADD_PIPELINE_STAGE`	bit	`0`	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
`ENABLE_COMPL_LOGIC`	bit	`1'b1`	Synthesise the completion-packet cone. 0 removes the logic entirely.
`ENABLE_DEBUG_LOGIC`	bit	`1'b0`	Synthesise the debug-packet cone. 0 removes the logic entirely.
`ENABLE_ERROR_LOGIC`	bit	`1'b1`	Synthesise the error detection cone. 0 removes the logic entirely.
`ENABLE_FILTERING`	bit	`1`	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.
`ENABLE_PERF_LOGIC`	bit	`1'b1`	Synthesise the reporter's performance cone (`g_perf`). Does NOT gate the perfmon window state machine or its counters.

Parameter	Type	Default	Description
ENABLE_THRESHOLD_LOGIC	bit	1'b1	Synthesise the threshold-packet cone. 0 removes the logic entirely.
ENABLE_TIMEOUT_LOGIC	bit	1'b1	Synthesise the timeout detection cone. 0 removes the logic entirely.
SKID_DEPTH_AR	int	2	Skid-buffer depth on the AR channel. Legal range 2..8 inclusive; odd depths are legal.
SKID_DEPTH_R	int	4	Skid-buffer depth on the R channel. Legal range 2..8 inclusive; odd depths are legal.
AGENT_ID	logic	16'h000A	Agent identifier emitted in the `agent_id` field of every monitor packet. Pairs with `UNIT_ID` to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID	logic	8'h01	Unit identifier emitted in the `unit_id` field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The gating controls are RUNTIME INPUTS: `cfg_cg_enable` and `cfg_cg_idle_count`; status outputs are `cg_gating` / `cg_idle`. ONE `amba_clock_gate_ctrl` gates the entire inner monitor module – there are no per-domain gates and no `cg_*_gated` status signals. (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

7.2.2 Base Module Parameters

MOST base-module parameters pass through. As of 2026-09-01 only `ACTIVE_TRANS_THRESHOLD` is NOT forwarded, and that one is harmless: its inner default is `MAX_TRANSACTIONS/2`, computed from the `MAX_TRANSACTIONS` this wrapper DOES forward.

An earlier version of this page listed nine more as unforwarded, which was accurate when written. `ACLK_MHZ`, `CFI_MIN_FREQ_MHZ`, `CFI_MAX_FREQ_MHZ`, `USE_WDATA_ORDER_Q`, `NUM_BANKS` and `ADDR_RANGE_IS_ERROR` were threaded through every `_cg` wrapper on 2026-09-01, and `ID_FILTER_ENABLE` / `ID_MATCH_BASE` / `ID_MATCH_COUNT` the day before. Until then a clock-gated build could not state its clock frequency, so the 1 us timer tick was pinned to the 100 MHz default and every microsecond-denominated timeout was miscalibrated on any other clock – silently.

Among the forwarded ones:

Parameter	Type	Default	Description
`USE_MONITOR`	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
`N_ADDR_RANGES`	int	0	Number of address-range comparators (forwarded to base module).
`ADDR_FILTER_ENABLE`	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves `cfg_addr_filter_enable` low filters nothing and looks broken.
`ID_FILTER_ENABLE`	bit	0	Synthesises the ID report filter (see `cfg_id_*` for the runtime override).
`ID_MATCH_BASE`	int	0	First ID this instance owns.

Parameter	Type	Default	Description
ID_MATCH_COUNT	int	0	How many IDs; 0 means ALL, so a zeroed register block does not silently filter everything away.

Base-module ports are forwarded EXCEPT debug_block_ready, which the wrapper ties off (use the base module when you need the backpressure tap). cam_clear is forwarded (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - and the full performance-monitoring interface (see Performance Monitoring below). The six ENABLE_*_LOGIC synthesis-cone parameters and the cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable control enables are also passed straight through.

7.2.3 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
cfg_addr_filter_enable	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever ADDR_FILTER_ENABLE says
cfg_addr_filter_low	Input	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	Input	ADDR_WIDTH	Window limit, inclusive
cfg_id_filter_enable	Input	1	High: use the runtime window below instead of the ID_MATCH_* parameters
cfg_id_match_base	Input	ID_WIDTH	First ID to accept
cfg_id_match_count	Input	ID_WIDTH+1	How many; 0 means ALL

Neither filter un-filters entries already admitted to the transaction table, which is what makes changing them at runtime safe.

7.2.4 Parameter Relationships

- **`cfg_cg_enable = 0`**: bypasses the gate at runtime; behavior is identical to base
- **`cfg_cg_idle_count`**: higher values keep the clock running longer after traffic stops, so the block is gated less often. It does not change wake-up latency, which is fixed at 1 register stage / 2 clocks to the first usable edge.

7.2.5 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

7.3 Ports

Port	Dir	Width	Description
`aclk`	In	1	
`aresetn`	In	1	
`cam_clear`	In	1	sync clear of the monitor trans CAM
`fub_axi_arid`	In	`[IW-1:0]`	
`fub_axi_araddr`	In	`[AW-1:0]`	
`fub_axi_arlen`	In	`[7:0]`	
`fub_axi_arsize`	In	`[2:0]`	
`fub_axi_arburst`	In	`[1:0]`	
`fub_axi_arlock`	In	1	

Port	Dir	Width	Description
fub_axi_arcache	In	[3:0]	
fub_axi_arprot	In	[2:0]	
fub_axi_arqos	In	[3:0]	
fub_axi_arregion	In	[3:0]	
fub_axi_aruser	In	[UW-1:0]	
fub_axi_arvalid	In	1	
fub_axi_arready	Out	1	
fub_axi_rid	Out	[IW-1:0]	
fub_axi_rdata	Out	[DW-1:0]	
fub_axi_rresp	Out	[1:0]	
fub_axi_rlast	Out	1	
fub_axi_ruser	Out	[UW-1:0]	
fub_axi_rvalid	Out	1	
fub_axi_rready	In	1	
m_axi_arid	Out	[IW-1:0]	
m_axi_araddr	Out	[AW-1:0]	
m_axi_arlen	Out	[7:0]	
m_axi_arsize	Out	[2:0]	
m_axi_arburst	Out	[1:0]	
m_axi_arlock	Out	1	
m_axi_arcache	Out	[3:0]	
m_axi_arprot	Out	[2:0]	
m_axi_arqos	Out	[3:0]	
m_axi_arregion	Out	[3:0]	
m_axi_aruser	Out	[UW-1:0]	
m_axi_arvalid	Out	1	
m_axi_arready	In	1	
m_axi_rid	In	[IW-1:0]	
m_axi_rdata	In	[DW-1:0]	
m_axi_rresp	In	[1:0]	

Port	Dir	Width	Description
m_axi_rlast	In	1	
m_axi_ruser	In	[UW-1:0]	
m_axi_rvalid	In	1	
m_axi_rready	Out	1	
cfg_monitor_enable	In	1	Enable monitoring
cfg_error_enable	In	1	Enable error detection
cfg_timeout_enable	In	1	Enable timeout detection
cfg_perf_enable	In	1	Enable performance monitoring
cfg_compl_enable	In	1	Enable completion packets
cfg_threshold_enable	In	1	Enable threshold packets
cfg_debug_enable	In	1	Enable debug packets
cfg_timeout_cycles	In	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	In	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	In	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	In	[15:0]	Drop mask for packet types
cfg_axi_err_select	In	[15:0]	Error select for packet types (for

Port	Dir	Width	Description
			future routing)
cfg_axi_error_mask	In	[15:0]	Individual error event mask
cfg_axi_timeout_mask	In	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	In	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	In	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	In	[15:0]	Individual performance event mask
cfg_axi_addr_mask	In	[15:0]	Individual address match event mask
cfg_axi_debug_mask	In	[15:0]	Individual debug event mask
cfg_cg_enable	In	1	Enable clock gating
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	In	1	
cfg_addr_range_enable	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_low	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_high	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES :	

Port	Dir	Width	Description
		1)-1:0]	
cfg_addr_filter_enable	In	1	
cfg_addr_filter_low	In	[AW-1:0]	
cfg_addr_filter_high	In	[AW-1:0]	
cfg_id_filter_enable	In	1	
cfg_id_match_base	In	[IW-1:0]	
cfg_id_match_count	In	[IW:0]	
i_mon_time	In	1	
monbus_valid	Out	1	Monitor bus valid
monbus_ready	In	1	Monitor bus ready
monbus_packet	Out	1	Monitor packet (128-bit)
monbus_timestamp	Out	1	Side-band sampled time
busy	Out	1	
active_transactions	Out	[7:0]	Number of active transactions
error_count	Out	[15:0]	Total error count
transaction_count	Out	[31:0]	Total transaction count
cg_gating	Out	1	Gated clock is stopped
cg_idle	Out	1	No activity observed
cfg_conflict_error	Out	1	Configuration conflict detected
cfg_start_event_sel	In	[2:0]	
cfg_end_event_sel	In	[2:0]	

Port	Dir	Width	Description
cfg_start_trigger	In	1	
cfg_end_trigger	In	1	
cfg_window_force_close	In	1	
window_active	Out	1	
window_cycles	Out	[31:0]	
perf_prod_cycles	Out	[31:0]	
perf_bp_cycles	Out	[31:0]	
perf_starv_cycles	Out	[31:0]	
perf_idle_cycles	Out	[31:0]	
perf_beat_count	Out	[31:0]	
perf_byte_count	Out	[63:0]	
perf_burst_count	Out	[31:0]	

7.4 Functional Description

7.4.1 Clock Gating Architecture

ONE `amba_clock_gate_ctrl` gates the whole inner monitor module. The wake term is bus activity plus pending monitor work (`fub_axi_arvalid || fub_axi_rvalid || int_busy || w_monbus_valid || (|active_transactions)` on the user side, `m_axi_arvalid || m_axi_rvalid` on the AXI side); when both sides idle for `cfg_cg_idle_count` cycles with nothing pending on the monitor bus and the monitor CAM empty, everything inside – transaction tracking, reporter, timers, perf counters – stops together. There are no per-domain gates. The external `monbus_valid` is masked with `!cg_gating`, so monitor-bus delivery is exactly-once across gating at any idle count (`val/amba/test_mon_cg_gating.py` phase 6): a packet pending delivery holds the block awake, and the CAM-occupancy term covers the reporter's few-cycle retire -> FIFO -> output emission window, so a trailing packet cannot be stranded by the clock stopping before its valid rises – even at `cfg_cg_idle_count = 0`.

7.4.2 Gating State Machine

```
ACTIVE  IDLE_COUNT  GATED



        (Activity Detected)
```

```
States:
- ACTIVE: Clocks enabled, monitoring activity
- IDLE_COUNT: Counting cfg_cg_idle_count before gating
- GATED: Clocks disabled, waiting for activity
```

7.4.3 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_master_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi4_master_rd_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

WARNING – gating vs window accounting. The whole inner monitor runs on the gated clock, window state machine and counters included. An open measurement window is not one of the terms that keeps the clock alive: `user_valid` is bus valids, `busy`, and pending monitor-bus work, and nothing else.

Two consequences:

- **Counters freeze.** If the bus idles past the countdown while a window is open, the clock gates and `window_cycles` and the perf counters stop while wall-clock time keeps passing. The host reads an under-counted window and has no indication it was short.
- **Pulses are dropped.** `cam_clear`, `cfg_start_trigger`, `cfg_end_trigger` and `cfg_window_force_close` are sampled in the gated domain, so a pulse that arrives while the clock is gated is lost entirely.

For exact wall-clock windows, or for triggering on an idle bus, hold `cfg_cg_enable` low across the measurement or use the base module.

7.5 Timing Characteristics

7.5.1 Wake-Up and Gating Latency

Wake-up latency does not depend on `cfg_cg_idle_count`. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This monitor wrapper drives the activity terms combinationally, so it has **1 register stage** and the first usable gated-clock edge arrives **2 clock cycles** after activity asserts.

`cfg_cg_idle_count` sets the going-to-sleep delay only: gating engages `cfg_cg_idle_count + 1` clocks after the internal wakeup deasserts, which is `cfg_cg_idle_count + 2` clocks after the last bus activity.

Configuration	Clocks from last activity to gating	Use Case
`cfg_cg_idle_count=4`	6 clock cycles	Low-latency, frequent bursts
`cfg_cg_idle_count=8`	10 clock cycles	Balanced
`cfg_cg_idle_count=15`	17 clock cycles	Maximum power savings, infrequent traffic

7.6 Usage Examples

7.6.1 Example 1: Maximum Power Savings (Burst Traffic)

```
axi4_master_rd_mon_cg #(
    // Base parameters (see axi4_master_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // Clock gating -- aggressive: gate quickly after 4 idle cycles
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd4),
```

```
    .cg_gating(), .cg_idle(),
    // ... connect signals same as base module
);
```

7.6.2 Example 2: Balanced Performance and Power

```
axi4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // Clock gating -- balanced: wait longer before gating (gated less
often)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd15),
    .cg_gating(), .cg_idle(),
    // ... connect signals same as base module
);
```

7.6.3 Example 3: Clock Gating Disabled (Functional Verification)

```
axi4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // Clock gating -- DISABLED for verification (runtime bypass)
    .cfg_cg_enable(1'b0),
    .cfg_cg_idle_count(4'd0),
    .cg_gating(), .cg_idle(),
    // ... connect signals same as base module
);
```

Note: With `cfg_cg_enable=0`, this module is functionally identical to the base module.

7.7 Design Notes

7.7.1 Power Savings Analysis

These numbers are illustrative, not measured. No power analysis has been run on this library – `axi4_master_rd_cg.md` states the ground truth: power saving is “traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized.” The shape of the curve (more idle time and a shorter idle count save more) is sound; the specific percentages are not sourced and disagree with the guide’s table at comparable duty cycles. Treat the rows as a sketch of the trend.

Traffic Pattern	Clock Gating Enabled	Power Savings
10% Utilization	Aggressive (`cfg_cg_idle_count =4`)	60-70%
25% Utilization	Balanced (`cfg_cg_idle_count =8`)	45-55%
50% Utilization	Conservative (`cfg_cg_idle_count =15`)	25-35%
90% Utilization	Any configuration	5-10%

Note: Actual savings depend on FPGA/ASIC technology, tool implementation, and traffic patterns.

7.7.2 Power Monitoring Signals

The module provides these status signals for power analysis:

Signal	Width	Description
`cg_gating`	1	The (single) gated clock is currently stopped
`cg_idle`	1	No activity this cycle (~`r_wakeup` – asserts one cycle after the interfaces go quiet; it does NOT wait for the countdown)

7.7.3 Verification Considerations

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
axi4_master_rd_mon_cg dut (
    .cfg_cg_enable(1'b0),   // runtime bypass for faster simulation
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Faster simulation (no gating overhead) - Easier debug (no timing dependencies)

For power-specific verification:

1. **Enable clock gating** with realistic parameters
2. **Monitor gating signals** (`cg_gating` / `cg_idle`) to verify expected behavior
3. **Vary traffic patterns** to test gating effectiveness
4. **Check wake-up timing** meets system requirements

7.7.4 Synthesis Considerations

FPGA Implementations

Xilinx: - Drive `cfg_cg_enable=1` and let synthesis map the ICG to `BUFGCE` primitives - Tool will infer clock enables automatically - Verify with post-synthesis power analysis

Intel (Altera): - Drive `cfg_cg_enable=1` and let synthesis map the ICG to `ALTCLKCTRL` - May need vendor-specific clock gating primitives - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

ASIC Implementations: - Work with foundry to select appropriate clock gating cells - Integrated Clock Gating (ICG) cells provide best results - Consider hold-time implications of clock gating - Verify power intent with UPF (Unified Power Format)

7.8 Related Modules

- **axi4_master_rd_mon** - Base module (non-clock-gated)
- **axi_monitor_base** - Core monitoring infrastructure
- **axi_monitor_filtered** - Filtering capabilities

7.8.1 See Also

- **Power Optimization Guide:** `docs/POWER_OPTIMIZATION_GUIDE.md`
- **Clock Gating Best Practices:** `docs/CLOCK_GATING_GUIDE.md`
- **AMBA Subsystem Overview:** `docs/markdown/rtl-amba/overview.md`

7.9 Testing

`val/amba/test_axi4_master_rd_mon_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_rd_mon_cg.py -v
```

7.10 Navigation

- **← Back to Base Module**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

8 AXI4 Master Read Stub

Module: `axi4_master_rd_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

8.1 Overview

Drive AXI4 read transactions without writing an AXI4 master into your testbench. The AXI4 Master Read Stub packs the AR (read address) and R (read data) channels into single wide packet buses through skid buffers, so the stimulus side sees one valid/ready handshake and one concatenated payload per channel instead of a dozen individual signals. That makes it a natural fit for testbenches and integration scenarios where a simplified interface beats a protocol-faithful one.

8.1.1 Key Features

- Packed packet interface for AR and R channels
- Configurable skid buffer depth on each channel
- Full AXI4 read transaction support
- Burst, ID, and user signal support
- Parameterized data widths

8.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	R channel skid buffer depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

8.3 Ports

8.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock

Port	Direction	Width	Description
aresetn	Input	1	AXI reset (active low)

8.3.2 AXI4 Read Address Channel (AR)

Port	Direction	Width	Description
m_axi_arid	Output	AXI_ID_WIDTH	Read address ID
m_axi_araddr	Output	AXI_ADDR_WIDTH	Read address
m_axi_arlen	Output	8	Burst length
m_axi_arsize	Output	3	Burst size
m_axi_arburst	Output	2	Burst type
m_axi_arlock	Output	1	Lock type
m_axi_arcache	Output	4	Cache type
m_axi_arprot	Output	3	Protection type
m_axi_arqos	Output	4	Quality of service
m_axi_arregion	Output	4	Region identifier
m_axi_aruser	Output	AXI_USER_WIDTH	User signal
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready

8.3.3 AXI4 Read Data Channel (R)

Port	Direction	Width	Description
m_axi_rid	Input	AXI_ID_WIDTH	Read data ID
m_axi_rdata	Input	AXI_DATA_WIDTH	Read data
m_axi_rresp	Input	2	Read response
m_axi_rlast	Input	1	Read last

Port	Direction	Width	Description
m_axi_ruser	Input	AXI_USER_WIDTH	User signal
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready

8.3.4 AR Packet Interface

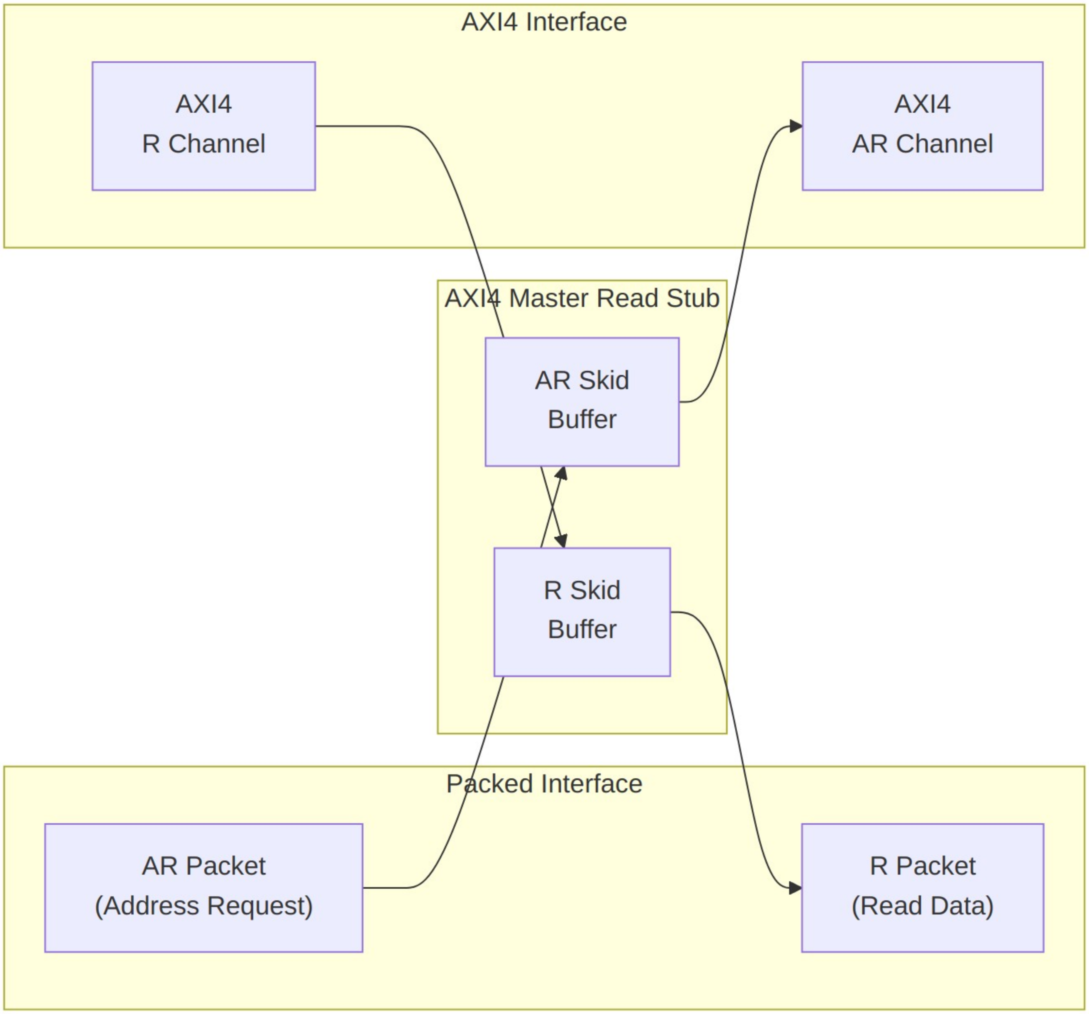
Port	Direction	Width	Description
fub_axi_arvalid	Input	1	AR packet valid
fub_axi_arready	Output	1	Ready to accept AR packet
fub_axi_ar_count	Output	4	AR skid occupancy (driven since the round_20 fix)
fub_axi_ar_pkt	Input	ARSize	Packed AR packet data

8.3.5 R Packet Interface

Port	Direction	Width	Description
fub_axi_rvalid	Output	1	R packet valid
fub_axi_rready	Input	1	Ready to accept R packet
fub_axi_r_pkt	Output	RSize	Packed R packet data

8.4 Functional Description

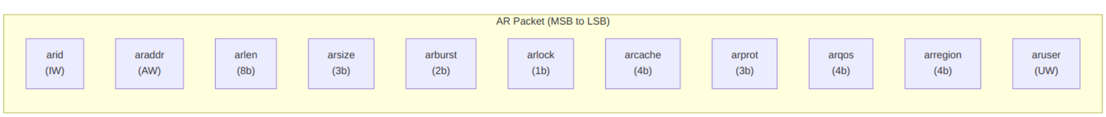
Two skid buffers do all the real work here. The AR buffer unpacks your packet onto the AXI4 AR channel; the R buffer packs the returning read data back into a packet. Everything else is wiring.



Architecture

8.4.1 Packet Formats

Packets are packed MSB-to-LSB in AXI signal order, so building one in the testbench is a single concatenation.

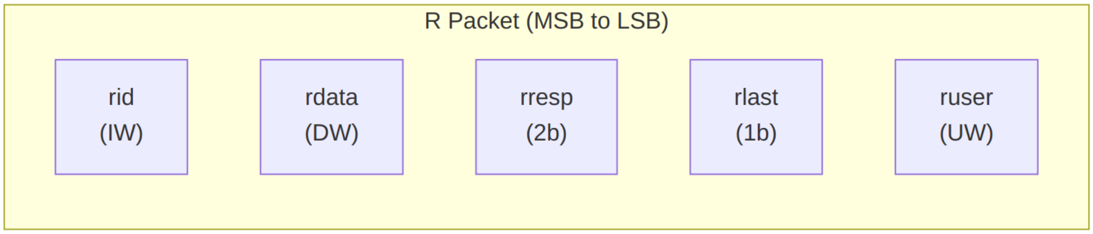


AR Packet Structure (Read Address)

Bit Positions:

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
```

```
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

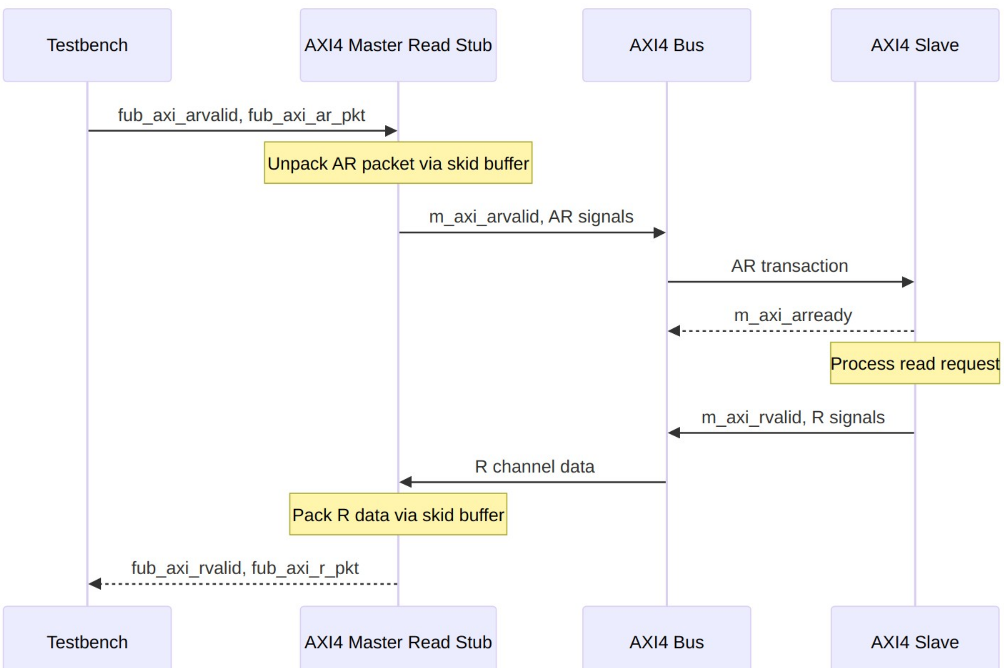


R Packet Structure (Read Data)

Bit Positions:

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
```

```
Width = IW + DW + 2 + 1 + UW
```



Transaction Flow

8.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axi_arvalid, fub_axi_arready, fub_axi_ar_pkt
- AXI AR signals (m_axi_arvalid, m_axi_araddr, m_axi_arlen, etc.)
- AXI R signals (m_axi_rvalid, m_axi_rdata, m_axi_rlast, etc.)
- fub_axi_rvalid, fub_axi_rready, fub_axi_r_pkt
- Packet-to-AXI timing relationship with skid buffer operation

---

8.6 Usage Examples

```
axi4_master_rd_stub #(
    .SKID_DEPTH_AR    (2),
    .SKID_DEPTH_R     (4),
    .AXI_ID_WIDTH     (8),
    .AXI_ADDR_WIDTH   (32),
    .AXI_DATA_WIDTH   (64),
    .AXI_USER_WIDTH   (4)
) u_axi4_master_rd_stub (
    .aclk             (axi_clk),
    .aresetn          (axi_rst_n),

    // AXI4 master read interface
    .m_axi_arid       (m_axi_arid),
    .m_axi_araddr     (m_axi_araddr),
    .m_axi_arlen      (m_axi_arlen),
    .m_axi_arsize     (m_axi_arsize),
    .m_axi_arburst    (m_axi_arburst),
    .m_axi_arlock     (m_axi_arlock),
    .m_axi_arcache    (m_axi_arcache),
    .m_axi_arprot     (m_axi_arprot),
    .m_axi_arqos      (m_axi_arqos),
    .m_axi_arregion   (m_axi_arregion),
    .m_axi_aruser     (m_axi_aruser),
    .m_axi_arvalid    (m_axi_arvalid),
    .m_axi_arready    (m_axi_arready),

    .m_axi_rid        (m_axi_rid),
    .m_axi_rdata      (m_axi_rdata),
    .m_axi_rresp      (m_axi_rresp),
    .m_axi_rlast      (m_axi_rlast),
```

```
    .m_axi_ruser     (m_axi_ruser),
    .m_axi_rvalid    (m_axi_rvalid),
    .m_axi_rready    (m_axi_rready),

    // Packed AR interface
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    // Packed R interface
    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Build AR packet (single beat read at address 0x1000)
localparam ARSize = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + 4;  //
Calculate size
assign tb_ar_pkt = {
    8'd0,            // arid
    32'h0000_1000,   // araddr
    8'd0,            // arlen (1 beat)
    3'b011,          // arsize (8 bytes)
    2'b01,           // arburst (INCR)
    1'b0,            // arlock
    4'b0011,         // arcache
    3'b000,          // arprot
    4'b0000,         // arqos
    4'b0000,         // arregion
    4'h0             // aruser
};

// Parse R packet
localparam int RSize = 8+64+2+1+4;  // match your instance widths
wire [7:0]  r_id   = tb_r_pkt[RSize-1:RSize-8];
wire [63:0] r_data = tb_r_pkt[RSize-9:RSize-72];
wire [1:0]  r_resp = tb_r_pkt[6:5];
wire        r_last = tb_r_pkt[4];
wire [3:0]  r_user = tb_r_pkt[3:0];
```

8.7 Design Notes

8.7.1 Skid Buffer Operation

The stub is only as smart as its `gaxi_skid_buffer` instances, which is exactly the point. They:

- Decouple timing between testbench and AXI bus
- Provide configurable buffering depth per channel
- Handle backpressure gracefully
- Support burst transactions without stalling

Recommended Depths: - **AR Channel:** 2-4 (address transactions) - **R Channel:** 4-8 (data beats for bursts)

8.7.2 Packet Packing Order

AR and R packets are packed MSB-to-LSB following AXI signal order:

- Simplifies testbench packet creation
- Matches common concatenation order
- Efficient for burst transaction handling

8.7.3 Internal Architecture

The stub instantiates two `gaxi_skid_buffer` modules:

- **AR Skid Buffer:** Unpacks AR packets to AXI AR channel
- **R Skid Buffer:** Packs AXI R channel to R packets

All AXI protocol handling is done by the skid buffers and downstream modules.

8.8 Related Modules

- AXI4 Master Read - Full AXI4 master read module (if wrapping one)
- AXI4 Master Write Stub - Corresponding write stub
- AXI4 Master Stub - Combined read/write stub
- AXI4 Slave Read Stub - Slave-side read stub

8.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

8.10 Navigation

- ← **Back to AXI4 Index**
- ← **Back to rtl-amba Index**
- ← **Back to Main Documentation Index**

9 AXI4 Master Stub

Module: `axi4_master_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

9.1 Overview

Why instantiate two stubs when one will do? The AXI4 Master Stub combines the read and write stubs into a single module — it internally instantiates `axi4_master_rd_stub` and `axi4_master_wr_stub` — giving you packed packet interfaces on all five AXI4 channels behind one complete AXI4 master port. It's built for testbenches and integration scenarios where packet-based control beats wrangling individual AXI signals.

9.1.1 Key Features

- Combined read and write channel support
- Packed packet interfaces for all channels (AW, W, B, AR, R)
- Configurable skid buffer depths per channel
- Full AXI4 protocol support (bursts, IDs, user signals)
- Simplified testbench integration
- Parameterized data widths

9.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_W	int	4	W channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_B	int	2	B channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	R channel skid buffer depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)

Parameter	Type	Default	Description
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

9.3 Ports

9.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	AXI reset (active low)

9.3.2 AXI4 Write Channels

Port	Direction	Width	Description
m_axi_awid	Output	AXI_ID_WIDTH	Write address ID
m_axi_awaddr	Output	AXI_ADDR_WIDTH	Write address
m_axi_awlen	Output	8	Burst length
m_axi_awsize	Output	3	Burst size
m_axi_awburst	Output	2	Burst type
m_axi_awlock	Output	1	Lock type
m_axi_awcache	Output	4	Cache type
m_axi_awprot	Output	3	Protection type
m_axi_awqos	Output	4	Quality of service
m_axi_awregio	Output	4	Region

Port	Direction	Width	Description
n			identifier
m_axi_awuser	Output	AXI_USER_WIDTH	User signal
m_axi_awvalid	Output	1	Write address valid
m_axi_awready	Input	1	Write address ready
m_axi_wdata	Output	AXI_DATA_WIDTH	Write data
m_axi_wstrb	Output	AXI_WSTRB_WIDTH	Write strobes
m_axi_wlast	Output	1	Write last
m_axi_wuser	Output	AXI_USER_WIDTH	User signal
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready
m_axi_bid	Input	AXI_ID_WIDTH	Response ID
m_axi_bresp	Input	2	Write response
m_axi_buser	Input	AXI_USER_WIDTH	User signal
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready

9.3.3 AXI4 Read Channels

Port	Direction	Width	Description
m_axi_arid	Output	AXI_ID_WIDTH	Read address ID
m_axi_araddr	Output	AXI_ADDR_WIDTH	Read address

Port	Direction	Width	Description
m_axi_arlen	Output	8	Burst length
m_axi_arsize	Output	3	Burst size
m_axi_arburst	Output	2	Burst type
m_axi_arlock	Output	1	Lock type
m_axi_arcache	Output	4	Cache type
m_axi_arprot	Output	3	Protection type
m_axi_arqos	Output	4	Quality of service
m_axi_arregion	Output	4	Region identifier
m_axi_aruser	Output	AXI_USER_WIDTH	User signal
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready
m_axi_rid	Input	AXI_ID_WIDTH	Read data ID
m_axi_rdata	Input	AXI_DATA_WIDTH	Read data
m_axi_rresp	Input	2	Read response
m_axi_rlast	Input	1	Read last
m_axi_ruser	Input	AXI_USER_WIDTH	User signal
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready

9.3.4 Write Packet Interfaces

Port	Direction	Width	Description
fub_axi_awvalid	Input	1	AW packet valid

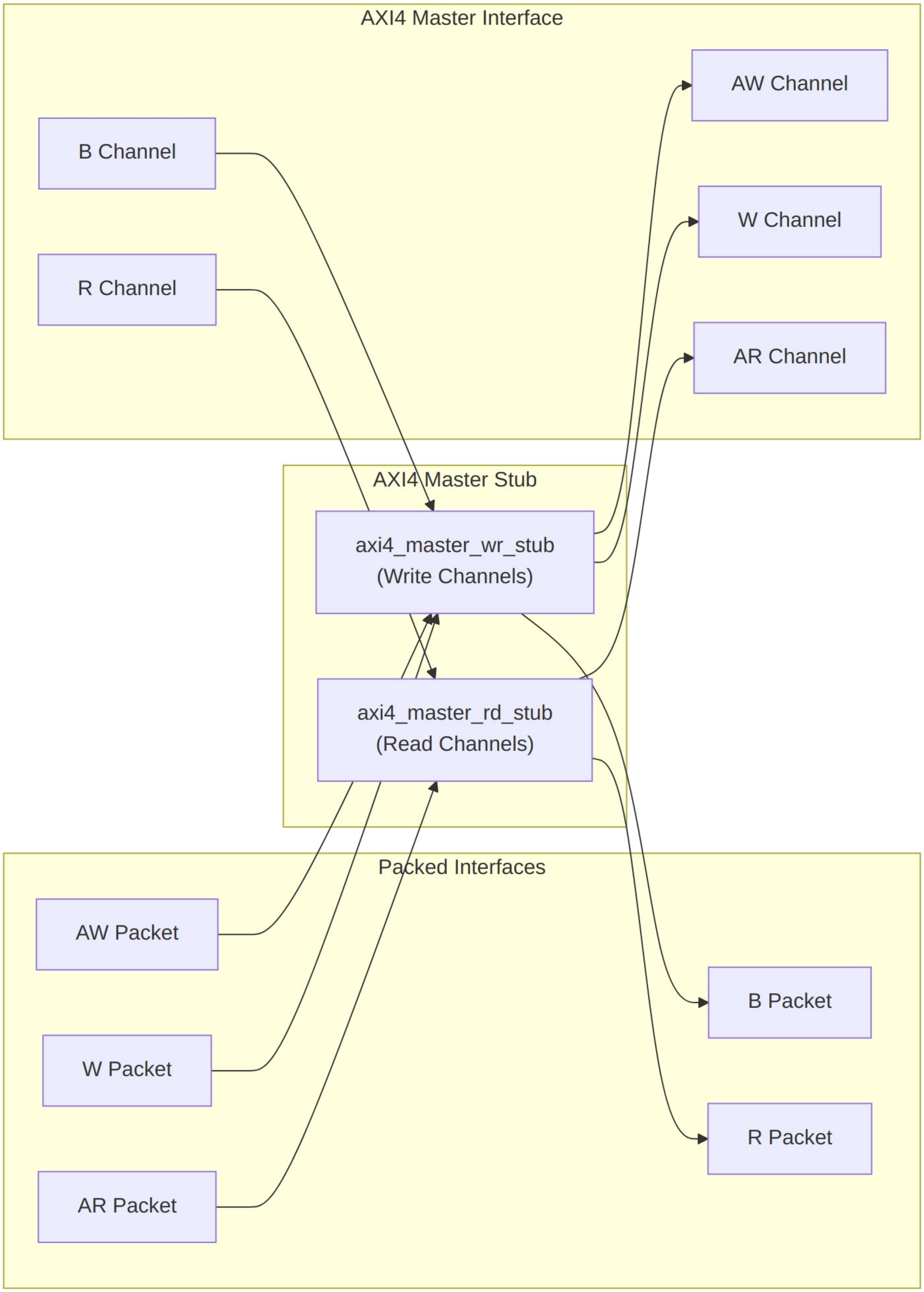
Port	Direction	Width	Description
fub_axi_awready	Output	1	Ready to accept AW packet
fub_axi_aw_count	Output	4	AW buffer occupancy
fub_axi_aw_pkt	Input	AWSize	Packed AW packet data
fub_axi_wvalid	Input	1	W packet valid
fub_axi_wready	Output	1	Ready to accept W packet
fub_axi_w_pkt	Input	WSize	Packed W packet data
fub_axi_bvalid	Output	1	B packet valid
fub_axi_bready	Input	1	Ready to accept B packet
fub_axi_b_pkt	Output	BSize	Packed B packet data

9.3.5 Read Packet Interfaces

Port	Direction	Width	Description
fub_axi_arvalid	Input	1	AR packet valid
fub_axi_arready	Output	1	Ready to accept AR packet
fub_axi_ar_count	Output	4	AR skid occupancy (driven since the round_20 fix; was an undriven 3-bit port)
fub_axi_ar_pkt	Input	ARSize	Packed AR packet data
fub_axi_rvalid	Output	1	R packet valid

Port	Direction	Width	Description
fub_axi_rready	Input	1	Ready to accept R packet
fub_axi_r_pkt	Output	RSize	Packed R packet data

9.4 Functional Description



Architecture

9.4.1 Packet Formats

9.4.1.1 Write Channel Packets

AW Packet (Write Address):

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

W Packet (Write Data):

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}
Width = DW + SW + 1 + UW
```

B Packet (Write Response):

```
fub_axi_b_pkt = {bid, bresp, buser}
Width = IW + 2 + UW
```

9.4.1.2 Read Channel Packets

AR Packet (Read Address):

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

R Packet (Read Data):

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
Width = IW + DW + 2 + 1 + UW
```

Complete packet format details: See AXI4 Master Write Stub and AXI4 Master Read Stub

9.4.2 Transaction Flow

Read and write run independently and can overlap — the sequence below shows both in flight at once.

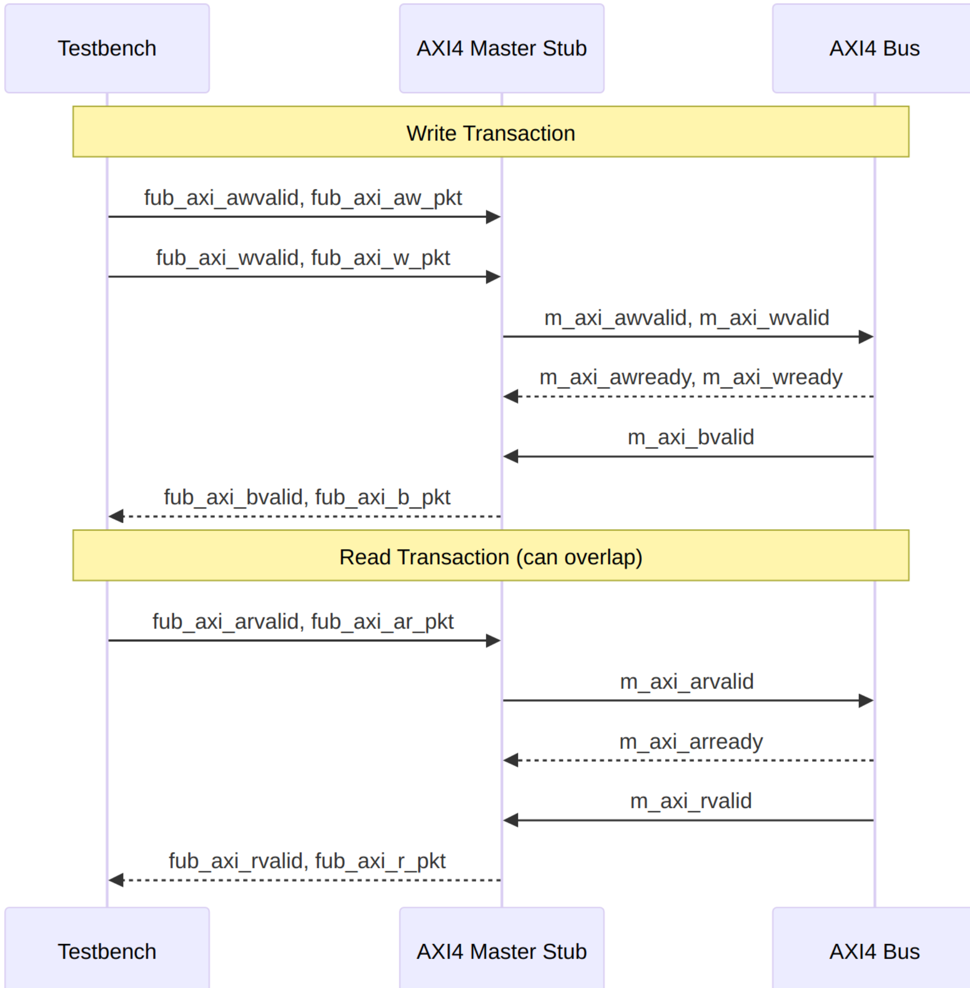


Diagram 11

9.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- Write packet interfaces (fub_axi_aw, *fub_axi_w*, fub_axi_b*)
- Read packet interfaces (fub_axi_ar, *fub_axi_r*)
- AXI write channels (m_axi_aw, *m_axi_w*, m_axi_b*)
- AXI read channels (m_axi_ar, *m_axi_r*)
- Overlapping read/write operations

9.6 Usage Examples

```
axi4_master_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_master_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 master interface (all channels)
    .m_axi_awid      (m_axi_awid),
    .m_axi_awaddr    (m_axi_awaddr),
    .m_axi_awlen     (m_axi_awlen),
    .m_axi_awsize    (m_axi_awsize),
    .m_axi_awburst   (m_axi_awburst),
    .m_axi_awlock    (m_axi_awlock),
    .m_axi_awcache   (m_axi_awcache),
    .m_axi_awprot    (m_axi_awprot),
    .m_axi_awqos     (m_axi_awqos),
    .m_axi_awregion  (m_axi_awregion),
    .m_axi_awuser    (m_axi_awuser),
    .m_axi_awvalid   (m_axi_awvalid),
    .m_axi_awready   (m_axi_awready),

    .m_axi_wdata     (m_axi_wdata),
    .m_axi_wstrb     (m_axi_wstrb),
    .m_axi_wlast     (m_axi_wlast),
    .m_axi_wuser     (m_axi_wuser),
    .m_axi_wvalid    (m_axi_wvalid),
    .m_axi_wready    (m_axi_wready),

    .m_axi_bid       (m_axi_bid),
    .m_axi_bresp     (m_axi_bresp),
    .m_axi_buser     (m_axi_buser),
    .m_axi_bvalid    (m_axi_bvalid),
    .m_axi_bready    (m_axi_bready),

    .m_axi_arid      (m_axi_arid),
```

```
    .m_axi_araddr    (m_axi_araddr),
    .m_axi_arlen     (m_axi_arlen),
    .m_axi_arsize    (m_axi_arsize),
    .m_axi_arburst   (m_axi_arburst),
    .m_axi_arlock    (m_axi_arlock),
    .m_axi_arcache   (m_axi_arcache),
    .m_axi_arprot    (m_axi_arprot),
    .m_axi_arqos     (m_axi_arqos),
    .m_axi_arregion  (m_axi_arregion),
    .m_axi_aruser    (m_axi_aruser),
    .m_axi_arvalid   (m_axi_arvalid),
    .m_axi_arready   (m_axi_arready),

    .m_axi_rid       (m_axi_rid),
    .m_axi_rdata     (m_axi_rdata),
    .m_axi_rresp     (m_axi_rresp),
    .m_axi_rlast     (m_axi_rlast),
    .m_axi_ruser     (m_axi_ruser),
    .m_axi_rvalid    (m_axi_rvalid),
    .m_axi_rready    (m_axi_rready),

    // Write packet interfaces
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt),

    // Read packet interfaces
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);
```

```
// Example: Build write transaction packets
assign tb_aw_pkt = {
    8'd1,            // awid
    32'h1000_0000,   // awaddr
    8'd0,            // awlen (1 beat)
    3'b011,          // awsize (8 bytes)
    2'b01,           // awburst (INCR)
    1'b0,            // awlock
    4'b0011,         // awcache
    3'b000,          // awprot
    4'b0000,         // awqos
    4'b0000,         // awregion
    4'h0             // awuser
};

assign tb_w_pkt = {
    64'hDEAD_BEEF_CAFE_BABE,  // wdata
    8'hFF,                     // wstrb
    1'b1,                      // wlast
    4'h0                       // wuser
};

// Example: Build read transaction packet
assign tb_ar_pkt = {
    8'd2,            // arid
    32'h2000_0000,   // araddr
    8'd3,            // arlen (4 beats)
    3'b011,          // arsize (8 bytes)
    2'b01,           // arburst (INCR)
    1'b0,            // arlock
    4'b0011,         // arcache
    3'b000,          // arprot
    4'b0000,         // arqos
    4'b0000,         // arregion
    4'h0             // aruser
};

// Parse responses
localparam int BSize = 8 + 2 + 4;        // IW + resp + UW -- match
your instance
localparam int RSize = 8 + 64 + 2 + 1 + 4; // IW + DW + resp + last +
UW
wire [7:0] b_id   = tb_b_pkt[BSize-1:BSize-8];
wire [1:0] b_resp = tb_b_pkt[5:4];

wire [7:0]  r_id   = tb_r_pkt[RSize-1:RSize-8];
wire [63:0] r_data = tb_r_pkt[RSize-9:RSize-72];
```

```
wire [1:0]  r_resp = tb_r_pkt[6:5];
wire        r_last = tb_r_pkt[4];
```

9.7 Design Notes

9.7.1 Internal Architecture

The stub instantiates two sub-modules:

- **`axi4_master_wr_stub`** - Handles AW, W, and B channels
- **`axi4_master_rd_stub`** - Handles AR and R channels

This hierarchical design:

- Reuses proven read/write stub modules
- Maintains clear channel separation
- Simplifies verification and testing
- Allows independent read/write operation

9.7.2 Read/Write Independence

Read and write channels are completely independent:

- Can overlap in time (simultaneous read and write)
- Each has independent skid buffers
- No ordering constraints between reads and writes
- Testbench can drive at different rates

9.7.3 Skid Buffer Depths

Recommended configurations:

Low latency (fast memory):

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(2), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(2)
```

Typical system:

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(4), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(4)
```

High throughput bursts:

```
.SKID_DEPTH_AW(4), .SKID_DEPTH_W(8), .SKID_DEPTH_B(4),
.SKID_DEPTH_AR(4), .SKID_DEPTH_R(8)
```

9.8 Related Modules

- **AXI4 Master Read Stub** - Read-only stub (instantiated internally)
- **AXI4 Master Write Stub** - Write-only stub (instantiated internally)
- **AXI4 Slave Stub** - Corresponding combined slave stub
- **AXI4 Master Read** - Full AXI4 master read module
- **AXI4 Master Write** - Full AXI4 master write module

9.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

9.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

10 AXI4 Master Write

Module: `axi4_master_wr.sv` **Location:** `rtl/amba/axi4/` **Status:** Production Ready

10.1 Overview

Think of this module as an elastic buffer that speaks fluent AXI4. The AXI4 Master Write sits between a write initiator and an AXI4 interconnect with an independent, configurable-depth skid buffer on each of the AW, W, and B channels, so timing on one side never leaks into the other and backpressure always has somewhere to park.

10.1.1 Key Features

- **Full AXI4 Write Support:** Complete AW, W, and B channel implementation
- **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- **Elastic Buffering:** Decouples upstream and downstream timing domains
- **Burst Support:** Full burst transaction handling with WLAST tracking
- **User Signal Support:** Carries AWUSER, WUSER, and BUSER signals
- **Clock Gating Support:** Busy signal for dynamic power management

---

10.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	Depth of write address (AW) channel skid buffer
`SKID_DEPTH_W`	int	4	Depth of write data (W) channel skid buffer
`SKID_DEPTH_B`	int	2	Depth of write response (B) channel skid buffer
`AXI_ID_WIDTH`	int	8	Width of transaction ID signals (AWID, BID)
`AXI_ADDR_WIDTH`	int	32	Width of address bus (AWADDR)
`AXI_DATA_WIDTH`	int	32	Width of data bus (WDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024
`AXI_USER_WIDTH`	int	1	Width of user-defined signals (AWUSER, WUSER, BUSER)

10.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
AWSize	IW+AW+8+3+2+1+4+3+4+4+UW
WSize	DW+SW+1+UW
BSize	IW+2+UW

10.3 Ports

The full port list, straight from the RTL:

```
module axi4_master_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1
) (
    // Clock and Reset
    input  logic                       aclk,
    input  logic                       aresetn,

    // Frontend AXI Interface (fub_axi_*)
    // Write address channel (AW)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_awid,
    input  logic [AXI_ADDR_WIDTH-1:0] fub_axi_awaddr,
    input  logic [7:0]                 fub_axi_awlen,
    input  logic [2:0]                 fub_axi_awsize,
    input  logic [1:0]                 fub_axi_awburst,
    input  logic                       fub_axi_awlock,
    input  logic [3:0]                 fub_axi_awcache,
    input  logic [2:0]                 fub_axi_awprot,
    input  logic [3:0]                 fub_axi_awqos,
    input  logic [3:0]                 fub_axi_awregion,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_awuser,
    input  logic                       fub_axi_awvalid,
    output logic                       fub_axi_awready,
```

```
    // Write data channel (W)
    input  logic [AXI_DATA_WIDTH-1:0] fub_axi_wdata,
    input  logic [AXI_DATA_WIDTH/8-1:0] fub_axi_wstrb,
    input  logic                       fub_axi_wlast,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_wuser,
    input  logic                       fub_axi_wvalid,
    output logic                       fub_axi_wready,

    // Write response channel (B)
    output logic [AXI_ID_WIDTH-1:0]   fub_axi_bid,
    output logic [1:0]                 fub_axi_bresp,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_buser,
    output logic                       fub_axi_bvalid,
    input  logic                       fub_axi_bready,

    // Master AXI Interface (m_axi_*)
    // Write address channel (AW)
    output logic [AXI_ID_WIDTH-1:0]   m_axi_awid,
    output logic [AXI_ADDR_WIDTH-1:0] m_axi_awaddr,
    output logic [7:0]                 m_axi_awlen,
    output logic [2:0]                 m_axi_awsize,
    output logic [1:0]                 m_axi_awburst,
    output logic                       m_axi_awlock,
    output logic [3:0]                 m_axi_awcache,
    output logic [2:0]                 m_axi_awprot,
    output logic [3:0]                 m_axi_awqos,
    output logic [3:0]                 m_axi_awregion,
    output logic [AXI_USER_WIDTH-1:0] m_axi_awuser,
    output logic                       m_axi_awvalid,
    input  logic                       m_axi_awready,

    // Write data channel (W)
    output logic [AXI_DATA_WIDTH-1:0] m_axi_wdata,
    output logic [AXI_DATA_WIDTH/8-1:0] m_axi_wstrb,
    output logic                       m_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0] m_axi_wuser,
    output logic                       m_axi_wvalid,
    input  logic                       m_axi_wready,

    // Write response channel (B)
    input  logic [AXI_ID_WIDTH-1:0]   m_axi_bid,
    input  logic [1:0]                 m_axi_bresp,
    input  logic [AXI_USER_WIDTH-1:0] m_axi_buser,
    input  logic                       m_axi_bvalid,
    output logic                       m_axi_bready,
```

```
    // Status
    output logic                        busy
);
```

10.3.1 Clock and Reset

Port	Direction	Width	Description
`aclk`	Input	1	AXI clock - all signals sampled on rising edge
`aresetn`	Input	1	Active-low asynchronous reset

10.3.2 Frontend AXI Interface (fub_axi_*)

Write Address Channel (AW)

Port	Direction	Width	Description
`fub_axi_awid`	Input	`AXI_ID_WIDTH`	Write transaction ID
`fub_axi_awaddr`	Input	`AXI_ADDR_WIDTH`	Write address
`fub_axi_awlen`	Input	8	Burst length, AXI-encoded: beats - 1 (`0x00` = 1 beat, `0xFF` = 256)
`fub_axi_awsize`	Input	3	Burst size (bytes per beat)
`fub_axi_awburst`	Input	2	Burst type (FIXED, INCR, WRAP)
`fub_axi_awlock`	Input	1	Lock type (atomic access support)
`fub_axi_awcache`	Input	4	Cache attributes
`fub_axi_awprot`	Input	3	Protection attributes
`fub_axi_awqos`	Input	4	Quality of Service identifier
`fub_axi_awregion`	Input	4	Region identifier

Port	Direction	Width	Description
`fub_axi_awuser`	Input	`AXI_USER_WIDTH`	User-defined signal
`fub_axi_awvalid`	Input	1	Write address valid
`fub_axi_awready`	Output	1	Write address ready

Write Data Channel (W)

Port	Direction	Width	Description
`fub_axi_wdata`	Input	`AXI_DATA_WIDTH`	Write data
`fub_axi_wstrb`	Input	`AXI_DATA_WIDTH/8`	Write strobes (byte enables)
`fub_axi_wlast`	Input	1	Last beat of burst indicator
`fub_axi_wuser`	Input	`AXI_USER_WIDTH`	User-defined signal
`fub_axi_wvalid`	Input	1	Write data valid
`fub_axi_wready`	Output	1	Write data ready

Write Response Channel (B)

Port	Direction	Width	Description
`fub_axi_bid`	Output	`AXI_ID_WIDTH`	Response transaction ID
`fub_axi_bresp`	Output	2	Write response (OKAY, EXOKAY, SLVERR, DECERR)
`fub_axi_buser`	Output	`AXI_USER_WIDTH`	User-defined signal
`fub_axi_bvalid`	Output	1	Write response valid
`fub_axi_bready`	Input	1	Write response ready

10.3.3 Master AXI Interface (m_axi_*)

Mirrors the frontend interface but in the opposite direction (output on AW/W, input on B).

10.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions in buffers (for clock gating)

10.4 Functional Description

10.4.1 Architecture

Three independent `gaxi_skid_buffer` instances provide the elastic buffering, one per write channel:

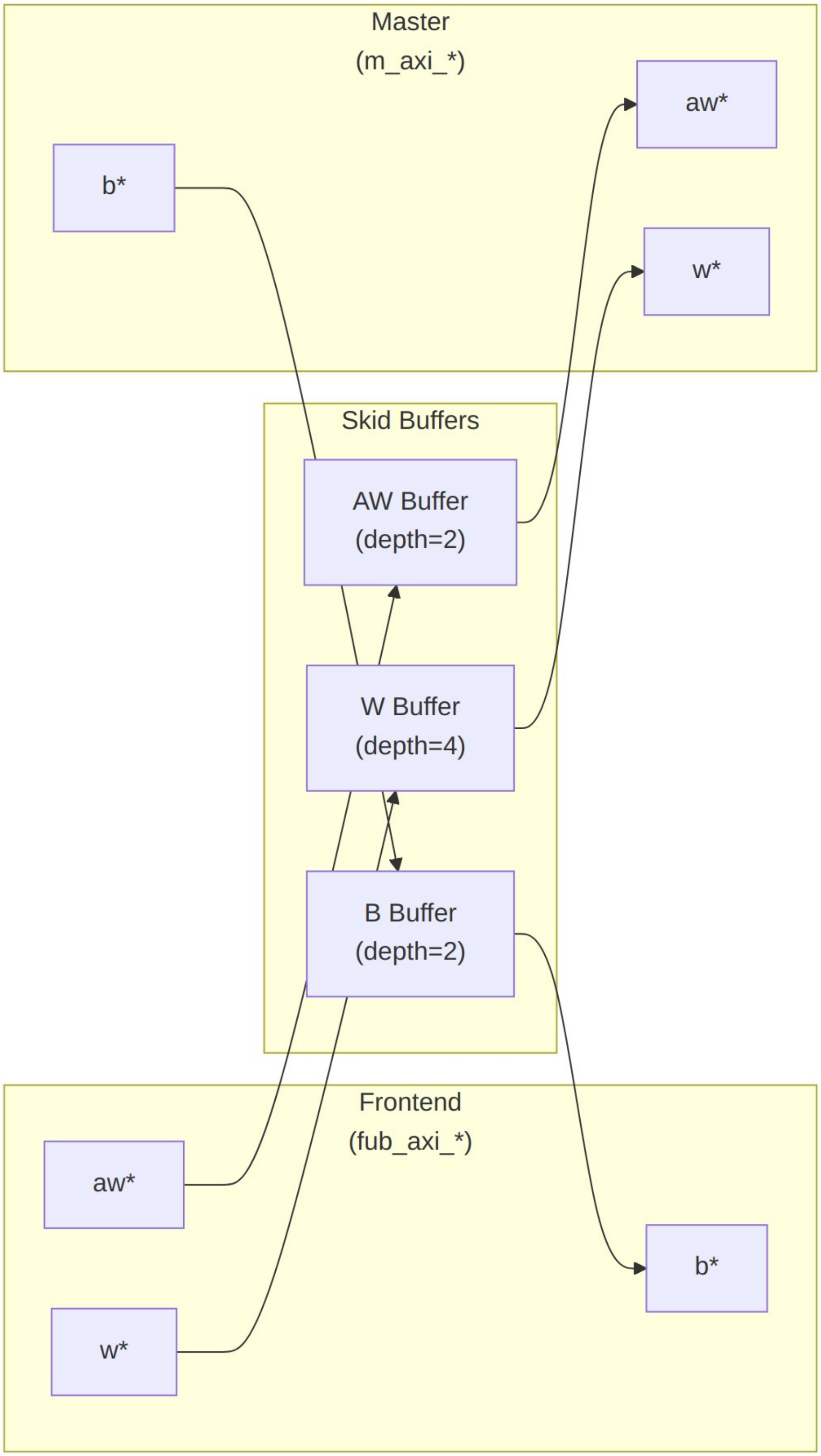

Master
(m_axi_*)
aw*
b*
w*
Skid Buffers
AW Buffer
(depth=2)
W Buffer
(depth=4)
B Buffer
(depth=2)
Frontend
(fub_axi_*)
aw*
b*
w*

Diagram 12

10.4.2 Channel Operations

10.4.2.1 *Write Address (AW) Channel*

1. Frontend presents write address and attributes
2. AW skid buffer accepts when space available (`fub_axi_awready` high)
3. Buffered address presented to master interface when ready
4. Configurable depth (`SKID_DEPTH_AW`) smooths timing variations

10.4.2.2 *Write Data (W) Channel*

1. Frontend presents write data with strobes
2. W skid buffer provides deeper buffering (default depth=4)
3. WLAST signal preserved to indicate burst boundaries
4. Independent from AW channel (AXI4 allows W before AW)

10.4.2.3 *Write Response (B) Channel*

1. Master returns write response with transaction ID
2. B skid buffer captures response when frontend busy
3. Frontend receives response when ready to accept
4. ID matching handled by AXI interconnect (not this module)

10.4.3 Busy Signal

The `busy` output indicates active transactions:

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count
> 0) ||
                fub_axi_awvalid || fub_axi_wvalid || m_axi_bvalid;
```

Use cases: - **Clock gating:** Disable clock when `busy` is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

10.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the**

unstalled case – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

10.6 Usage Examples

10.6.1 Basic Integration

```
// Instantiate AXI4 master write buffer
axi4_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_master_wr (
    .aclk                   (axi_aclk),
    .aresetn                (axi_aresetn),

    // Frontend interface (from write initiator)
    .fub_axi_awid           (write_awid),
    .fub_axi_awaddr         (write_awaddr),
    .fub_axi_awlen          (write_awlen),
    .fub_axi_awsize         (write_awsize),
    .fub_axi_awburst        (write_awburst),
    .fub_axi_awlock         (1'b0),
    .fub_axi_awcache        (4'b0010),
    .fub_axi_awprot         (3'b000),
    .fub_axi_awqos          (4'b0000),
    .fub_axi_awregion       (4'b0000),
    .fub_axi_awuser         (1'b0),
    .fub_axi_awvalid        (write_awvalid),
    .fub_axi_awready        (write_awready),

    .fub_axi_wdata          (write_wdata),
    .fub_axi_wstrb          (write_wstrb),
    .fub_axi_wlast          (write_wlast),
    .fub_axi_wuser          (1'b0),
    .fub_axi_wvalid         (write_wvalid),
```

```
    .fub_axi_wready     (write_wready),

    .fub_axi_bid        (write_bid),
    .fub_axi_bresp      (write_bresp),
    .fub_axi_buser      (write_buser),
    .fub_axi_bvalid     (write_bvalid),
    .fub_axi_bready     (write_bready),

    // Master interface (to AXI interconnect)
    .m_axi_awid         (m_axi_awid),
    .m_axi_awaddr       (m_axi_awaddr),
    .m_axi_awlen        (m_axi_awlen),
    .m_axi_awsize       (m_axi_awsize),
    .m_axi_awburst      (m_axi_awburst),
    .m_axi_awlock       (m_axi_awlock),
    .m_axi_awcache      (m_axi_awcache),
    .m_axi_awprot       (m_axi_awprot),
    .m_axi_awqos        (m_axi_awqos),
    .m_axi_awregion     (m_axi_awregion),
    .m_axi_awuser       (m_axi_awuser),
    .m_axi_awvalid      (m_axi_awvalid),
    .m_axi_awready      (m_axi_awready),

    .m_axi_wdata        (m_axi_wdata),
    .m_axi_wstrb        (m_axi_wstrb),
    .m_axi_wlast        (m_axi_wlast),
    .m_axi_wuser        (m_axi_wuser),
    .m_axi_wvalid       (m_axi_wvalid),
    .m_axi_wready       (m_axi_wready),

    .m_axi_bid          (m_axi_bid),
    .m_axi_bresp        (m_axi_bresp),
    .m_axi_buser        (m_axi_buser),
    .m_axi_bvalid       (m_axi_bvalid),
    .m_axi_bready       (m_axi_bready),

    // Status
    .busy               (wr_master_busy)
);
```

10.6.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .clk_in            (axi_clk),
```

```
    .aresetn           (axi_resetn),
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),
    .wakeup            (wr_master_busy),
    .clk_out           (axi_clk_gated),
    .gating            ()
);

// Connect module to gated clock
axi4_master_wr #( ... ) u_master_wr (
    .aclk (axi_clk_gated),
    ...
);
```

10.7 Design Notes

10.7.1 Buffer Depth Selection

`SKID_DEPTH_*` is an entry count, not a log2 exponent. The underlying `gaxi_skid_buffer` allocates one register slot per entry and tracks occupancy with a 4-bit counter, so legal values are 2..8 inclusive (any integer). Values greater than 8 overflow the occupancy counter and are not supported.

Write Address (SKID_DEPTH_AW): - Default: 2 (sufficient for most cases) - Increase if: - High-latency address decode paths - Frequent address channel backpressure - Multiple outstanding bursts needed

Write Data (SKID_DEPTH_W): - Default: 4 (deeper than AW for burst data) - Increase if: - Large burst sizes (AWLEN > 4) - High-bandwidth streaming writes - Significant W channel backpressure

Write Response (SKID_DEPTH_B): - Default: 2 (responses are typically single-beat) - Increase if: - Frontend slow to accept responses - Many concurrent outstanding writes - Response channel backpressure observed

10.7.2 Recommended Configurations

Low-Latency System (single outstanding write):

```
axi4_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_master_wr ( ... );
```

High-Throughput Streaming (burst writes):

```
axi4_master_wr #(
    .SKID_DEPTH_AW(4),
    .SKID_DEPTH_W(8),    // Deep for burst data
    .SKID_DEPTH_B(4)
) u_master_wr ( ... );
```

Multiple Outstanding Writes:

```
axi4_master_wr #(
    .SKID_DEPTH_AW(8),
    .SKID_DEPTH_W(8),
    .SKID_DEPTH_B(8)
) u_master_wr ( ... );
```

10.7.3 Buffer Independence

The three skid buffers operate independently: - AW and W channels can progress at different rates - AXI4 allows W data before AW address - B responses return asynchronously based on slave timing

10.7.4 Packet Preservation

All signal groups packed and unpacked atomically: - AW channel: `{awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser}` - W channel: `{wdata, wstrb, wlast, wuser}` - B channel: `{bid, bresp, buser}`

10.7.5 Backpressure Handling

Ready signals propagate backpressure: - Frontend sees `awready`/`wready` low when buffers full - Master backpressure captured in buffers - Prevents data loss while decoupling timing

10.7.6 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

10.8 Related Modules

10.8.1 Companion Modules

- axi4_master_rd - AXI4 master read with buffering
- **axi4_master_wr_cg** - Clock-gated variant with additional CG logic
- **axi4_master_wr_mon** - Write transaction monitor for verification

10.8.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

10.8.3 Related Infrastructure

- **axi4_interconnect** - Multi-master/multi-slave crossbar
- **axi4_slave_wr** - Corresponding AXI4 write slave module

10.9 Testing

`val/amba/test_axi4_master_wr.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_wr.py -v
```

10.10 References

10.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

10.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_master_wr.sv`
- Tests: `val/amba/test_axi4_master_wr.py`
- Framework: `bin/TBClasses/components/axi4/`

10.10.3 Documentation

- Architecture: rtl-amba Overview
- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

10.11 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

11 AXI4 Master Write Interface (Clock-Gated)

Module: `axi4_master_wr_cg.sv` **Base Module:** axi4_master_wr **Location:** `rtl/amba/axi4/` **Status:** Production Ready

---

11.1 Overview

Same module, smaller power bill. This is the **clock-gated variant** of axi4_master_wr: functionally identical, with activity-based clock gating wrapped around it.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the **Clock-Gated Variants Guide**.

What the wrapper buys you:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable at runtime:** `cfg_cg_enable` / `cfg_cg_idle_count` inputs
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate

11.2 Parameters

In addition to all axi4_master_wr parameters:

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`SKID_DEPTH_AW`	2	Skid-buffer depth on the AW channel. Legal range 2..8 inclusive; odd depths are legal.

Parameter	Default	Description
`SKID_DEPTH_B`	2	Skid-buffer depth on the B channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_W`	4	Skid-buffer depth on the W channel. Legal range 2..8 inclusive; odd depths are legal.

The gating controls are RUNTIME INPUTS, not parameters: `cfg_cg_enable` and `cfg_cg_idle_count`; status outputs `cg_gating` / `cg_idle`. One `amba_clock_gate_ctrl` gates the whole module – no per-domain gates. The base module's `busy` output is NOT re-exported (consumed internally as a wake term). (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

11.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`
`AWSize`	`IW+AW+8+3+2+1+4+3+4+4+UW`
`WSize`	`DW+SW+1+UW`
`BSize`	`IW+2+UW`

11.3 Ports

Port	Dir	Width	Description
`aclk`	In	1	
`aresetn`	In	1	

Port	Dir	Width	Description
cfg_cg_enable	In	1	
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_axi_awid	In	[IW-1:0]	
fub_axi_awaddr	In	[AW-1:0]	
fub_axi_awlen	In	[7:0]	
fub_axi_awsize	In	[2:0]	
fub_axi_awburst	In	[1:0]	
fub_axi_awlock	In	1	
fub_axi_awcache	In	[3:0]	
fub_axi_awprot	In	[2:0]	
fub_axi_awqos	In	[3:0]	
fub_axi_awregion	In	[3:0]	
fub_axi_awuser	In	[UW-1:0]	
fub_axi_awvalid	In	1	
fub_axi_awready	Out	1	
fub_axi_wdata	In	[DW-1:0]	
fub_axi_wstrb	In	[SW-1:0]	
fub_axi_wlast	In	1	
fub_axi_wuser	In	[UW-1:0]	
fub_axi_wvalid	In	1	
fub_axi_wready	Out	1	
fub_axi_bid	Out	[IW-1:0]	
fub_axi_bresp	Out	[1:0]	
fub_axi_buser	Out	[UW-1:0]	

Port	Dir	Width	Description
fub_axi_bvalid	Out	1	
fub_axi_bready	In	1	
m_axi_awid	Out	[IW-1:0]	
m_axi_awaddr	Out	[AW-1:0]	
m_axi_awlen	Out	[7:0]	
m_axi_awsize	Out	[2:0]	
m_axi_awburst	Out	[1:0]	
m_axi_awlock	Out	1	
m_axi_awcache	Out	[3:0]	
m_axi_awprot	Out	[2:0]	
m_axi_awqos	Out	[3:0]	
m_axi_awregion	Out	[3:0]	
m_axi_awuser	Out	[UW-1:0]	
m_axi_awvalid	Out	1	
m_axi_awready	In	1	
m_axi_wdata	Out	[DW-1:0]	
m_axi_wstrb	Out	[SW-1:0]	
m_axi_wlast	Out	1	
m_axi_wuser	Out	[UW-1:0]	
m_axi_wvalid	Out	1	
m_axi_wready	In	1	
m_axi_bid	In	[IW-1:0]	
m_axi_bresp	In	[1:0]	
m_axi_buser	In	[UW-1:0]	
m_axi_bvalid	In	1	
m_axi_bready	Out	1	
cg_gating	Out	1	
cg_idle	Out	1	

11.4 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

11.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

11.6 Usage Examples

```
axi4_master_wr_cg #(
    // Base module parameters (see axi4_master_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
```

```
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_master_wr (except busy)
);
```

11.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

11.8 Related Modules

- **Base Module Functionality:** axi4_master_wr.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
    - axi4_master_rd_mon_cg.md (AXI4 monitor)
    - axil4_master_rd_mon_cg.md (AXIL4 monitor)
    - apb4_slave_cg.md (APB interface)

11.9 Testing

`val/amba/test_axi4_master_wr_cg.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_wr_cg.py -v
```

11.10 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

12 AXI4 Master Write Monitor

Module: `axi4_master_wr_mon.sv` **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

12.1 Overview

Some modules move data; this one moves data and tells you how the transfer went. The AXI4 Master Write Monitor pairs a fully functional `axi4_master_wr` with an `axi_monitor_filtered`, so you get a working buffered master write interface plus real-time protocol checking, error detection, performance metrics, and configurable packet filtering on write transactions. It's built for verification environments that need eyes on the bus without getting in the way of it.

12.1.1 Key Features

- **Integrated Monitoring:** Combines `axi4_master_wr` with `axi_monitor_filtered`
- **2-Level Filtering:** packet-type drop masks + per-event masks (err_select is reserved – no routing)
- **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions

- **Timeout Monitoring:** Configurable timeout detection for stuck transactions
- **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
- **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
- **Configuration Validation:** Detects conflicting configuration settings
- **Clock Gating Support:** Busy signal for power management

12.2 Parameters

12.2.1 AXI4 Master Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth
`SKID_DEPTH_W`	int	4	W channel skid buffer depth
`SKID_DEPTH_B`	int	2	B channel skid buffer depth
`AXI_ID_WIDTH`	int	8	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	Address bus width
`AXI_DATA_WIDTH`	int	32	Data bus width
`AXI_USER_WIDTH`	int	1	User signal width

12.2.2 Monitor Parameters

Parameter	Type	Default	Description
`UNIT_ID`	logic [7:0]	8'h01	8-bit unit identifier in monitor packets
`AGENT_ID`	logic	16'h000B	16-bit agent identifier in

Parameter	Type	Default	Description
	[15:0]		monitor packets
`MAX_TRANSACTIONS`	int	16	Maximum concurrent outstanding transactions
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. >**1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
`ACLK_MHZ`	int	100	Clock frequency in MHz – keeps the 1 us tick exact off-100MHz
`CFI_MIN_FREQ_MHZ` / `CFI_MAX_FREQ_MHZ`	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
`ACTIVE_TRANS_THRESHOLD`	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when `cfg_threshold_enable=1`. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing

Parameter	Type	Default	Description
`ENABLE_FILTERING`	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
`ADD_PIPELINE_STAGE`	bit	0	Add register stage for timing closure
`USE_MONITOR`	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
`N_ADDR_RANGES`	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
`ADDR_RANGE_IS_ERROR`	logic [N_ADDR_RANGES-1:0]	`'0`	Per-range flavor: bit i = 0 -> DEBUG range (hit -> AddrMatch), 1 -> ERROR range (allowlist miss -> Error/ADDR_RANGE). Default all-0 (feature inert).

12.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a `generate`-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
`ENABLE_ERROR_LOGIC`	bit	1	Drop the error-detection cone
`ENABLE_TIMEOUT_LOGIC`	bit	1	Drop the timeout cone **and** the `axi_monitor_timeout` instance
`ENABLE_COMPL_LOGIC`	bit	1	Drop the completion cone
`ENABLE_THRESHOLD_LOGIC`	bit	1	Drop the threshold cone
`ENABLE_PERF_LOGIC`	bit	1	Gates only the reporter's legacy perf-packet cone and the two lifetime counters – the window FSM and meters are unconditional
`ENABLE_DEBUG_LOGIC`	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (the reporter's legacy perf cone is built; `ENABLE_PERF_LOGIC` gates THAT cone only, never the measurement window) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

12.2.4 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`

Derived parameter	Default expression
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH

12.3 Ports

12.3.1 AXI4 Write Channels

Frontend Interface (fub_axi_*): - AW channel: `awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser, awvalid, awready` - W channel: `wdata, wstrb, wlast, wuser, wvalid, wready` - B channel: `bid, bresp, buser, bvalid, bready`

Master Interface (m_axi_*): - Same signals as frontend, mirrored direction

12.3.2 Monitor Configuration

Configuration ports are identical to axi4_master_rd_mon: - Basic enables: `cfg_monitor_enable` (master runtime gate: 0 = monitor inert, CAM held clear, never stalls the datapath), `cfg_error_enable`, `cfg_timeout_enable`, `cfg_perf_enable`, `cfg_compl_enable` (completion packets), `cfg_threshold_enable` (threshold-crossed packets), `cfg_debug_enable` (debug/trace cone — the 6th reporter sub-block) - Clear: `cam_clear` (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: `cfg_timeout_cycles` (unified coarse timeout, a MICROSECOND count at full 16-bit width: 1..65535 us per phase, 0 = 16'hFFFF ~ effectively never; drives all three phase counts), `cfg_latency_threshold` - Filtering: 8 mask signals (`cfg_axi_*_mask`: pkt, error, timeout, compl, thresh, perf, addr, debug) plus `cfg_axi_err_select` - Performance window control: `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close` (see Performance Monitoring)

> The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) are fixed inside the wrapper; `cfg_active_trans_threshold` is driven by the `ACTIVE_TRANS_THRESHOLD` parameter (default MAX_TRANSACTIONS/2), not hardwired to 8, and all three are **not** top-level ports on this module.

12.3.3 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Downstream ready to accept packet

Port	Direction	Width	Description
`monbus_packet`	Output	128	`monitor_packet_t` (see format below)
`monbus_timestamp`	Output	64	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_block_ready	Output	1	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)
`i_mon_time`	Input	64	Free-running counter from `monbus_group_core`, sampled at packet emission

12.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions (for clock gating)
`active_transactions`	Output	8	Current number of outstanding transactions
`error_count`	Output	16	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`transaction_count`	Output	32	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when

Port	Direction	Width	Description
			ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
cfg_conflict_error	Output	1	Configuration conflict detected

12.3.5 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
cfg_addr_filter_enable	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
cfg_addr_filter_low	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	ADDR_WIDTH	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

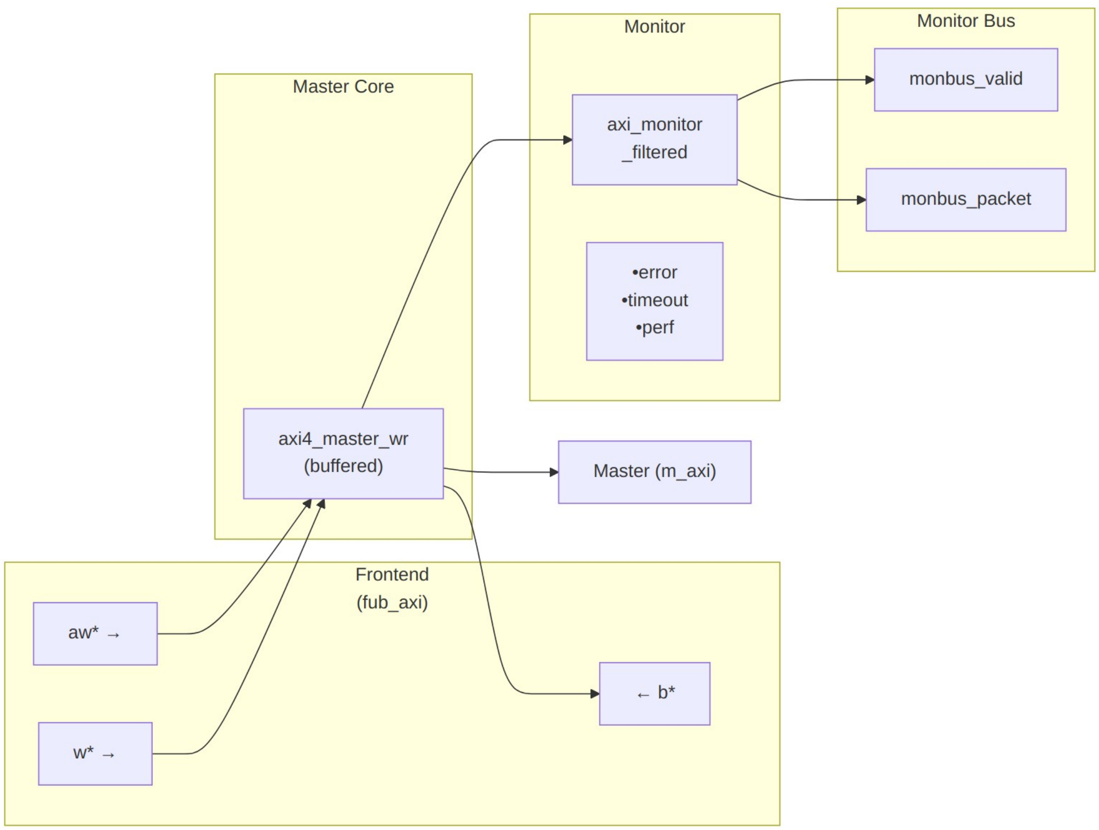
Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
cfg_id_filter_enable	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
cfg_id_match_base	ID_WIDTH	First ID owned

Port	Width	Description
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

12.4 Functional Description



Architecture

The module instantiates two sub-modules: 1. **axi4_master_wr** - Core AXI4 write functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)

12.4.1 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axi_awready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (in-RTL formal property `ap_disabled_never_stalls`).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `monitor_common_pkg`), and `block_ready` re-asserts at occupancy < `MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

12.4.2 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper exposes **`ADDR_RANGE_IS_ERROR`** (per-range) as a parameter, so ranges can be assigned to either flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

12.4.3 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`. The **measurement-window state machine** and its counters are unconditional `always_ff` blocks in `axi_monitor_base` – outside every generate – so they are present and running in EVERY build, including `ENABLE_PERF_LOGIC = 0`. That parameter reaches one consumer: the `g_perf` generate in `axi_monitor_reporter` holding the legacy perf-packet cone and the two 16-bit lifetime counters behind `perf_completed_count` / `perf_error_count`. Setting it to 0 saves that cone and nothing else. `USE_MONITOR = 0` is what ties the perfmon outputs off. The wrapper instantiates plus a bank of W-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

12.4.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

12.4.3.2 Utilization Counters (W Data Channel)

Every cycle inside the window is classified by the W channel's `wvalid` / `wready` into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	wvalid && wready	productive beat transferred
`perf_bp_cycles`	32	wvalid && !wready	back-pressure

Output	Width	Condition	Meaning
			(data offered, sink not ready)
`perf_starv_cycles`	32	!wvalid && wready	starvation (sink ready, no data)
`perf_idle_cycles`	32	!wvalid && !wready	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); the one-count skew is negligible for long windows.

12.4.3.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	W data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << AWSIZE), using the AWSIZE captured at the most recent AW address phase (upper bound: counts full-width beats and does not subtract bytes masked off by `WSTRB` on unaligned or partial beats)
`perf_burst_count`	32	AW address-phase handshakes

The integrator computes average burst length as `perf_beat_count / perf_burst_count`.

12.4.3.4 Performance Monitoring Ports

Port	Direction	Width	Description
`cfg_perf_enable`	Input	1	Enable performance-metric packet generation
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end** event source select

Port	Direction	Width	Description
cfg_start_trigger	Input	1	Pulse: open the measurement window
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod_cycles	Output	32	wvalid && wready cycles
perf_bp_cycles	Output	32	wvalid && !wready cycles (back-pressure)
perf_starv_cycles	Output	32	!wvalid && wready cycles (starvation)
perf_idle_cycles	Output	32	!wvalid && !wready cycles
perf_beat_count	Output	32	W data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AW address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

12.4.4 Monitor Packet Format

The 128-bit `monbus_packet` (paired with the 64-bit `monbus_timestamp` side-band signal) follows the standardized AMBA monitor bus format. Identical format as axi4_master_rd_mon:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, DECERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished)
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
```

```
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits — AXI ID or channel index)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits — full address, latency, etc.)
```

12.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

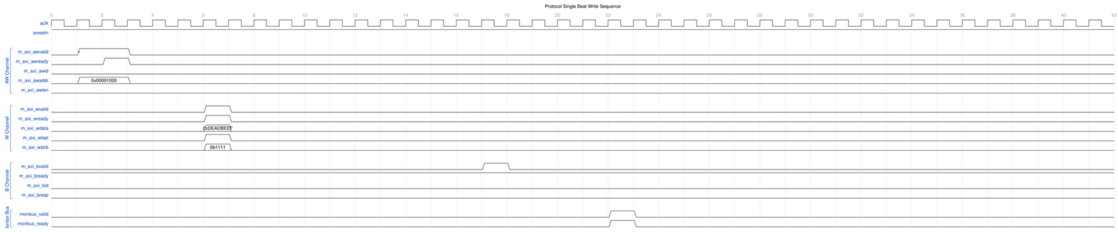
No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

12.6 Waveforms

The following waveforms show AXI4 master write monitor behavior:

12.6.1 Scenario 1: Single-Beat Write Transaction

Complete AXI4 write transaction showing AW, W, and B channels:



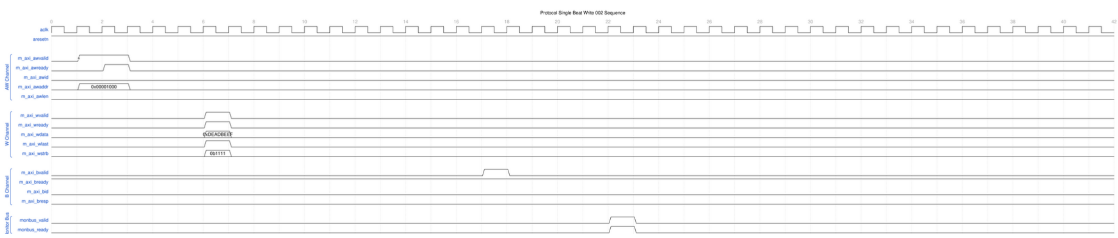
Single Beat Write

WaveJSON: single_beat_write_001.json

Key Observations: - AW channel handshake (AWVALID/AWREADY) - W channel data (WVALID/WREADY/WLAST/WSTRB) - B channel response (BVALID/BREADY/BRESP) - Monitor bus packet generation - Three-channel write protocol coordination

12.6.2 Scenario 2: Alternative Single-Beat Write

Variant single-beat write with different backpressure pattern:



Single Beat Write Alt

WaveJSON: single_beat_write_002_001.json

Key Observations: - Different ready signal timing - Channel interleaving effects - Write strobe (WSTRB) handling - Response latency variation - Monitor packet correlation with transaction ID

12.7 Usage Examples

12.7.1 Configuration Strategies

12.7.1.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch write errors and track completion

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),      // Catch SLVERR/DECERR
.cfg_timeout_enable     (1'b1),      // Detect stuck writes
.cfg_perf_enable        (1'b0),      // Disable (reduces traffic)
```

```
// Filtering - pass error and timeout only
.cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR, TIMEOUT
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask   (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask      (16'hFFFF),  // Drop performance

// Timeouts
.cfg_timeout_cycles     (16'd10),    // 10 microseconds per phase
(full 16-bit range)
.cfg_freq_sel         (4'd0),   // counter_freq_invariant LUT index;
scales the 1 us tick
.cam_clear            (1'b0),   // hold high one cycle while idle to
clear the CAM -- do NOT leave unconnected
.cfg_latency_threshold  (32'd500)
```

12.7.1.2 Strategy 2: Performance Analysis

Goal: Collect write performance metrics

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),      // Still catch errors
.cfg_timeout_enable     (1'b0),      // Disable timeouts
.cfg_perf_enable        (1'b1),      // Enable performance

// Filtering - pass error and performance only
.cfg_axi_pkt_mask       (16'hFFEE),  // Drop all but ERROR, PERF (set
bit = drop)
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_perf_mask      (16'h0000),  // Pass all performance
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_timeout_mask   (16'hFFFF)   // Drop timeouts
```

12.7.1.3 Strategy 3: Debug Mode

Goal: Maximum visibility for write transactions

```
// Enable everything
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),
.cfg_timeout_enable     (1'b1),
.cfg_perf_enable        (1'b1),

// Filtering - pass all packets
.cfg_axi_pkt_mask       (16'h0000),  // Pass all
// All individual masks set to 16'h0000
```

WARNING: Avoid enabling all packet types in high-throughput write scenarios — the monitor bus sustains at most one packet per two cycles and will congest. (Congestion or runtime-disabled classes can drop packets but, since `95c9490a`, can no longer leak table slots or wedge the bus.)

12.7.2 Basic Integration

```
axi4_master_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (1),
    .AGENT_ID           (11),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_master_wr_mon (
    .aclk               (axi_aclk),
    .aresetn            (axi_aresetn),

    // Frontend interface (from write initiator)
    .fub_axi_awid       (write_awid),
    .fub_axi_awaddr     (write_awaddr),
    // ... rest of AW/W/B signals

    // Master interface (to interconnect)
    .m_axi_awid         (m_axi_awid),
    .m_axi_awaddr       (m_axi_awaddr),
    // ... rest of AW/W/B signals

    // Monitor configuration (Strategy 1 - Functional)
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd10),    // 10 microseconds per phase
(full 16-bit range)
    .cfg_latency_threshold  (32'd500),

    .cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR,
TIMEOUT
    .cfg_axi_error_mask     (16'h0000),
    .cfg_axi_timeout_mask   (16'h0000),
    .cfg_axi_compl_mask     (16'hFFFF),
    // ... rest of mask signals
```

```
    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (wr_busy),
    .active_transactions    (wr_active),
    .error_count            (wr_errors),
    .transaction_count      (wr_count),
    .cfg_conflict_error     (cfg_conflict)
);
```

12.8 Design Notes

12.8.1 Write-Specific Monitoring

Write Response Tracking: - Monitors AW channel for write address capture - Tracks W channel beats (WLAST detection) - Correlates B channel responses with AWID - Completes write data on WLAST or the expected beat count – there is NO mismatch event; a short-terminated or over-long burst is not flagged

Common Write Errors Detected: - **SLVERR:** Slave error response (decode failure, access violation) - **DECERR:** Decode error (address not mapped) - **Orphan responses:** a B response (BID) arriving with NO matching tracked command – the detection direction is response-side; an AW that never gets its B surfaces as a TIMEOUT, not an orphan - **Timeout:** Write address or response stuck beyond threshold - **Protocol violations:** response-before-data ordering only (WSTRB never reaches the monitor – no strobe check exists; WLAST is used for completion, not validated)

12.8.2 Performance Considerations

Write Transaction Bandwidth: - AW channel: 1 transaction/cycle (when buffer not full) - W channel: 1 beat/cycle sustained (burst pipelining) - B channel: 1 response/cycle (sparse compared to W beats)

Monitor Packet Budget (Write): - Typical: 2-3 packets per write transaction - Burst writes: More efficient packet/beat ratio - Single writes: Higher packet overhead

12.8.3 Buffer Depth Guidelines

Same as axi4_master_wr: - **SKID_DEPTH_AW:** 2 (default) - sufficient for most systems - **SKID_DEPTH_W:** 4 (default) - accommodates moderate bursts - **SKID_DEPTH_B:** 2 (default) - responses are single-beat

Increase depths for high-latency or high-throughput scenarios.

12.9 Related Modules

12.9.1 Companion Monitors

- **axi4_master_rd_mon** - AXI4 master read with monitoring
- **axi4_slave_rd_mon** - AXI4 slave read with monitoring
- **axi4_slave_wr_mon** - AXI4 slave write with monitoring

12.9.2 Base Modules

- **axi4_master_wr** - Functional AXI4 master write (without monitoring)
- **axi_monitor_filtered** - Monitoring engine with filtering (monitor/)

12.9.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **axi_monitor_base** - Core monitoring logic (monitor/)
- **axi_monitor_trans_mgr** - Transaction tracking (monitor/)

12.10 Testing

`val/amba/test_axi4_master_wr_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_wr_mon.py -v
```

12.11 References

12.11.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: monitor_package_spec.md

12.11.2 Source Code

- RTL: `rtl/amba/axi4/axi4_master_wr_mon.sv`
- Tests: `val/amba/test_axi4_master_wr_mon.py`

- Framework: `bin/TBClasses/components/axi4/`

12.11.3 Documentation

- Configuration Guide: AXI Monitor Base
- Architecture: rtl-amba Overview
- AXI4 Index: README.md

Last Updated: 2026-07-18

12.12 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

13 AXI4 Master Write Monitor (Clock-Gated)

Module: `axi4_master_wr_mon_cg.sv` **Base Module:** axi4_master_wr_mon **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

13.1 Overview

This is the **clock-gated variant** of axi4_master_wr_mon — the same monitored master write, with activity-based clock gating wrapped around it.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the **Clock-Gated Variants Guide**.

What the wrapper buys you:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable:** Idle threshold, gating domains, enable/disable
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate at runtime

13.2 Parameters

MOST axi4_master_wr_mon parameters pass through. As of 2026-09-01 only `ACTIVE_TRANS_THRESHOLD` is NOT forwarded, and that one is harmless: its inner default is `MAX_TRANSACTIONS/2`, computed from the `MAX_TRANSACTIONS` this wrapper DOES forward.

An earlier version of this page listed nine more as unforwarded, which was accurate when written. `ACLK_MHZ`, `CFI_MIN_FREQ_MHZ`, `CFI_MAX_FREQ_MHZ`, `USE_WDATA_ORDER_Q`, `NUM_BANKS` and `ADDR_RANGE_IS_ERROR` were threaded through every `_cg` wrapper on 2026-09-01, and `ID_FILTER_ENABLE` / `ID_MATCH_BASE` / `ID_MATCH_COUNT` the day before. Until then a clock-gated build could not state its clock frequency, so the 1 us timer tick was pinned to the 100 MHz default and every microsecond-denominated timeout was miscalibrated on any other clock – silently.

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`USE_MONITOR`	1	Synthesis-time monitor enable (forwarded to inner monitor).
`N_ADDR_RANGES`	0	Number of address-range comparators (forwarded to base module).
`ADDR_FILTER_ENABLE`	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves `cfg_addr_filter_enable` low filters nothing and looks broken.
`ID_FILTER_ENABLE`	0	Synthesises the ID report filter (see `cfg_id_*` for the runtime override).
`ID_MATCH_BASE`	0	First ID this instance owns.
`ID_MATCH_COUNT`	0	How many IDs; `0` means ALL, so a zeroed register block does not silently filter

Parameter	Default	Description
		everything away.
ADD_PIPELINE_STAGE	0	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
ENABLE_COMPL_LOGIC	1'b1	Synthesise the completion-packet cone. 0 removes the logic entirely.
ENABLE_DEBUG_LOGIC	1'b0	Synthesise the debug-packet cone. 0 removes the logic entirely.
ENABLE_ERROR_LOGIC	1'b1	Synthesise the error detection cone. 0 removes the logic entirely.
ENABLE_FILTERING	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.
ENABLE_PERF_LOGIC	1'b1	Synthesise the reporter's performance cone (`g_perf`). Does NOT gate the perfmon window state machine or its counters.
ENABLE_THRESHOLD_LOGIC	1'b1	Synthesise the threshold-packet cone. 0 removes the logic entirely.
ENABLE_TIMEOUT_LOGIC	1'b1	Synthesise the timeout detection cone. 0 removes the logic entirely.
SKID_DEPTH_AW	2	Skid-buffer depth on the AW channel. Legal range 2..8 inclusive; odd depths are legal.

Parameter	Default	Description
SKID_DEPTH_B	2	Skid-buffer depth on the B channel. Legal range 2..8 inclusive; odd depths are legal.
SKID_DEPTH_W	4	Skid-buffer depth on the W channel. Legal range 2..8 inclusive; odd depths are legal.
AGENT_ID	16'h000B	Agent identifier emitted in the `agent_id` field of every monitor packet. Pairs with `UNIT_ID` to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID	8'h01	Unit identifier emitted in the `unit_id` field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

Gating is controlled by RUNTIME inputs `cfg_cg_enable` / `cfg_cg_idle_count` with status outputs `cg_gating` / `cg_idle`; ONE `amba_clock_gate_ctrl` gates the entire inner module (no per-domain gates). (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

Base-module ports are forwarded EXCEPT `debug_block_ready`, which this wrapper ties off (use the base module for the backpressure tap). `cam_clear` is forwarded (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - and the full performance-monitoring interface (see Performance Monitoring below). The six `ENABLE_*_LOGIC` synthesis-cone parameters and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables are also passed straight through.

13.2.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

Neither filter un-filters entries already admitted to the transaction table, which is what makes changing them at runtime safe.

13.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

13.3 Ports

Port	Dir	Width	Description
aclk	In	1	
aresetn	In	1	
cam_clear	In	1	sync clear of the monitor trans CAM
fub_axi_awid	In	[IW-1:0]	
fub_axi_awaddr	In	[AW-1:0]	
fub_axi_awlen	In	[7:0]	
fub_axi_awsize	In	[2:0]	
fub_axi_awburst	In	[1:0]	
fub_axi_awlock	In	1	
fub_axi_awcache	In	[3:0]	
fub_axi_awprot	In	[2:0]	
fub_axi_awqos	In	[3:0]	
fub_axi_awregion	In	[3:0]	
fub_axi_awuser	In	[UW-1:0]	
fub_axi_awvalid	In	1	
fub_axi_awready	Out	1	
fub_axi_wdata	In	[DW-1:0]	
fub_axi_wstrb	In	[SW-1:0]	
fub_axi_wlast	In	1	
fub_axi_wuser	In	[UW-1:0]	
fub_axi_wvalid	In	1	
fub_axi_wready	Out	1	
fub_axi_bid	Out	[IW-1:0]	
fub_axi_bresp	Out	[1:0]	
fub_axi_buser	Out	[UW-1:0]	
fub_axi_bvalid	Out	1	
fub_axi_bready	In	1	

m_axi_awid	Out	[IW-1:0]	
m_axi_awaddr	Out	[AW-1:0]	
m_axi_awlen	Out	[7:0]	
m_axi_awsize	Out	[2:0]	
m_axi_awburst	Out	[1:0]	
m_axi_awlock	Out	1	
m_axi_awcache	Out	[3:0]	
m_axi_awprot	Out	[2:0]	
m_axi_awqos	Out	[3:0]	
m_axi_awregion	Out	[3:0]	
m_axi_awuser	Out	[UW-1:0]	
m_axi_awvalid	Out	1	
m_axi_awready	In	1	
m_axi_wdata	Out	[DW-1:0]	
m_axi_wstrb	Out	[SW-1:0]	
m_axi_wlast	Out	1	
m_axi_wuser	Out	[UW-1:0]	
m_axi_wvalid	Out	1	
m_axi_wready	In	1	
m_axi_bid	In	[IW-1:0]	
m_axi_bresp	In	[1:0]	
m_axi_buser	In	[UW-1:0]	
m_axi_bvalid	In	1	
m_axi_bready	Out	1	
cfg_monitor_enable	In	1	Enable monitoring
cfg_error_enable	In	1	Enable error detection
cfg_timeout_enable	In	1	Enable timeout detection
cfg_perf_enable	In	1	Enable performance

			monitoring
cfg_compl_enable	In	1	Enable completion packets
cfg_threshold_enable	In	1	Enable threshold packets
cfg_debug_enable	In	1	Enable debug packets
cfg_timeout_cycles	In	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	In	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	In	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	In	[15:0]	Drop mask for packet types
cfg_axi_err_select	In	[15:0]	Error select for packet types (for future routing)
cfg_axi_error_mask	In	[15:0]	Individual error event mask
cfg_axi_timeout_mask	In	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	In	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	In	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	In	[15:0]	Individual performance

			event mask
cfg_axi_addr_mask	In	[15:0]	Individual address match event mask
cfg_axi_debug_mask	In	[15:0]	Individual debug event mask
cfg_cg_enable	In	1	Enable clock gating
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	In	1	
cfg_addr_range_enable	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_low	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_high	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	In	1	
cfg_addr_filter_low	In	[AW-1:0]	
cfg_addr_filter_high	In	[AW-1:0]	
cfg_id_filter_enable	In	1	
cfg_id_match_base	In	[IW-1:0]	
cfg_id_match_count	In	[IW:0]	
i_mon_time	In	1	
monbus_valid	Out	1	Monitor bus valid
monbus_ready	In	1	Monitor bus ready

monbus_packet	Out	1	Monitor packet (128-bit)
monbus_timestamp	Out	1	Side-band sampled time
busy	Out	1	
active_transactions	Out	[7:0]	Number of active transactions
error_count	Out	[15:0]	Total error count
transaction_count	Out	[31:0]	Total transaction count
cg_gating	Out	1	Gated clock is stopped
cg_idle	Out	1	No activity observed
cfg_conflict_error	Out	1	Configuration conflict detected
cfg_start_event_sel	In	[2:0]	
cfg_end_event_sel	In	[2:0]	
cfg_start_trigger	In	1	
cfg_end_trigger	In	1	
cfg_window_force_close	In	1	
window_active	Out	1	
window_cycles	Out	[31:0]	
perf_prod_cycles	Out	[31:0]	
perf_bp_cycles	Out	[31:0]	
perf_starv_cycles	Out	[31:0]	
perf_idle_cycles	Out	[31:0]	
perf_beat_count	Out	[31:0]	
perf_byte_count	Out	[63:0]	
perf_burst_count	Out	[31:0]	

13.4 Functional Description

13.4.1 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_master_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi4_master_wr_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

WARNING – gating vs window accounting: an open measurement window is NOT a wake term, and the entire inner monitor (window state machine and counters included) runs on the gated clock. If the bus idles past `cfg_cg_idle_count` with a window open, the counters FREEZE while wall-clock time passes, and trigger pulses (`cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`, `cam_clear`) arriving while gated are DROPPED. For exact wall-clock windows or idle-bus triggering, hold `cfg_cg_enable` low around the measurement, or use the base module.

13.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

13.6 Usage Examples

```
axi4_master_wr_mon_cg #(
    // Base module parameters (see axi4_master_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_master_wr_mon (except
debug_block_ready)
);
```

13.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

13.8 Related Modules

- **Base Module Functionality:** axi4_master_wr_mon.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

13.9 Testing

`val/amba/test_axi4_master_wr_mon_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_master_wr_mon_cg.py -v
```

13.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

14 AXI4 Master Write Stub

Module: `axi4_master_wr_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

14.1 Overview

The write-side counterpart to the read stub. The AXI4 Master Write Stub packs the AW (write address), W (write data), and B (write response) channels into simple packet interfaces through skid buffers, so your testbench drives one wide payload per channel instead of handshaking a dozen AXI signals. Ideal for testbenches and integration scenarios where a simplified interface is the whole point.

14.1.1 Key Features

- Packed packet interface for AW, W, and B channels
- Configurable skid buffer depth on each channel
- Full AXI4 write transaction support
- Burst, ID, strobe, and user signal support
- Parameterized data widths

14.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_W	int	4	W channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_B	int	2	B channel skid buffer depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width

Parameter	Type	Default	Description
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)

14.3 Ports

14.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	AXI reset (active low)

14.3.2 AXI4 Write Address Channel (AW)

Port	Direction	Width	Description
m_axi_awid	Output	IW	Write address ID
m_axi_awaddr	Output	AW	Write address
m_axi_awlen	Output	8	Burst length
m_axi_awsize	Output	3	Burst size
m_axi_awburst	Output	2	Burst type
m_axi_awlock	Output	1	Lock type
m_axi_awcache	Output	4	Cache type
m_axi_awprot	Output	3	Protection type
m_axi_awqos	Output	4	Quality of service
m_axi_awregion	Output	4	Region identifier
m_axi_awuser	Output	UW	User signal
m_axi_awvalid	Output	1	Write address

Port	Direction	Width	Description
			valid
m_axi_awready	Input	1	Write address ready

14.3.3 AXI4 Write Data Channel (W)

Port	Direction	Width	Description
m_axi_wdata	Output	DW	Write data
m_axi_wstrb	Output	SW	Write strobes
m_axi_wlast	Output	1	Write last
m_axi_wuser	Output	UW	User signal
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready

14.3.4 AXI4 Write Response Channel (B)

Port	Direction	Width	Description
m_axi_bid	Input	IW	Response ID
m_axi_bresp	Input	2	Write response
m_axi_buser	Input	UW	User signal
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready

14.3.5 AW Packet Interface

Port	Direction	Width	Description
fub_axi_awvalid	Input	1	AW packet valid
fub_axi_awready	Output	1	Ready to accept AW packet
fub_axi_aw_co	Output	4	AW buffer

Port	Direction	Width	Description
unt			occupancy
fub_axi_aw_pkt	Input	AWSize	Packed AW packet data

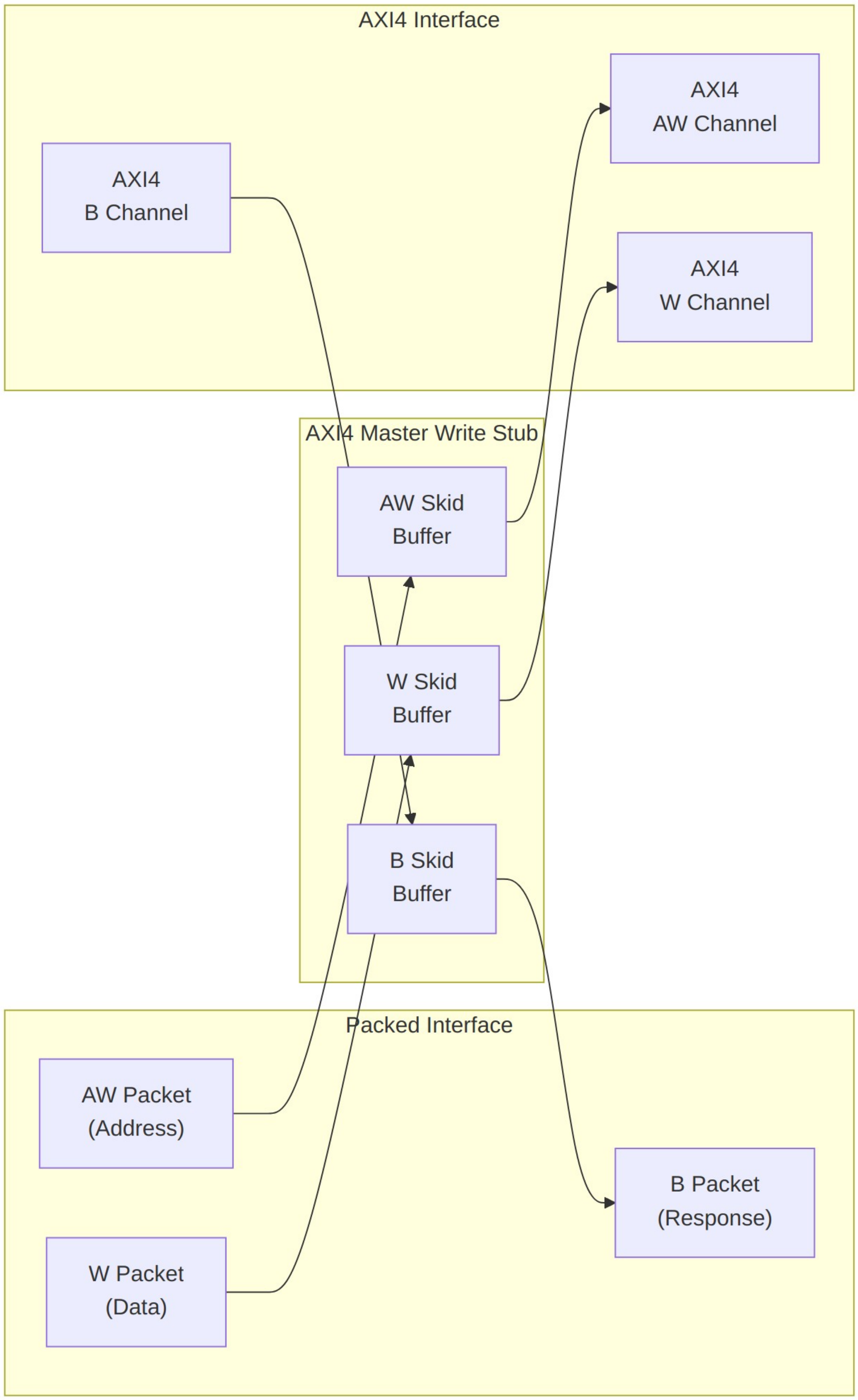
14.3.6 W Packet Interface

Port	Direction	Width	Description
fub_axi_wvalid	Input	1	W packet valid
fub_axi_wready	Output	1	Ready to accept W packet
fub_axi_w_pkt	Input	WSize	Packed W packet data

14.3.7 B Packet Interface

Port	Direction	Width	Description
fub_axi_bvalid	Output	1	B packet valid
fub_axi_bready	Input	1	Ready to accept B packet
fub_axi_b_pkt	Output	BSize	Packed B packet data

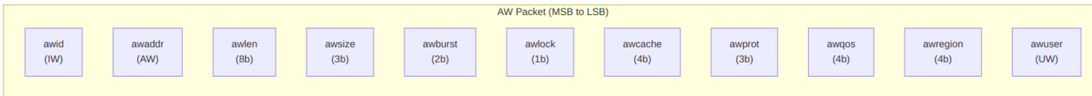
14.4 Functional Description



Architecture

14.4.1 Packet Formats

Packets are packed MSB-to-LSB in AXI signal order — one concatenation per channel, no bit shuffling required on the testbench side.

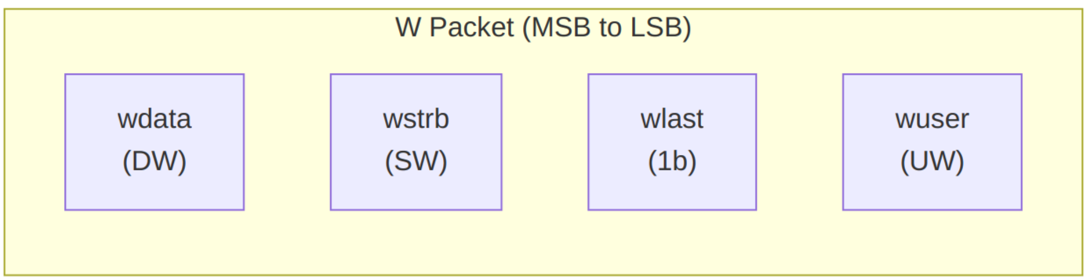


AW Packet Structure (Write Address)

Bit Positions:

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

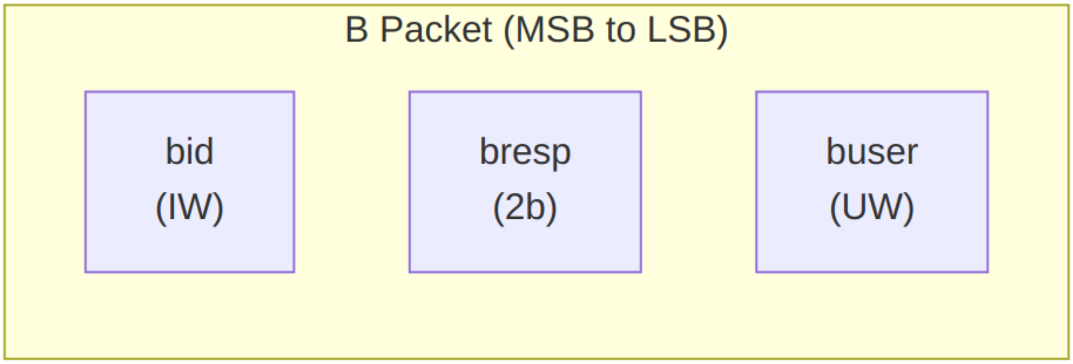


W Packet Structure (Write Data)

Bit Positions:

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}

Width = DW + SW + 1 + UW
```

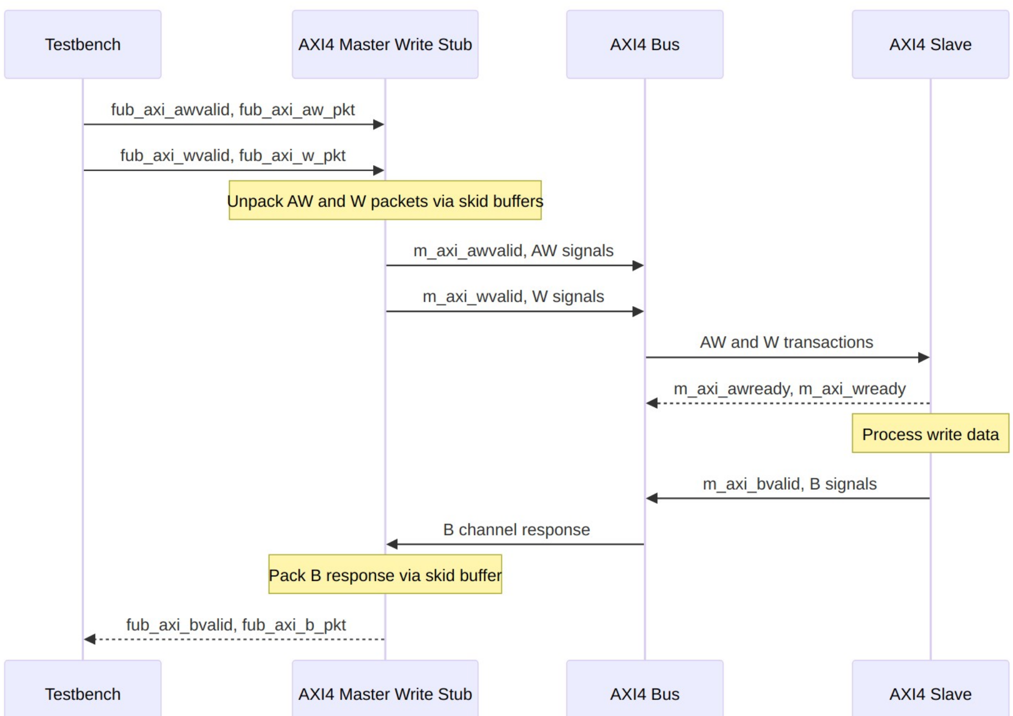


B Packet Structure (Write Response)

Bit Positions:

```
fub_axi_b_pkt = {bid, bresp, buser}

Width = IW + 2 + UW
```



Transaction Flow

14.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axi_awvalid, fub_axi_awready, fub_axi_aw_pkt
- fub_axi_wvalid, fub_axi_wready, fub_axi_w_pkt
- AXI AW signals (m_axi_awvalid, m_axi_awaddr, m_axi_awlen, etc.)
- AXI W signals (m_axi_wvalid, m_axi_wdata, m_axi_wstrb, m_axi_wlast, etc.)
- AXI B signals (m_axi_bvalid, m_axi_bid, m_axi_bresp, etc.)
- fub_axi_bvalid, fub_axi_bready, fub_axi_b_pkt
- Packet-to-AXI timing relationship with skid buffer operation

---

14.6 Usage Examples

```
axi4_master_wr_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_master_wr_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 master write interface
    .m_axi_awid      (m_axi_awid),
    .m_axi_awaddr    (m_axi_awaddr),
    .m_axi_awlen     (m_axi_awlen),
    .m_axi_awsize    (m_axi_awsize),
    .m_axi_awburst   (m_axi_awburst),
    .m_axi_awlock    (m_axi_awlock),
    .m_axi_awcache   (m_axi_awcache),
    .m_axi_awprot    (m_axi_awprot),
    .m_axi_awqos     (m_axi_awqos),
    .m_axi_awregion  (m_axi_awregion),
    .m_axi_awuser    (m_axi_awuser),
    .m_axi_awvalid   (m_axi_awvalid),
    .m_axi_awready   (m_axi_awready),
```

```
    .m_axi_wdata     (m_axi_wdata),
    .m_axi_wstrb     (m_axi_wstrb),
    .m_axi_wlast     (m_axi_wlast),
    .m_axi_wuser     (m_axi_wuser),
    .m_axi_wvalid    (m_axi_wvalid),
    .m_axi_wready    (m_axi_wready),

    .m_axi_bid       (m_axi_bid),
    .m_axi_bresp     (m_axi_bresp),
    .m_axi_buser     (m_axi_buser),
    .m_axi_bvalid    (m_axi_bvalid),
    .m_axi_bready    (m_axi_bready),

    // Packed AW interface
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    // Packed W interface
    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    // Packed B interface
    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt)
);

// Build AW packet (single beat write at address 0x1000)
localparam AWSize = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + 4;  //
Calculate size
assign tb_aw_pkt = {
    8'd0,            // awid
    32'h0000_1000,   // awaddr
    8'd0,            // awlen (1 beat)
    3'b011,          // awsize (8 bytes)
    2'b01,           // awburst (INCR)
    1'b0,            // awlock
    4'b0011,         // awcache
    3'b000,          // awprot
    4'b0000,         // awqos
    4'b0000,         // awregion
    4'h0             // awuser
};
```

```
// Build W packet
localparam WSize = 64 + 8 + 1 + 4;  // Calculate size
assign tb_w_pkt = {
    64'hDEAD_BEEF_CAFE_BABE,  // wdata
    8'hFF,                     // wstrb (all bytes)
    1'b1,                      // wlast
    4'h0                       // wuser
};

// Parse B packet
localparam int BSize = 8 + 2 + 4;   // IW + resp + UW
wire [7:0] b_id   = tb_b_pkt[BSize-1:BSize-8];
wire [1:0] b_resp = tb_b_pkt[5:4];
wire [3:0] b_user = tb_b_pkt[3:0];
```

14.7 Design Notes

14.7.1 Skid Buffer Operation

The stub is three `gaxi_skid_buffer` instances and nothing fancier. They:

- Decouple timing between testbench and AXI bus
- Provide configurable buffering depth per channel
- Handle backpressure gracefully
- Support burst transactions without stalling

Recommended Depths: - **AW Channel:** 2-4 (address transactions) - **W Channel:** 4-8 (data beats for bursts) - **B Channel:** 2-4 (responses)

14.7.2 Channel Independence

The AW, W, and B channels are independent: - AW and W can be driven in any order - Stub handles proper AXI timing - B responses arrive asynchronously

14.7.3 Packet Packing Order

AW, W, and B packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet creation - Matches common concatenation order - Efficient for burst transaction handling

14.7.4 Internal Architecture

The stub instantiates three `gaxi_skid_buffer` modules: - **AW Skid Buffer:** Unpacks AW packets to AXI AW channel - **W Skid Buffer:** Unpacks W packets to AXI W channel - **B Skid Buffer:** Packs AXI B channel to B packets

All AXI protocol handling is done by the skid buffers and downstream modules.

14.8 Related Modules

- **AXI4 Master Write** - Full AXI4 master write module (if wrapping one)
- **AXI4 Master Read Stub** - Corresponding read stub
- **AXI4 Master Stub** - Combined read/write stub
- **AXI4 Slave Write Stub** - Slave-side write stub

14.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

14.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

15 AXI4 Slave Read

Module: `axi4_slave_rd.sv` **Location:** `rtl/amba/axi4/` **Status:** Production Ready

15.1 Overview

The mirror image of the master-side buffers. The AXI4 Slave Read sits between an AXI4 interconnect and your memory or backend processing element, with configurable-depth skid buffers on the AR and R channels. The interconnect can present addresses whenever it likes, the backend can return data whenever it's ready, and neither one stalls the other — that's the entire job, and it's worth doing well.

15.1.1 Key Features

- **Full AXI4 Read Support:** Complete AR and R channel implementation
- **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- **Elastic Buffering:** Decouples interconnect and backend timing domains
- **Burst Support:** Full burst transaction handling with RLAST tracking
- **User Signal Support:** Carries ARUSER and RUSER signals
- **Clock Gating Support:** Busy signal for dynamic power management

15.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	2	Depth of read address (AR) channel skid buffer
`SKID_DEPTH_R`	int	4	Depth of read data (R) channel skid buffer
`AXI_ID_WIDTH`	int	8	Width of transaction ID signals (ARID, RID)
`AXI_ADDR_WIDTH`	int	32	Width of address bus (ARADDR)
`AXI_DATA_WIDTH`	int	32	Width of data bus (RDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024
`AXI_USER_WIDTH`	int	1	Width of user-defined signals (ARUSER, RUSER)

15.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`

Derived parameter	Default expression
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
ARSize	IW+AW+8+3+2+1+4+3+4+4+UW
RSize	IW+DW+2+1+UW

15.3 Ports

The full port list, straight from the RTL:

```
module axi4_slave_rd #(
    parameter int SKID_DEPTH_AR      = 2,
    parameter int SKID_DEPTH_R       = 4,
    parameter int AXI_ID_WIDTH       = 8,
    parameter int AXI_ADDR_WIDTH     = 32,
    parameter int AXI_DATA_WIDTH     = 32,
    parameter int AXI_USER_WIDTH     = 1
) (
    // Clock and Reset
    input  logic                       aclk,
    input  logic                       aresetn,

    // Slave AXI Interface (s_axi_*)
    // Read address channel (AR)
    input  logic [AXI_ID_WIDTH-1:0]   s_axi_arid,
    input  logic [AXI_ADDR_WIDTH-1:0] s_axi_araddr,
    input  logic [7:0]                 s_axi_arlen,
    input  logic [2:0]                 s_axi_arsize,
    input  logic [1:0]                 s_axi_arburst,
    input  logic                       s_axi_arlock,
    input  logic [3:0]                 s_axi_arcache,
    input  logic [2:0]                 s_axi_arprot,
    input  logic [3:0]                 s_axi_arqos,
    input  logic [3:0]                 s_axi_arregion,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_aruser,
    input  logic                       s_axi_arvalid,
    output logic                       s_axi_arready,

    // Read data channel (R)
    output logic [AXI_ID_WIDTH-1:0]   s_axi_rid,
    output logic [AXI_DATA_WIDTH-1:0] s_axi_rdata,
    output logic [1:0]                 s_axi_rresp,
```

```
    output logic                        s_axi_rlast,
    output logic [AXI_USER_WIDTH-1:0] s_axi_ruser,
    output logic                        s_axi_rvalid,
    input  logic                        s_axi_rready,

    // Backend Interface (fub_axi_* - to memory/backend)
    // Read address channel (AR)
    output logic [AXI_ID_WIDTH-1:0]   fub_axi_arid,
    output logic [AXI_ADDR_WIDTH-1:0] fub_axi_araddr,
    output logic [7:0]                  fub_axi_arlen,
    output logic [2:0]                  fub_axi_arsize,
    output logic [1:0]                  fub_axi_arburst,
    output logic                        fub_axi_arlock,
    output logic [3:0]                  fub_axi_arcache,
    output logic [2:0]                  fub_axi_arprot,
    output logic [3:0]                  fub_axi_arqos,
    output logic [3:0]                  fub_axi_arregion,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_aruser,
    output logic                        fub_axi_arvalid,
    input  logic                        fub_axi_arready,

    // Read data channel (R)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_rid,
    input  logic [AXI_DATA_WIDTH-1:0] fub_axi_rdata,
    input  logic [1:0]                  fub_axi_rresp,
    input  logic                        fub_axi_rlast,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_ruser,
    input  logic                        fub_axi_rvalid,
    output logic                        fub_axi_rready,

    // Status
    output logic                        busy
);
```

15.3.1 Clock and Reset

Port	Direction	Width	Description
`aclk`	Input	1	AXI clock - all signals sampled on rising edge
`aresetn`	Input	1	Active-low asynchronous reset

15.3.2 Slave AXI Interface (s_axi_*)

Read Address Channel (AR)

Port	Direction	Width	Description
`s_axi_arid`	Input	`AXI_ID_WIDTH`	Read transaction ID
`s_axi_araddr`	Input	`AXI_ADDR_WIDTH`	Read address
`s_axi_arlen`	Input	8	Burst length, AXI-encoded: beats - 1 (`0x00` = 1 beat, `0xFF` = 256)
`s_axi_arsize`	Input	3	Burst size (bytes per beat)
`s_axi_arburst`	Input	2	Burst type (FIXED, INCR, WRAP)
`s_axi_arlock`	Input	1	Lock type (atomic access support)
`s_axi_arcache`	Input	4	Cache attributes
`s_axi_arprot`	Input	3	Protection attributes
`s_axi_arqos`	Input	4	Quality of Service identifier
`s_axi_arregion`	Input	4	Region identifier
`s_axi_aruser`	Input	`AXI_USER_WIDTH`	User-defined signal
`s_axi_arvalid`	Input	1	Read address valid
`s_axi_arready`	Output	1	Read address ready

Read Data Channel (R)

Port	Direction	Width	Description
`s_axi_rid`	Output	`AXI_ID_WIDTH`	Read transaction ID
`s_axi_rdata`	Output	`AXI_DATA_WIDTH`	Read data
`s_axi_rresp`	Output	2	Read response (OKAY,

Port	Direction	Width	Description
			EXOKAY, SLVERR, DECERR)
`s_axi_rlast`	Output	1	Last beat of burst indicator
`s_axi_ruser`	Output	`AXI_USER_WIDTH`	User-defined signal
`s_axi_rvalid`	Output	1	Read data valid
`s_axi_rready`	Input	1	Read data ready

15.3.3 Backend Interface (fub_axi_*)

Mirrors the slave interface but in the opposite direction (output on AR, input on R).

15.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions in buffers (for clock gating)

15.4 Functional Description

15.4.1 Architecture

Two independent `gaxi_skid_buffer` instances provide the elastic buffering, one per read channel:

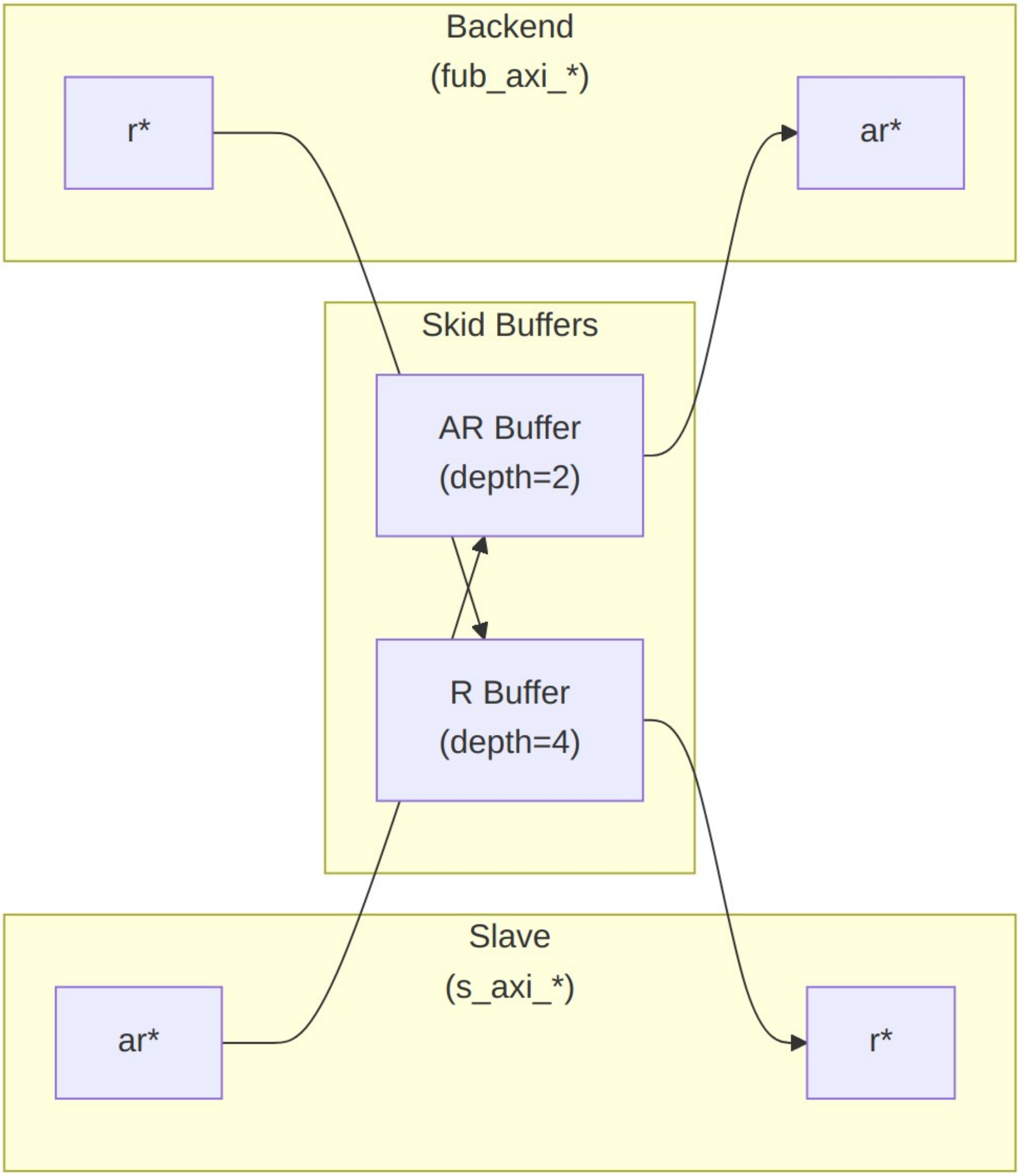


Diagram 19

15.4.2 Channel Operations

15.4.2.1 Read Address (AR) Channel

1. Master presents read address and attributes via interconnect
2. AR skid buffer accepts when space available (`s_axi_arready` high)
3. Buffered address presented to backend when ready

4. Configurable depth (`SKID_DEPTH_AR`) smooths timing variations

15.4.2.2 Read Data (R) Channel

1. Backend returns read data with transaction ID
2. R skid buffer provides deeper buffering (default depth=4)
3. RLAST signal preserved to indicate burst boundaries
4. Data forwarded to master when ready to accept

15.4.3 Busy Signal

The `busy` output indicates active transactions:

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
                s_axi_arvalid || fub_axi_rvalid;
```

Use cases: - **Clock gating:** Disable clock when `busy` is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

15.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

15.6 Usage Examples

15.6.1 Basic Integration with Memory

```
// Instantiate AXI4 slave read buffer
axi4_slave_rd #(
    .SKID_DEPTH_AR(2),
```

```
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_slave_rd (
    .aclk                   (axi_aclk),
    .aresetn                (axi_aresetn),

    // Slave interface (from AXI interconnect)
    .s_axi_arid             (s_axi_arid),
    .s_axi_araddr           (s_axi_araddr),
    .s_axi_arlen            (s_axi_arlen),
    .s_axi_arsize           (s_axi_arsize),
    .s_axi_arburst          (s_axi_arburst),
    .s_axi_arlock           (s_axi_arlock),
    .s_axi_arcache          (s_axi_arcache),
    .s_axi_arprot           (s_axi_arprot),
    .s_axi_arqos            (s_axi_arqos),
    .s_axi_arregion         (s_axi_arregion),
    .s_axi_aruser           (s_axi_aruser),
    .s_axi_arvalid          (s_axi_arvalid),
    .s_axi_arready          (s_axi_arready),

    .s_axi_rid              (s_axi_rid),
    .s_axi_rdata            (s_axi_rdata),
    .s_axi_rresp            (s_axi_rresp),
    .s_axi_rlast            (s_axi_rlast),
    .s_axi_ruser            (s_axi_ruser),
    .s_axi_rvalid           (s_axi_rvalid),
    .s_axi_rready           (s_axi_rready),

    // Backend interface (to memory controller)
    .fub_axi_arid           (mem_arid),
    .fub_axi_araddr         (mem_araddr),
    .fub_axi_arlen          (mem_arlen),
    .fub_axi_arsize         (mem_arsize),
    .fub_axi_arburst        (mem_arburst),
    .fub_axi_arlock         (mem_arlock),
    .fub_axi_arcache        (mem_arcache),
    .fub_axi_arprot         (mem_arprot),
    .fub_axi_arqos          (mem_arqos),
    .fub_axi_arregion       (mem_arregion),
    .fub_axi_aruser         (mem_aruser),
    .fub_axi_arvalid        (mem_arvalid),
    .fub_axi_arready        (mem_arready),
```

```
    .fub_axi_rid        (mem_rid),
    .fub_axi_rdata      (mem_rdata),
    .fub_axi_rresp      (mem_rresp),
    .fub_axi_rlast      (mem_rlast),
    .fub_axi_ruser      (mem_ruser),
    .fub_axi_rvalid     (mem_rvalid),
    .fub_axi_rready     (mem_rready),

    // Status
    .busy               (rd_slave_busy)
);

// Memory controller backend
memory_controller u_mem_ctrl (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_arid       (mem_arid),
    .axi_araddr     (mem_araddr),
    // ... rest of signals
);
```

15.6.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .clk_in            (axi_clk),
    .aresetn           (axi_resetn),
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),
    .wakeup            (rd_slave_busy),
    .clk_out           (axi_clk_gated),
    .gating            ()
);

// Connect module to gated clock
axi4_slave_rd #( ... ) u_slave_rd (
    .aclk (axi_clk_gated),
    ...
);
```

15.7 Design Notes

15.7.1 Buffer Depth Selection

`SKID_DEPTH_*` is an entry count, not a log2 exponent. The underlying `gaxi_skid_buffer` allocates one register slot per entry and tracks occupancy with a 4-bit counter, so legal values are 2..8 inclusive (any integer). Values greater than 8 overflow the occupancy counter and are not supported.

Read Address (SKID_DEPTH_AR): - Default: 2 (sufficient for most cases) - Increase if: - High-latency backend address processing - Frequent address channel backpressure - Multiple outstanding bursts needed

Read Data (SKID_DEPTH_R): - Default: 4 (deeper than AR for burst data) - Increase if: - Large burst sizes (ARLEN > 4) - High-bandwidth streaming reads - Variable backend read latency

15.7.2 Recommended Configurations

Low-Latency Memory (single outstanding read):

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(2)
) u_slave_rd ( ... );
```

High-Throughput Streaming (burst reads):

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(4),
    .SKID_DEPTH_R(8)      // Deep for burst data
) u_slave_rd ( ... );
```

Variable Latency Backend:

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(8),
    .SKID_DEPTH_R(8)
) u_slave_rd ( ... );
```

15.7.3 Buffer Independence

The two skid buffers operate independently: - AR channel can accept new addresses while R channel returns data - Burst reads can pipeline - next burst starts before previous completes - Backend can have variable read latency without stalling interconnect

15.7.4 Packet Preservation

All signal groups packed and unpacked atomically: - AR channel: `{arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser}` - R channel: `{rid, rdata, rresp, rlast, ruser}`

15.7.5 Backpressure Handling

Ready signals propagate backpressure: - Interconnect sees `arready` low when AR buffer full - Backend sees `rready` low when R buffer full - Prevents data loss while decoupling timing

15.7.6 ID and Burst Handling

This module is a pass-through buffer: - Does NOT track transaction IDs - Does NOT enforce burst ordering - Relies on backend to: - Match RID to ARID - Return correct number of beats (ARLEN+1) - Assert RLAST on final beat

15.7.7 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

15.8 Related Modules

15.8.1 Companion Modules

- **axi4_slave_wr** - AXI4 slave write with buffering
- **axi4_slave_rd_cg** - Clock-gated variant with additional CG logic
- **axi4_slave_rd_mon** - Read transaction monitor for verification

15.8.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

15.8.3 Related Infrastructure

- **axi4_master_rd** - Corresponding AXI4 read master module
- **axi4_interconnect** - Multi-master/multi-slave crossbar

15.9 Testing

`val/amba/test_axi4_slave_rd.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_slave_rd.py -v
```

15.10 References

15.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

15.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_rd.sv`
- Tests: `val/amba/test_axi4_slave_rd.py`
- Framework: `bin/TBClasses/components/axi4/`

15.10.3 Documentation

- Architecture: rtl-amba Overview
- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

15.11 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

16 AXI4 Slave Read Interface (Clock-Gated)

Module: `axi4_slave_rd_cg.sv` **Base Module:** axi4_slave_rd **Location:** `rtl/amba/axi4/` **Status:** Production Ready

---

16.1 Overview

This is the **clock-gated variant** of axi4_slave_rd — same elastic buffer, with activity-based clock gating wrapped around it for power.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the **Clock-Gated Variants Guide**.

What the wrapper buys you:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable at runtime:** `cfg_cg_enable` / `cfg_cg_idle_count` inputs
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate

16.2 Parameters

In addition to all axi4_slave_rd parameters:

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`SKID_DEPTH_AR`	2	Skid-buffer depth on the AR channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_R`	4	Skid-buffer depth on the R channel. Legal range 2..8 inclusive; odd depths are legal.

The gating controls are RUNTIME INPUTS, not parameters: `cfg_cg_enable` and `cfg_cg_idle_count`; status outputs `cg_gating` / `cg_idle`. One `amba_clock_gate_ctrl` gates the whole module – no per-domain gates. The base module's `busy` output is NOT re-exported (consumed internally as a wake term). (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

16.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`

Derived parameter	Default expression
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
ARSize	IW+AW+8+3+2+1+4+3+4+4+UW
RSize	IW+DW+2+1+UW

16.3 Ports

Port	Dir	Width	Description
aclk	In	1	
aresetn	In	1	
cfg_cg_enable	In	1	
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axi_arid	In	[IW-1:0]	
s_axi_araddr	In	[AW-1:0]	
s_axi_arlen	In	[7:0]	
s_axi_arsize	In	[2:0]	
s_axi_arburst	In	[1:0]	
s_axi_arlock	In	1	
s_axi_arcache	In	[3:0]	
s_axi_arprot	In	[2:0]	
s_axi_arqos	In	[3:0]	
s_axi_arregion	In	[3:0]	
s_axi_aruser	In	[UW-1:0]	
s_axi_arvalid	In	1	
s_axi_arready	Out	1	
s_axi_rid	Out	[IW-1:0]	
s_axi_rdata	Out	[DW-1:0]	
s_axi_rresp	Out	[1:0]	

Port	Dir	Width	Description
s_axi_rlast	Out	1	
s_axi_ruser	Out	[UW-1:0]	
s_axi_rvalid	Out	1	
s_axi_rready	In	1	
fub_axi_arid	Out	[IW-1:0]	
fub_axi_araddr	Out	[AW-1:0]	
fub_axi_arlen	Out	[7:0]	
fub_axi_arsize	Out	[2:0]	
fub_axi_arburst	Out	[1:0]	
fub_axi_arlock	Out	1	
fub_axi_arcache	Out	[3:0]	
fub_axi_arprot	Out	[2:0]	
fub_axi_arqos	Out	[3:0]	
fub_axi_arregion	Out	[3:0]	
fub_axi_aruser	Out	[UW-1:0]	
fub_axi_arvalid	Out	1	
fub_axi_arready	In	1	
fub_axi_rid	In	[IW-1:0]	
fub_axi_rdata	In	[DW-1:0]	
fub_axi_rresp	In	[1:0]	
fub_axi_rlast	In	1	
fub_axi_ruser	In	[UW-1:0]	
fub_axi_rvalid	In	1	
fub_axi_rready	Out	1	

Port	Dir	Width	Description
cg_gating	Out	1	
cg_idle	Out	1	

16.4 Functional Description

This wrapper is the base module plus one amba_clock_gate_ctrl instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. amba_clock_gate_ctrl watches two activity terms, user_valid (this side's valids plus the base module's busy) and axi_valid (the far side's valids), registers their OR into r_wakeup, and stops the inner module's clock once both have been quiet for cfg_cg_idle_count cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with !cg_gating, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

cfg_cg_enable arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

16.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

16.6 Usage Examples

```
axi4_slave_rd_cg #(
    // Base module parameters (see axi4_slave_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_slave_rd (except busy)
);
```

16.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

16.8 Related Modules

- **Base Module Functionality:** axi4_slave_rd.md
- **Clock Gating Guide:** clock_gated_variants.md

- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

16.9 Testing

`val/amba/test_axi4_slave_rd_cg.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_slave_rd_cg.py -v
```

16.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

17 AXI4 Slave Read Monitor

Module: `axi4_slave_rd_mon.sv` **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

17.1 Overview

The slave-side counterpart to the master monitors. The AXI4 Slave Read Monitor combines a working `axi4_slave_rd` with an `axi_monitor_filtered`, watching read transactions as they pass from the interconnect through to your backend and back. You get real-time protocol checking, error detection, performance metrics, and configurable packet filtering — from the slave's point of view, which is usually the angle that catches backend problems first.

17.1.1 Key Features

- **Integrated Monitoring:** Combines `axi4_slave_rd` with `axi_monitor_filtered`
- **2-Level Filtering:** packet-type drop masks + per-event masks (err_select is reserved – no routing)

- **Error Detection:** SLVERR, DECERR, orphaned read data, timeouts (protocol-violation events are write-monitor only – see Error Detection Events)
- **Timeout Monitoring:** Configurable timeout detection for stuck transactions
- **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
- **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
- **Configuration Validation:** Detects conflicting configuration settings
- **Clock Gating Support:** Busy signal for power management

17.2 Parameters

17.2.1 AXI4 Slave Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	2	AR channel skid buffer depth
`SKID_DEPTH_R`	int	4	R channel skid buffer depth
`AXI_ID_WIDTH`	int	8	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	Address bus width
`AXI_DATA_WIDTH`	int	32	Data bus width
`AXI_USER_WIDTH`	int	1	User signal width

17.2.2 Monitor Parameters

Parameter	Type	Default	Description
`UNIT_ID`	logic [7:0]	8'h02	8-bit unit identifier in monitor packets
`AGENT_ID`	logic	16'h0014	16-bit agent identifier in

Parameter	Type	Default	Description
	[15:0]		monitor packets
`MAX_TRANSACTIONS`	int	16	Maximum concurrent outstanding transactions
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. >**1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
`ACLK_MHZ`	int	100	Clock frequency in MHz – keeps the 1 us tick exact off-100MHz
`CFI_MIN_FREQ_MHZ` / `CFI_MAX_FREQ_MHZ`	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
`ACTIVE_TRANS_THRESHOLD`	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when `cfg_threshold_enable=1`. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing

Parameter	Type	Default	Description
`ENABLE_FILTERING`	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
`ADD_PIPELINE_STAGE`	bit	0	Add register stage for timing closure
`USE_MONITOR`	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
`N_ADDR_RANGES`	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
`ADDR_RANGE_IS_ERROR`	logic [N_ADDR_RANGES-1:0]	`'0`	Per-range flavor: bit i = 0 -> DEBUG range (hit -> AddrMatch), 1 -> ERROR range (allowlist miss -> Error/ADDR_RANGE). Default all-0 (feature inert).

17.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a `generate`-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
`ENABLE_ERROR_LOGIC`	bit	1	Drop the error-detection cone
`ENABLE_TIMEOUT_LOGIC`	bit	1	Drop the timeout cone **and** the `axi_monitor_timeout` instance
`ENABLE_COMPL_LOGIC`	bit	1	Drop the completion cone
`ENABLE_THRESHOLD_LOGIC`	bit	1	Drop the threshold cone
`ENABLE_PERF_LOGIC`	bit	1	Gates only the reporter's legacy perf-packet cone and the two lifetime counters – the window FSM and meters are unconditional
`ENABLE_DEBUG_LOGIC`	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (the reporter's legacy perf cone is built; `ENABLE_PERF_LOGIC` gates THAT cone only, never the measurement window) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

17.2.4 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`

Derived parameter	Default expression
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH

17.3 Ports

17.3.1 AXI4 Read Channels

Slave Interface (s_axi_*): - AR channel: `arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser, arvalid, arready` - R channel: `rid, rdata, rresp, rlast, ruser, rvalid, rready`

Backend Interface (fub_axi_*): - Same signals as slave, mirrored direction (to memory/backend)

17.3.2 Monitor Configuration

Configuration ports are identical to other AXI4 monitors: - Basic enables: `cfg_monitor_enable` (master runtime gate: 0 = monitor inert, CAM held clear, never stalls the datapath), `cfg_error_enable`, `cfg_timeout_enable`, `cfg_perf_enable`, `cfg_compl_enable` (completion packets), `cfg_threshold_enable` (threshold-crossed packets), `cfg_debug_enable` (debug/trace cone — the 6th reporter sub-block) - Clear: `cam_clear` (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: `cfg_timeout_cycles` (unified coarse timeout, a MICROSECOND count at full 16-bit width: 1..65535 us per phase, 0 = 16'hFFFF ~ effectively never; drives all three phase counts), `cfg_latency_threshold` - Filtering: 8 mask signals (`cfg_axi_*_mask`: pkt, error, timeout, compl, thresh, perf, addr, debug) plus `cfg_axi_err_select` - Performance window control: `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close` (see Performance Monitoring)

> The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) are fixed inside the wrapper; `cfg_active_trans_threshold` is driven by the `ACTIVE_TRANS_THRESHOLD` parameter (default MAX_TRANSACTIONS/2), not hardwired to 8, and all three are **not** top-level ports on this module.

17.3.3 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Downstream ready to

Port	Direction	Width	Description
			accept packet
`monbus_packet`	Output	128	`monitor_packet_t` (see format below)
`monbus_timestamp`	Output	64	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_block_ready	Output	1	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)
`i_mon_time`	Input	64	Free-running counter from `monbus_group_core`, sampled at packet emission

17.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions (for clock gating)
`active_transactions`	Output	8	Current number of outstanding transactions
`error_count`	Output	16	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`transaction_count`	Output	32	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0

Port	Direction	Width	Description
			when ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
cfg_conflict_error	Output	1	Configuration conflict detected

17.3.5 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
cfg_addr_filter_enable	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
cfg_addr_filter_low	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	ADDR_WIDTH	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

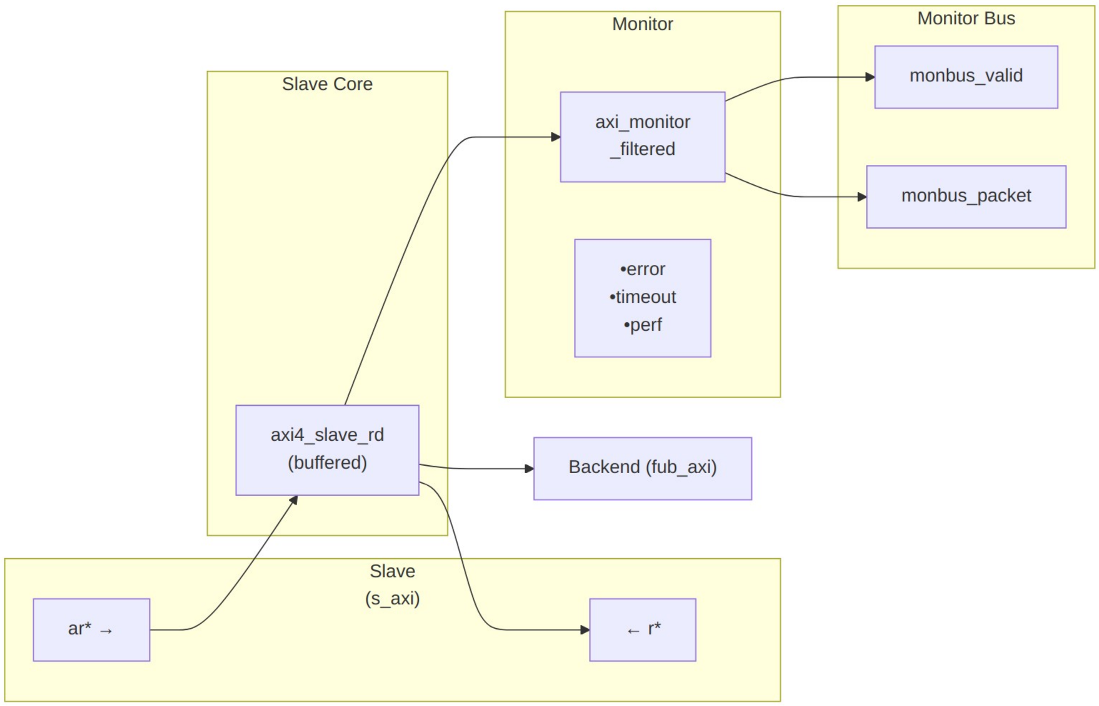
Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
cfg_id_filter_enable	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature

Port	Width	Description
`cfg_id_match_base`	`ID_WIDTH`	First ID owned
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

17.4 Functional Description



Architecture

The module instantiates two sub-modules: 1. **axi4_slave_rd** - Core AXI4 slave read functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)

17.4.1 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of MAX_TRANSACTIONS; the reporter FIFO depth INTR_FIFO_DEPTH has no path to it). The wrapper ANDs it into the upstream-facing s_axi_arready so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream s_axi_arready is forced low until the monitor drains.
- **When USE_MONITOR=0**: block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When cfg_monitor_enable=0**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (in-RTL formal property ap_disabled_never_stalls).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS) (reserve = 4 for tables of 16 or more, 0 below; the function lives in monitor_common_pkg), and block_ready re-asserts at occupancy < MAX_TRANSACTIONS - (reserve - 1) – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size MAX_TRANSACTIONS to cover NUM_CHANNELS x per-channel outstanding plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's --unroll-count raised (default 64) in sim builds.

17.4.2 Address-Range Checker

With N_ADDR_RANGES > 0 the wrapper instantiates the shared allowlist checker (axi_monitor_addr_check). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a PktTypeAddrMatch (4'h8) packet with event code AXI_ADDR_RANGE_MATCH (8'h01), gated by cfg_debug_enable.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a PktTypeError (4'h0) packet with event code AXI_ERR_ADDR_RANGE (8'h0D), gated by cfg_error_enable.

This wrapper exposes **ADDR_RANGE_IS_ERROR** (per-range) as a parameter, so ranges can be assigned to either flavor.

Config inputs (active only when N_ADDR_RANGES > 0): - cfg_addr_check_enable — master on/off for the checker. - cfg_addr_range_enable[N-1:0] — per-range enable bit. - cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0] — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

17.4.3 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`. The **measurement-window state machine** and its counters are unconditional `always_ff` blocks in `axi_monitor_base` – outside every generate – so they are present and running in EVERY build, including `ENABLE_PERF_LOGIC = 0`. That parameter reaches one consumer: the `g_perf` generate in `axi_monitor_reporter` holding the legacy perf-packet cone and the two 16-bit lifetime counters behind `perf_completed_count` / `perf_error_count`. Setting it to 0 saves that cone and nothing else. `USE_MONITOR = 0` is what ties the perfmon outputs off. The wrapper instantiates plus a bank of R-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

17.4.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

17.4.3.2 Utilization Counters (R Data Channel)

Every cycle inside the window is classified by the R channel's `rvalid` / `rready` into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	rvalid && rready	productive beat transferred
`perf_bp_cycles`	32	rvalid && !rready	back-pressure (data offered, sink not ready)
`perf_starv_cyc`	32	!rvalid && rready	starvation (sink

Output	Width	Condition	Meaning
les			ready, no data)
perf_idle_cycles	32	!rvalid && !rready	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); the one-count skew is negligible for long windows.

17.4.3.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	R data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats × (1 << ARSIZE), using the ARSIZE captured at the most recent AR address phase (upper bound: counts full-width beats; an unaligned start address means the first beat carries fewer useful bytes)
`perf_burst_count`	32	AR address-phase handshakes

The integrator computes average burst length as `perf_beat_count / perf_burst_count`.

17.4.3.4 Performance Monitoring Ports

Port	Direction	Width	Description
`cfg_perf_enable`	Input	1	Enable performance-metric packet generation
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end** event source select
`cfg_start_trigger`	Input	1	Pulse: open the measurement window

Port	Direction	Width	Description
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod_cycles	Output	32	rvalid && rready cycles
perf_bp_cycles	Output	32	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	Output	32	!rvalid && rready cycles (starvation)
perf_idle_cycles	Output	32	!rvalid && !rready cycles
perf_beat_count	Output	32	R data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AR address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

17.4.4 Monitor Packet Format

Identical 128-bit format (with 64-bit side-band timestamp) as other AXI4 monitors. See axi4_master_rd_mon for complete specification.

17.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

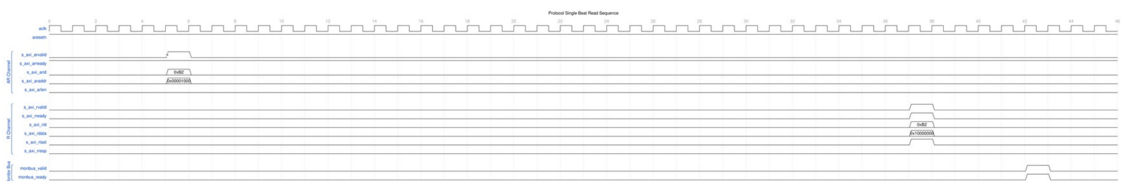
No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

17.6 Waveforms

The following waveforms show AXI4 slave read monitor behavior from the slave perspective:

17.6.1 Scenario 1: Single-Beat Read (Slave View)

AXI4 read transaction from slave interface perspective:



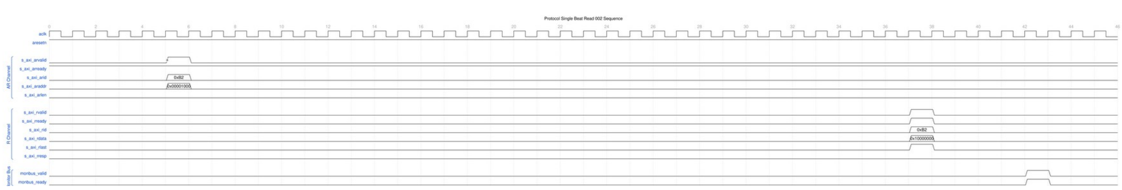
Single Beat Read Slave

WaveJSON: single_beat_read_001.json

Key Observations: - Slave-side AR channel (s_axi_ar*) - Slave-side R channel response (s_axi_r*) - Monitor packet generation from slave perspective - Slave RRESP status monitoring - Transaction tracking in slave context

17.6.2 Scenario 2: Alternative Single-Beat Read (Slave View)

Variant read transaction with different timing from slave:



Single Beat Read Slave Alt

WaveJSON: single_beat_read_002_001.json

Key Observations: - Slave ready signal behavior - ARREADY backpressure effects - RVALID generation timing - Slave latency monitoring - Error detection from slave side

---

17.7 Usage Examples

17.7.1 Configuration Strategies

17.7.1.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch slave-side read errors

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),      // Detect SLVERR/DECERR
.cfg_timeout_enable     (1'b1),      // Detect backend timeouts
.cfg_perf_enable        (1'b0),      // Disable (reduces traffic)

// Filtering - pass error and timeout only
.cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR, TIMEOUT
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask   (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask      (16'hFFFF),  // Drop performance

// Timeouts
.cfg_timeout_cycles     (16'd10),    // 10 microseconds per phase
(full 16-bit range)  // Backend response timeout
.cfg_freq_sel         (4'd0),    // counter_freq_invariant LUT index;
scales the 1 us tick
.cam_clear            (1'b0),    // hold high one cycle while idle to
clear the CAM -- do NOT leave unconnected
.cfg_latency_threshold  (32'd500)
```

17.7.1.2 Strategy 2: Performance Analysis

Goal: Analyze slave read performance

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),
.cfg_timeout_enable     (1'b0),
.cfg_perf_enable        (1'b1),      // Enable performance metrics

// Filtering - pass error and performance only
.cfg_axi_pkt_mask       (16'hFFEE),  // Drop all but ERROR, PERF (set
bit = drop)
```

```
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_perf_mask      (16'h0000),  // Pass all performance
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
```

17.7.2 Basic Integration with Memory

```
axi4_slave_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (2),
    .AGENT_ID           (20),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_slave_rd_mon (
    .aclk               (axi_aclk),
    .aresetn            (axi_aresetn),

    // Slave interface (from interconnect)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    // ... rest of AR/R signals

    // Backend interface (to memory controller)
    .fub_axi_arid       (mem_arid),
    .fub_axi_araddr     (mem_araddr),
    // ... rest of AR/R signals

    // Monitor configuration
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd15),    // 15 microseconds per phase: 
allow memory latency
    .cfg_latency_threshold  (32'd1000),

    .cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR, 
TIMEOUT
    .cfg_axi_error_mask     (16'h0000),
    .cfg_axi_timeout_mask   (16'h0000),
    .cfg_axi_compl_mask     (16'hFFFF),
    // ... rest of mask signals

    // Monitor bus output
```

```
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (rd_slave_busy),
    .active_transactions    (rd_active),
    .error_count            (rd_errors),
    .transaction_count      (rd_count),
    .cfg_conflict_error     (cfg_conflict)
);

// Memory controller backend
memory_controller u_mem (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_arid       (mem_arid),
    .axi_araddr     (mem_araddr),
    // ...
);
```

17.8 Design Notes

17.8.1 Slave-Side Monitoring

Key Differences from Master Monitors: - Monitors from slave perspective (interconnect → backend) - Tracks backend response latency - Detects backend timeout scenarios - Default UNIT_ID=2, AGENT_ID=20 (distinguishes from master)

Slave Read Sequence: 1. AR channel: Master request arrives at slave 2. Monitor captures: Address, ID, burst parameters 3. AR forwarded to backend (memory/logic) 4. R channel: Backend returns data 5. Monitor tracks: Response latency, RLAST, RRESP 6. R forwarded back to master via interconnect

Common Slave Errors Detected: - **Backend timeout:** AR accepted but R never returned - **SLVERR:** Slave error (access violation, parity error) - **DECERR:** Decode error (shouldn't occur at slave, but detected) - (Read burst-length mismatch is NOT checked: reads complete on RLAST or error RRESP; expected_beats is stored but never compared) - **ID corruption:** RID doesn't match tracked ARID

17.8.2 Performance Considerations

Backend Latency Monitoring: - Tracks AR -> R LAST-beat latency (data_timestamp is written on the final beat; there is no first-beat timestamp) - Configurable timeout threshold - Separate from master-side latency

Typical Timeout Values: - SRAM backend: 1-5 us - DDR controller: 10-50 us - PCIe/external: 100+ us (the knob is a MICROSECOND count, not clock cycles)

17.8.3 Buffer Depth Guidelines

Same as axi4_slave_rd: - **SKID_DEPTH_AR:** 2 (default) - handles interconnect backpressure - **SKID_DEPTH_R:** 4 (default) - buffers backend burst data - Increase for high-latency backends or large bursts

---

17.9 Related Modules

17.9.1 Companion Monitors

- **axi4_master_rd_mon** - AXI4 master read with monitoring
- **axi4_master_wr_mon** - AXI4 master write with monitoring
- **axi4_slave_wr_mon** - AXI4 slave write with monitoring

17.9.2 Base Modules

- **axi4_slave_rd** - Functional AXI4 slave read (without monitoring)
- **axi_monitor_filtered** - Monitoring engine with filtering (monitor/)

17.9.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **axi_monitor_base** - Core monitoring logic (monitor/)
- **axi_monitor_trans_mgr** - Transaction tracking (monitor/)

17.10 Testing

`val/amba/test_axi4_slave_rd_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_slave_rd_mon.py -v
```

17.11 References

17.11.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: monitor_package_spec.md

17.11.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_rd_mon.sv`
- Tests: `val/amba/test_axi4_slave_rd_mon.py`
- Framework: `bin/TBClasses/components/axi4/`

17.11.3 Documentation

- Configuration Guide: AXI Monitor Base
- Architecture: rtl-amba Overview
- AXI4 Index: README.md

Last Updated: 2026-07-18

17.12 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

18 AXI4 Slave Read Monitor (Clock-Gated)

Module: `axi4_slave_rd_mon_cg.sv` **Base Module:** axi4_slave_rd_mon **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

18.1 Overview

This is the **clock-gated variant** of axi4_slave_rd_mon — the same monitored slave read, with activity-based clock gating wrapped around it.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the **Clock-Gated Variants Guide**.

What the wrapper buys you:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable:** Idle threshold, gating domains, enable/disable
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate at runtime

---

18.2 Parameters

MOST axi4_slave_rd_mon parameters pass through. As of 2026-09-01 only `ACTIVE_TRANS_THRESHOLD` is NOT forwarded, and that one is harmless: its inner default is `MAX_TRANSACTIONS/2`, computed from the `MAX_TRANSACTIONS` this wrapper DOES forward.

An earlier version of this page listed nine more as unforwarded, which was accurate when written. `ACLK_MHZ`, `CFI_MIN_FREQ_MHZ`, `CFI_MAX_FREQ_MHZ`, `USE_WDATA_ORDER_Q`, `NUM_BANKS` and `ADDR_RANGE_IS_ERROR` were threaded through every `_cg` wrapper on 2026-09-01, and `ID_FILTER_ENABLE` / `ID_MATCH_BASE` / `ID_MATCH_COUNT` the day before. Until then a clock-gated build could not state its clock frequency, so the 1 us timer tick was pinned to the 100 MHz default and every microsecond-denominated timeout was miscalibrated on any other clock – silently.

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`USE_MONITOR`	1	Synthesis-time monitor enable (forwarded to inner monitor).
`N_ADDR_RANGES`	0	Number of address-range comparators (forwarded to base module).
`ADDR_FILTER_ENABLE`	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves

Parameter	Default	Description
		`cfg_addr_filter_enable` low filters nothing and looks broken.
`ID_FILTER_ENABLE`	0	Synthesises the ID report filter (see `cfg_id_*` for the runtime override).
`ID_MATCH_BASE`	0	First ID this instance owns.
`ID_MATCH_COUNT`	0	How many IDs; `0` means ALL, so a zeroed register block does not silently filter everything away.
`ADD_PIPELINE_STAGE`	`0`	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
`ENABLE_FILTERING`	`1`	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.
`SKID_DEPTH_AR`	`2`	Skid-buffer depth on the AR channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_R`	`4`	Skid-buffer depth on the R channel. Legal range 2..8 inclusive; odd depths are legal.
`AGENT_ID`	`16'h0014`	Agent identifier emitted in the `agent_id` field of every monitor packet. Pairs with `UNIT_ID` to identify the packet source. (16-bit Agent ID for monitor packets)
`UNIT_ID`	`8'h02`	Unit identifier emitted in the

Parameter	Default	Description
		`unit_id` field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

Gating is controlled by RUNTIME inputs `cfg_cg_enable` / `cfg_cg_idle_count` with status outputs `cg_gating` / `cg_idle`; ONE `amba_clock_gate_ctrl` gates the entire inner module. (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

Base-module ports are forwarded EXCEPT `debug_block_ready`, which this wrapper ties off (use the base module for the backpressure tap) – including the full performance-monitoring interface (see Performance Monitoring below). The six `ENABLE_*_LOGIC` synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables are also passed straight through.

18.2.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters

Port	Direction	Width	Description
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

Neither filter un-filters entries already admitted to the transaction table, which is what makes changing them at runtime safe.

18.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

18.3 Ports

Port	Dir	Width	Description
`aclk`	In	1	
`aresetn`	In	1	
`cam_clear`	In	1	sync clear of the monitor trans CAM
`s_axi_arid`	In	`[IW-1:0]`	
`s_axi_araddr`	In	`[AW-1:0]`	
`s_axi_arlen`	In	`[7:0]`	
`s_axi_arsize`	In	`[2:0]`	
`s_axi_arburst`	In	`[1:0]`	
`s_axi_arlock`	In	1	

Port	Dir	Width	Description
s_axi_arcache	In	[3:0]	
s_axi_arprot	In	[2:0]	
s_axi_arqos	In	[3:0]	
s_axi_arregion	In	[3:0]	
s_axi_aruser	In	[UW-1:0]	
s_axi_arvalid	In	1	
s_axi_arready	Out	1	
s_axi_rid	Out	[IW-1:0]	
s_axi_rdata	Out	[DW-1:0]	
s_axi_rresp	Out	[1:0]	
s_axi_rlast	Out	1	
s_axi_ruser	Out	[UW-1:0]	
s_axi_rvalid	Out	1	
s_axi_rready	In	1	
fub_axi_arid	Out	[IW-1:0]	
fub_axi_araddr	Out	[AW-1:0]	
fub_axi_arlen	Out	[7:0]	
fub_axi_arsize	Out	[2:0]	
fub_axi_arburst	Out	[1:0]	
fub_axi_arlock	Out	1	
fub_axi_arcache	Out	[3:0]	
fub_axi_arprot	Out	[2:0]	
fub_axi_arqos	Out	[3:0]	
fub_axi_arregion	Out	[3:0]	
fub_axi_aruser	Out	[UW-1:0]	
fub_axi_arvalid	Out	1	
fub_axi_arready	In	1	
fub_axi_rid	In	[IW-1:0]	
fub_axi_rdata	In	[DW-1:0]	
fub_axi_rresp	In	[1:0]	

Port	Dir	Width	Description
`fub_axi_rlast`	In	1	
`fub_axi_ruser`	In	`[UW-1:0]`	
`fub_axi_rvalid`	In	1	
`fub_axi_rready`	Out	1	
`cfg_monitor_enable`	In	1	Enable monitoring
`cfg_error_enable`	In	1	Enable error detection
`cfg_timeout_enable`	In	1	Enable timeout detection
`cfg_perf_enable`	In	1	Enable performance monitoring
`cfg_compl_enable`	In	1	Enable completion packets
`cfg_threshold_enable`	In	1	Enable threshold packets
`cfg_debug_enable`	In	1	Enable debug packets
`cfg_timeout_cycles`	In	`[15:0]`	Timeout threshold in MICROSECONDS (1 us tick), despite the name
`cfg_freq_sel`	In	`[3:0]`	counter_freq_invariant LUT index
`cfg_latency_threshold`	In	`[31:0]`	Latency threshold for alerts
`cfg_axi_pkt_mask`	In	`[15:0]`	Drop mask for packet types
`cfg_axi_err_select`	In	`[15:0]`	Error select for packet types (for

Port	Dir	Width	Description
			future routing)
cfg_axi_error_mask	In	[15:0]	Individual error event mask
cfg_axi_timeout_mask	In	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	In	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	In	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	In	[15:0]	Individual performance event mask
cfg_axi_addr_mask	In	[15:0]	Individual address match event mask
cfg_axi_debug_mask	In	[15:0]	Individual debug event mask
cfg_cg_enable	In	1	Enable clock gating
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	In	1	
cfg_addr_range_enable	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_low	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_high	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES :	

Port	Dir	Width	Description
		1)-1:0]	
cfg_addr_filter_enable	In	1	
cfg_addr_filter_low	In	[AW-1:0]	
cfg_addr_filter_high	In	[AW-1:0]	
cfg_id_filter_enable	In	1	
cfg_id_match_base	In	[IW-1:0]	
cfg_id_match_count	In	[IW:0]	
i_mon_time	In	1	
monbus_valid	Out	1	Monitor bus valid
monbus_ready	In	1	Monitor bus ready
monbus_packet	Out	1	Monitor packet (128-bit)
monbus_timestamp	Out	1	Side-band sampled time
busy	Out	1	
active_transactions	Out	[7:0]	Number of active transactions
error_count	Out	[15:0]	Total error count
transaction_count	Out	[31:0]	Total transaction count
cg_gating	Out	1	Gated clock is stopped
cg_idle	Out	1	No activity observed
cfg_conflict_error	Out	1	Configuration conflict detected
cfg_start_event_sel	In	[2:0]	
cfg_end_event_sel	In	[2:0]	

Port	Dir	Width	Description
cfg_start_trigger	In	1	
cfg_end_trigger	In	1	
cfg_window_force_close	In	1	
window_active	Out	1	
window_cycles	Out	[31:0]	
perf_prod_cycles	Out	[31:0]	
perf_bp_cycles	Out	[31:0]	
perf_starv_cycles	Out	[31:0]	
perf_idle_cycles	Out	[31:0]	
perf_beat_count	Out	[31:0]	
perf_byte_count	Out	[63:0]	
perf_burst_count	Out	[31:0]	

18.4 Functional Description

18.4.1 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_slave_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi4_slave_rd_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. **WARNING – gating vs window accounting:** an open measurement window is NOT a wake term, and the entire inner monitor (window state machine and counters included) runs on the gated clock. If the bus idles past `cfg_cg_idle_count` with a window open, the counters FREEZE while wall-clock time passes, and trigger pulses arriving while gated are DROPPED. For exact wall-clock windows or idle-bus triggering, hold `cfg_cg_enable` low around the measurement, or use the base module.

18.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AR`	2 entries
`SKID_DEPTH_R`	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

18.6 Usage Examples

```
axi4_slave_rd_mon_cg #(
    // Base module parameters (see axi4_slave_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_slave_rd_mon (except
debug_block_ready)
);
```

18.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

18.8 Related Modules

- **Base Module Functionality:** axi4_slave_rd_mon.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

18.9 Testing

`val/amba/test_axi4_slave_rd_mon_cg.py` exercises this module. It collects 3 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/amba/test_axi4_slave_rd_mon_cg.py -v
```

18.10 Navigation

- ← Back to AXI4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

19 AXI4 Slave Read Stub

Module: `axi4_slave_rd_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

19.1 Overview

The other end of the read transaction. The AXI4 Slave Read Stub receives AXI4 read transactions from a master and exposes them as simple packet interfaces — the AR channel arrives as one packed payload for your testbench to chew on, and you hand read data back the same way. Skid buffers do the pack/unpack work, which makes the stub ideal for testbenches and integration scenarios where a simplified slave interface is what you actually need.

19.1.1 Key Features

- Packed packet interface for AR and R channels
- Configurable skid buffer depth on each channel
- Full AXI4 read transaction support
- Burst, ID, and user signal support
- Parameterized data widths

19.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel skid buffer depth in entries (2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	R channel skid buffer depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width

Parameter	Type	Default	Description
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

19.3 Ports

19.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	AXI reset (active low)

19.3.2 AXI4 Read Address Channel (AR)

Port	Direction	Width	Description
s_axi_arid	Input	IW	Read address ID
s_axi_araddr	Input	AW	Read address
s_axi_arlen	Input	8	Burst length

Port	Direction	Width	Description
s_axi_arsize	Input	3	Burst size
s_axi_arburst	Input	2	Burst type
s_axi_arlock	Input	1	Lock type
s_axi_arcache	Input	4	Cache type
s_axi_arprot	Input	3	Protection type
s_axi_arqos	Input	4	Quality of service
s_axi_arregion	Input	4	Region identifier
s_axi_aruser	Input	UW	User signal
s_axi_arvalid	Input	1	Read address valid
s_axi_arready	Output	1	Read address ready

19.3.3 AXI4 Read Data Channel (R)

Port	Direction	Width	Description
s_axi_rid	Output	IW	Read data ID
s_axi_rdata	Output	DW	Read data
s_axi_rresp	Output	2	Read response
s_axi_rlast	Output	1	Read last
s_axi_ruser	Output	UW	User signal
s_axi_rvalid	Output	1	Read data valid
s_axi_rready	Input	1	Read data ready

19.3.4 AR Packet Interface

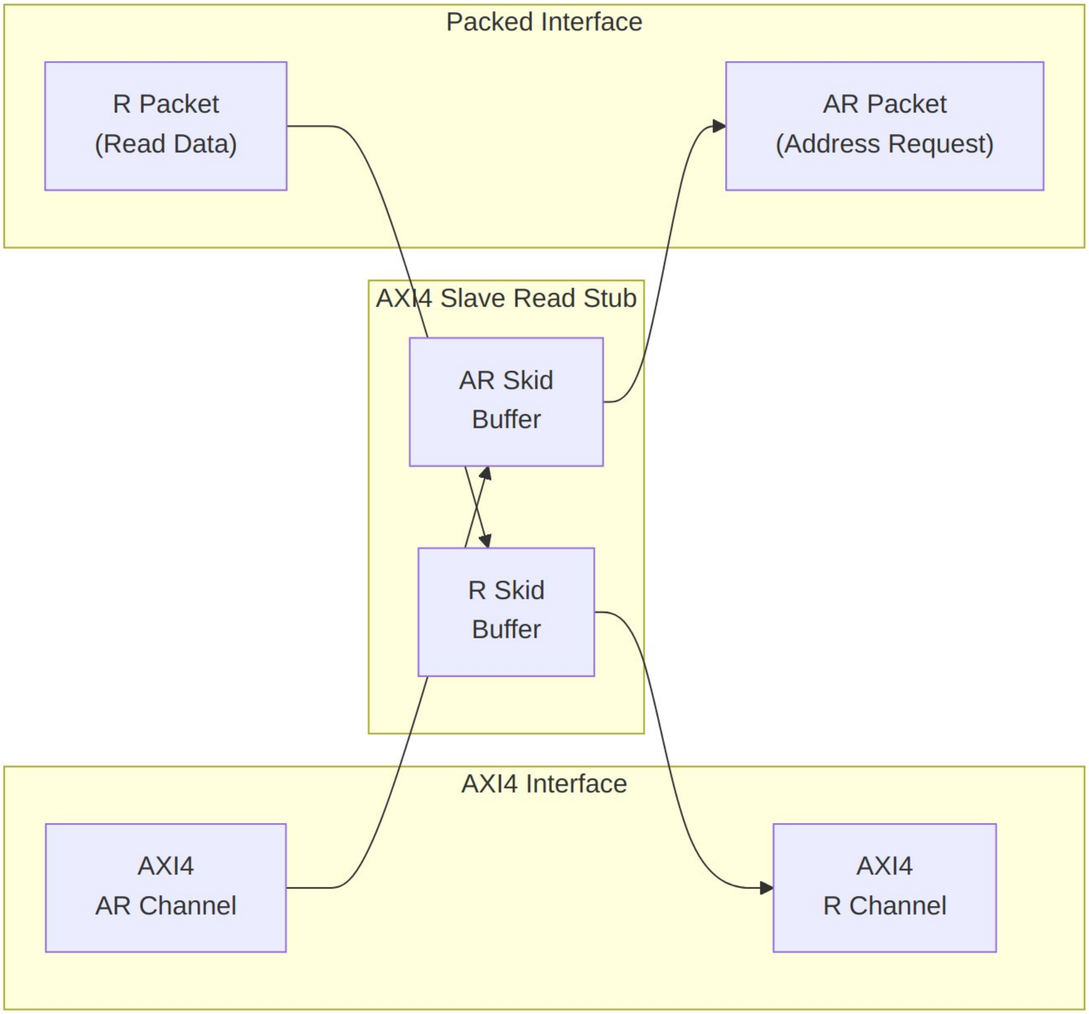
Port	Direction	Width	Description
fub_axi_arvalid	Output	1	AR packet valid
fub_axi_arready	Input	1	Ready to accept AR

Port	Direction	Width	Description
			packet
fub_axi_ar_count	Output	4	AR buffer occupancy
fub_axi_ar_pkt	Output	ARSize	Packed AR packet data

19.3.5 R Packet Interface

Port	Direction	Width	Description
fub_axi_rvalid	Input	1	R packet valid
fub_axi_rready	Output	1	Ready to accept R packet
fub_axi_r_pkt	Input	RSize	Packed R packet data

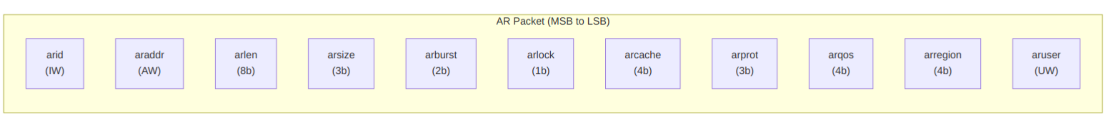
19.4 Functional Description



Architecture

19.4.1 Packet Formats

Packets are packed MSB-to-LSB in AXI signal order, so parsing one in the testbench is plain slicing.

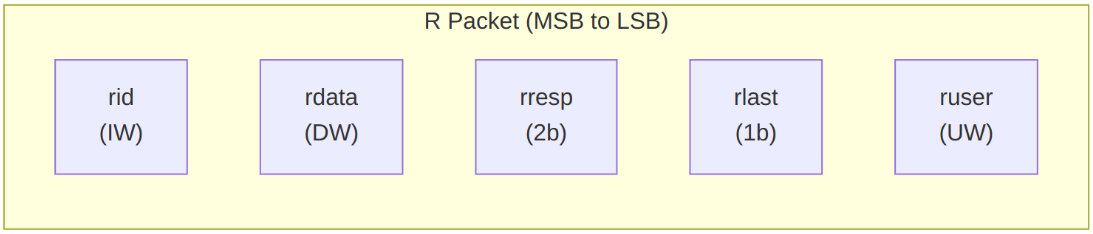


AR Packet Structure (Read Address)

Bit Positions:

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
```

```
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

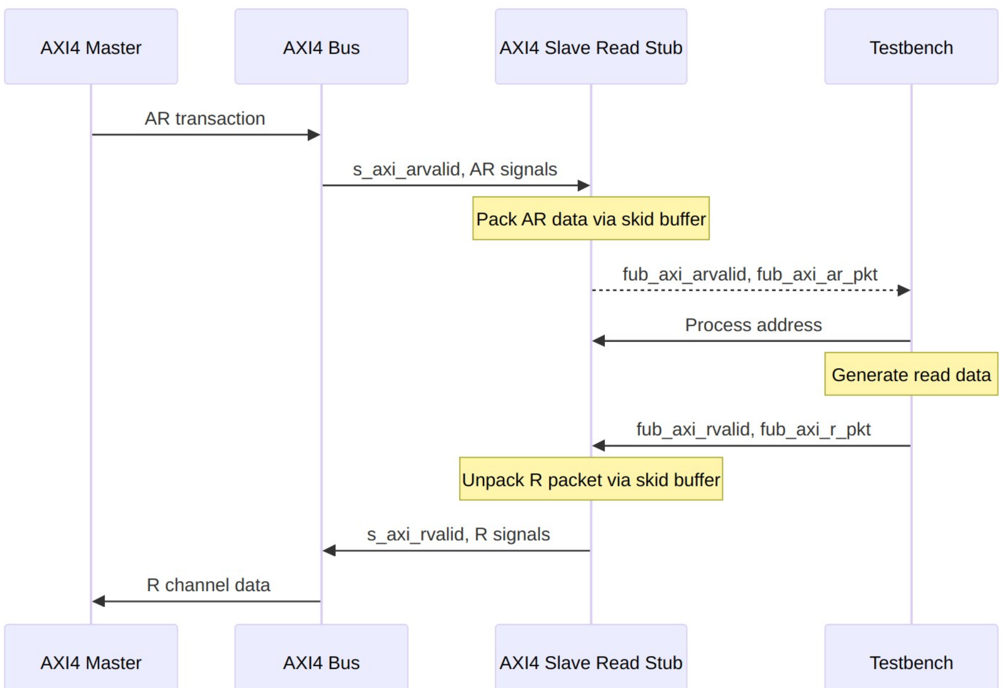


R Packet Structure (Read Data)

Bit Positions:

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
```

```
Width = IW + DW + 2 + 1 + UW
```



Transaction Flow

19.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- AXI AR signals (s_axi_arvalid, s_axi_araddr, s_axi_arlen, etc.)
- fub_axi_arvalid, fub_axi_arready, fub_axi_ar_pkt
- fub_axi_rvalid, fub_axi_rready, fub_axi_r_pkt
- AXI R signals (s_axi_rvalid, s_axi_rdata, s_axi_rlast, etc.)
- Packet-to-AXI timing relationship with skid buffer operation

---

19.6 Usage Examples

```
axi4_slave_rd_stub #(
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_rd_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave read interface
    .s_axi_arid      (s_axi_arid),
    .s_axi_araddr    (s_axi_araddr),
    .s_axi_arlen     (s_axi_arlen),
    .s_axi_arsize    (s_axi_arsize),
    .s_axi_arburst   (s_axi_arburst),
    .s_axi_arlock    (s_axi_arlock),
    .s_axi_arcache   (s_axi_arcache),
    .s_axi_arprot    (s_axi_arprot),
    .s_axi_arqos     (s_axi_arqos),
    .s_axi_arregion  (s_axi_arregion),
    .s_axi_aruser    (s_axi_aruser),
    .s_axi_arvalid   (s_axi_arvalid),
    .s_axi_arready   (s_axi_arready),

    .s_axi_rid       (s_axi_rid),
    .s_axi_rdata     (s_axi_rdata),
    .s_axi_rresp     (s_axi_rresp),
    .s_axi_rlast     (s_axi_rlast),
```

```
    .s_axi_ruser     (s_axi_ruser),
    .s_axi_rvalid    (s_axi_rvalid),
    .s_axi_rready    (s_axi_rready),

    // Packed AR interface
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    // Packed R interface
    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Parse AR packet
localparam int ARSize = 8+32+8+3+2+1+4+3+4+4+4;  // match your
instance widths
wire [7:0]  ar_id     = tb_ar_pkt[ARSize-1:ARSize-8];
wire [31:0] ar_addr   = tb_ar_pkt[ARSize-9:ARSize-40];
wire [7:0]  ar_len    = tb_ar_pkt[ARSize-41:ARSize-48];
wire [2:0]  ar_size   = tb_ar_pkt[ARSize-49:ARSize-51];
wire [1:0]  ar_burst  = tb_ar_pkt[ARSize-52:ARSize-53];
// ... additional fields as needed

// Build R packet (single beat response with data 0xDEADBEEFCAFEBABE)
localparam RSize = 8 + 64 + 2 + 1 + 4;  // Calculate size
assign tb_r_pkt = {
    ar_id,                       // rid (match request ID)
    64'hDEAD_BEEF_CAFE_BABE,    // rdata
    2'b00,                       // rresp (OKAY)
    1'b1,                        // rlast
    4'h0                         // ruser
};
```

19.7 Design Notes

19.7.1 Skid Buffer Operation

The stub is two `gaxi_skid_buffer` instances, and they earn their keep. They:

- Decouple timing between AXI bus and testbench
- Provide configurable buffering depth per channel

- Handle backpressure gracefully
- Support burst transactions without stalling

Recommended Depths: - **AR Channel:** 2-4 (address transactions) - **R Channel:** 4-8 (data beats for bursts)

19.7.2 Packet Packing Order

AR and R packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet parsing - Matches common concatenation order - Efficient for burst transaction handling

19.7.3 Internal Architecture

The stub instantiates two `gaxi_skid_buffer` modules: - **AR Skid Buffer:** Packs AXI AR channel to AR packets - **R Skid Buffer:** Unpacks R packets to AXI R channel

All AXI protocol handling is done by the skid buffers and upstream modules.

19.8 Related Modules

- **AXI4 Slave Read** - Full AXI4 slave read module (if wrapping one)
- **AXI4 Slave Write Stub** - Corresponding write stub
- **AXI4 Slave Stub** - Combined read/write stub
- **AXI4 Master Read Stub** - Master-side read stub

19.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

19.10 Navigation

- **← Back to AXI4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

20 AXI4 Slave Stub

Module: `axi4_slave_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

20.1 Overview

The AXI4 Slave Stub gives you a simplified packed-data interface for receiving both AXI4 read and write transactions from a master. It rolls the read and write slave stubs into one module – internally it just instantiates `axi4_slave_rd_stub` and `axi4_slave_wr_stub` – so you get a complete AXI4 slave interface with packet-based control and none of the protocol boilerplate. For testbenches and integration scenarios, this is the one I'd reach for first.

20.1.1 Key Features

- Combined read and write channel support
- Packed packet interfaces for all channels (AW, W, B, AR, R)
- Configurable skid buffer depths per channel
- Full AXI4 protocol support (bursts, IDs, user signals)
- Simplified testbench integration
- Parameterized data widths

20.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_W`	int	4	W channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_B`	int	2	B channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_AR`	int	2	AR channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_R`	int	4	R channel skid buffer

Parameter	Type	Default	Description
			depth in entries (2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

20.3 Ports

20.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock

Port	Direction	Width	Description
`aresetn`	Input	1	AXI reset (active low)

20.3.2 AXI4 Write Channels

Port	Direction	Width	Description
`s_axi_awid`	Input	AXI_ID_WIDTH	Write address ID
`s_axi_awaddr`	Input	AXI_ADDR_WIDTH	Write address
`s_axi_awlen`	Input	8	Burst length
`s_axi_awsize`	Input	3	Burst size
`s_axi_awburst`	Input	2	Burst type
`s_axi_awlock`	Input	1	Lock type
`s_axi_awcache`	Input	4	Cache type
`s_axi_awprot`	Input	3	Protection type
`s_axi_awqos`	Input	4	Quality of service
`s_axi_awregion`	Input	4	Region identifier
`s_axi_awuser`	Input	AXI_USER_WIDTH	User signal
`s_axi_awvalid`	Input	1	Write address valid
`s_axi_awready`	Output	1	Write address ready
`s_axi_wdata`	Input	AXI_DATA_WIDTH	Write data
`s_axi_wstrb`	Input	AXI_WSTRB_WIDTH	Write strobes
`s_axi_wlast`	Input	1	Write last
`s_axi_wuser`	Input	AXI_USER_WIDTH	User signal
`s_axi_wvalid`	Input	1	Write data

Port	Direction	Width	Description
			valid
`s_axi_wready`	Output	1	Write data ready
`s_axi_bid`	Output	AXI_ID_WIDTH	Response ID
`s_axi_bresp`	Output	2	Write response
`s_axi_buser`	Output	AXI_USER_WIDTH	User signal
`s_axi_bvalid`	Output	1	Write response valid
`s_axi_bready`	Input	1	Write response ready

20.3.3 AXI4 Read Channels

Port	Direction	Width	Description
`s_axi_arid`	Input	AXI_ID_WIDTH	Read address ID
`s_axi_araddr`	Input	AXI_ADDR_WIDTH	Read address
`s_axi_arlen`	Input	8	Burst length
`s_axi_arsize`	Input	3	Burst size
`s_axi_arburst`	Input	2	Burst type
`s_axi_arlock`	Input	1	Lock type
`s_axi_arcache`	Input	4	Cache type
`s_axi_arprot`	Input	3	Protection type
`s_axi_arqos`	Input	4	Quality of service
`s_axi_arregion`	Input	4	Region identifier
`s_axi_aruser`	Input	AXI_USER_WIDTH	User signal
`s_axi_arvalid`	Input	1	Read address valid

Port	Direction	Width	Description
s_axi_arready	Output	1	Read address ready
s_axi_rid	Output	AXI_ID_WIDTH	Read data ID
s_axi_rdata	Output	AXI_DATA_WIDTH	Read data
s_axi_rresp	Output	2	Read response
s_axi_rlast	Output	1	Read last
s_axi_ruser	Output	AXI_USER_WIDTH	User signal
s_axi_rvalid	Output	1	Read data valid
s_axi_rready	Input	1	Read data ready

20.3.4 Write Packet Interfaces

Port	Direction	Width	Description
fub_axi_awvalid	Output	1	AW packet valid
fub_axi_awready	Input	1	Ready to accept AW packet
fub_axi_aw_count	Output	4	AW buffer occupancy
fub_axi_aw_pkt	Output	AWSize	Packed AW packet data
fub_axi_wvalid	Output	1	W packet valid
fub_axi_wready	Input	1	Ready to accept W packet
fub_axi_w_pkt	Output	WSize	Packed W packet data
fub_axi_bvalid	Input	1	B packet valid

Port	Direction	Width	Description
`fub_axi_bready`	Output	1	Ready to accept B packet
`fub_axi_b_pkt`	Input	BSize	Packed B packet data

20.3.5 Read Packet Interfaces

Port	Direction	Width	Description
`fub_axi_arvalid`	Output	1	AR packet valid
`fub_axi_arready`	Input	1	Ready to accept AR packet
`fub_axi_ar_count`	Output	4	AR buffer occupancy
`fub_axi_ar_pkt`	Output	ARSize	Packed AR packet data
`fub_axi_rvalid`	Input	1	R packet valid
`fub_axi_rready`	Output	1	Ready to accept R packet
`fub_axi_r_pkt`	Input	RSize	Packed R packet data

20.4 Functional Description

20.4.1 Architecture

The combined stub is a thin wrapper – the channel-specific stubs do the real work, and the packed packet interfaces are what your testbench talks to:

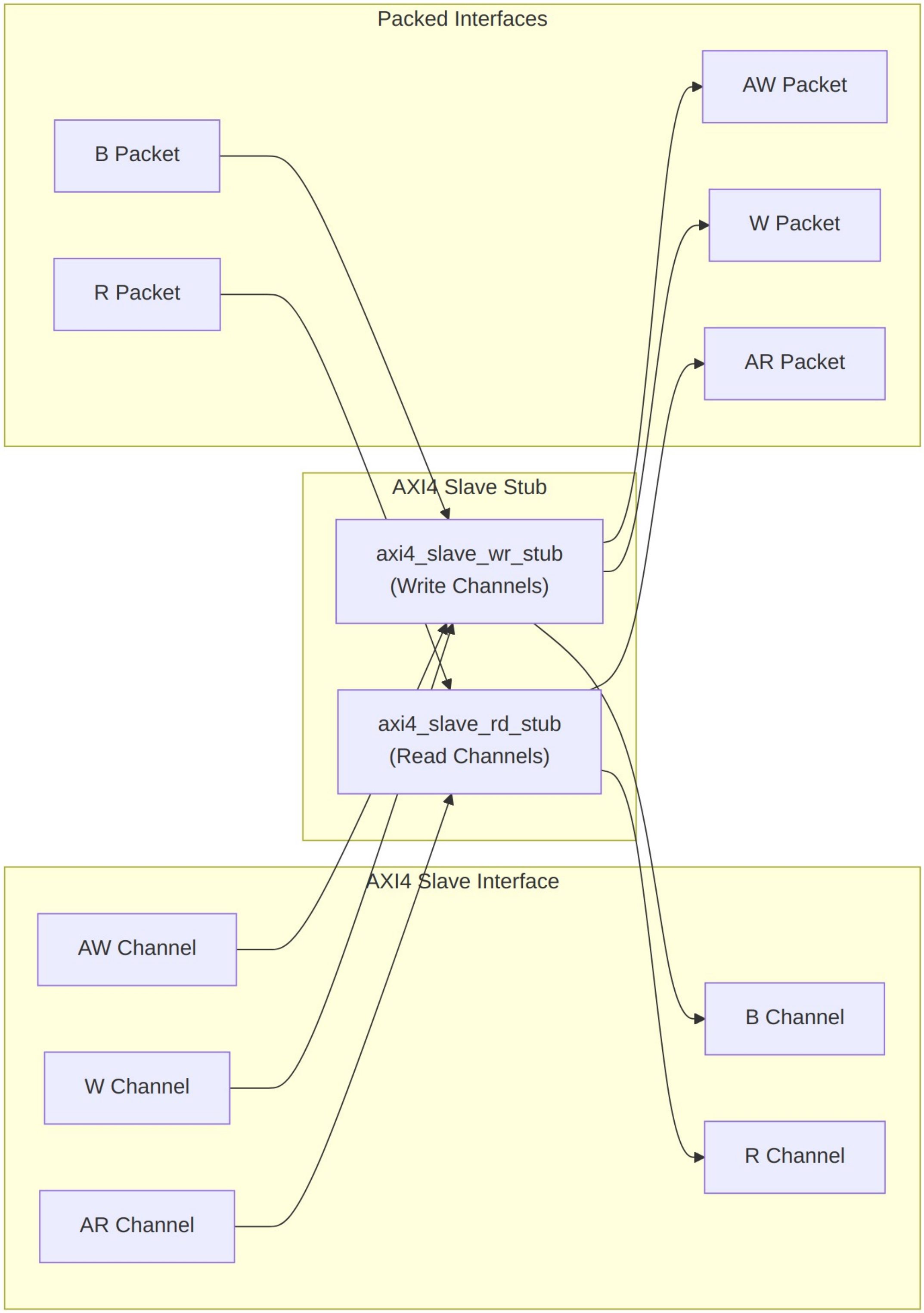

Packed Interfaces
AW Packet
B Packet
W Packet
R Packet
AR Packet
AXI4 Slave Stub
axi4_slave_wr_stub
(Write Channels)
axi4_slave_rd_stub
(Read Channels)
AXI4 Slave Interface
AW Channel
B Channel
W Channel
R Channel
AR Channel

Diagram 25

20.4.2 Packet Formats

AW Packet (Write Address):

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

W Packet (Write Data):

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}
Width = DW + SW + 1 + UW
```

B Packet (Write Response):

```
fub_axi_b_pkt = {bid, bresp, buser}
Width = IW + 2 + UW
```

AR Packet (Read Address):

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

R Packet (Read Data):

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
Width = IW + DW + 2 + 1 + UW
```

Complete packet format details: See AXI4 Slave Write Stub and AXI4 Slave Read Stub

20.4.3 Transaction Flow

Reads and writes move through the stub at the same time without getting in each other's way – here's what that looks like:

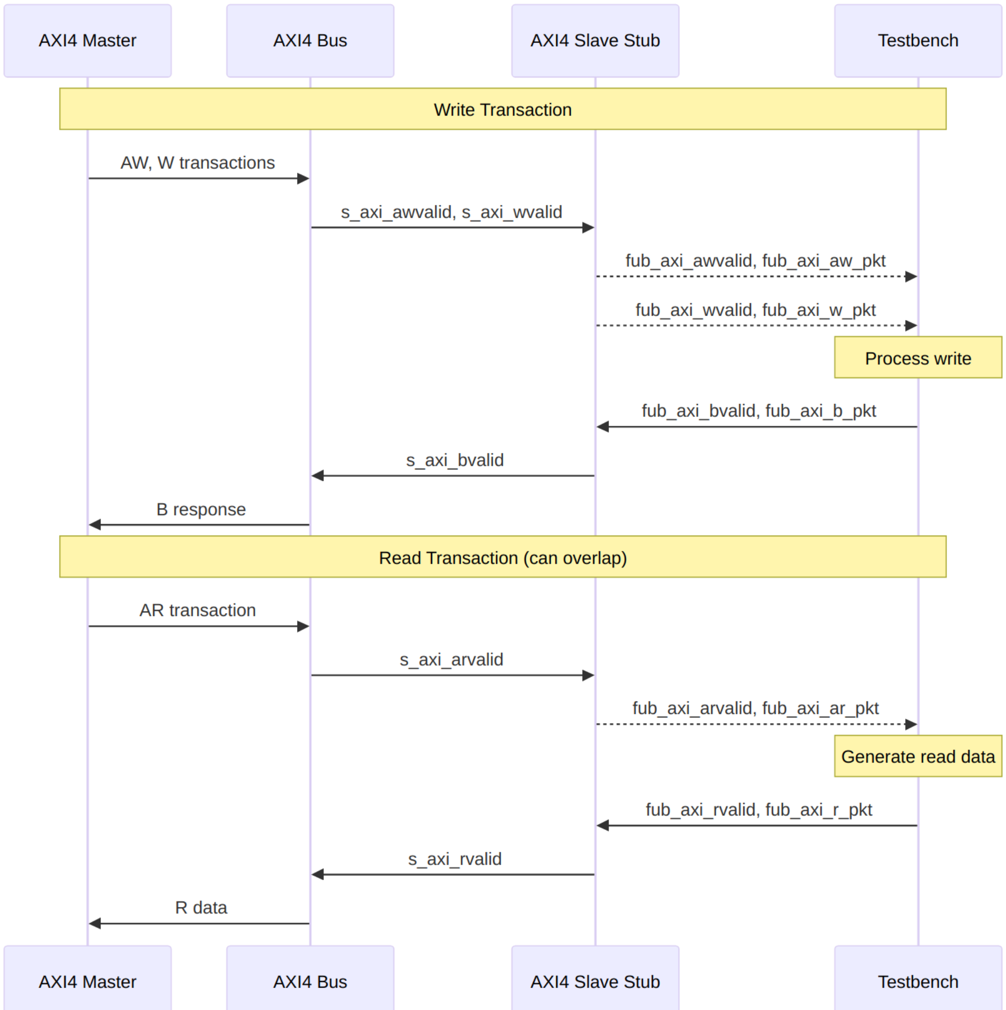


Diagram 26

20.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- AXI write channels (s_axi_aw, *s_axi_w*, s_axi_b*)
- Write packet interfaces (fub_axi_aw, *fub_axi_w*, fub_axi_b*)
- AXI read channels (s_axi_ar, *s_axi_r*)
- Read packet interfaces (fub_axi_ar, *fub_axi_r*)
- Overlapping read/write operations

20.6 Usage Examples

Wire it up, then parse and build packets in the testbench. The bit-slice localparams have to match your instance widths – get those wrong and you'll spend an afternoon chasing misaligned IDs. Don't ask how I know.

```
axi4_slave_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave interface (all channels)
    .s_axi_awid      (s_axi_awid),
    .s_axi_awaddr    (s_axi_awaddr),
    .s_axi_awlen     (s_axi_awlen),
    .s_axi_awsize    (s_axi_awsize),
    .s_axi_awburst   (s_axi_awburst),
    .s_axi_awlock    (s_axi_awlock),
    .s_axi_awcache   (s_axi_awcache),
    .s_axi_awprot    (s_axi_awprot),
    .s_axi_awqos     (s_axi_awqos),
    .s_axi_awregion  (s_axi_awregion),
    .s_axi_awuser    (s_axi_awuser),
    .s_axi_awvalid   (s_axi_awvalid),
    .s_axi_awready   (s_axi_awready),

    .s_axi_wdata     (s_axi_wdata),
    .s_axi_wstrb     (s_axi_wstrb),
    .s_axi_wlast     (s_axi_wlast),
    .s_axi_wuser     (s_axi_wuser),
    .s_axi_wvalid    (s_axi_wvalid),
    .s_axi_wready    (s_axi_wready),

    .s_axi_bid       (s_axi_bid),
    .s_axi_bresp     (s_axi_bresp),
    .s_axi_buser     (s_axi_buser),
```

```
    .s_axi_bvalid    (s_axi_bvalid),
    .s_axi_bready    (s_axi_bready),

    .s_axi_arid      (s_axi_arid),
    .s_axi_araddr    (s_axi_araddr),
    .s_axi_arlen     (s_axi_arlen),
    .s_axi_arsize    (s_axi_arsize),
    .s_axi_arburst   (s_axi_arburst),
    .s_axi_arlock    (s_axi_arlock),
    .s_axi_arcache   (s_axi_arcache),
    .s_axi_arprot    (s_axi_arprot),
    .s_axi_arqos     (s_axi_arqos),
    .s_axi_arregion  (s_axi_arregion),
    .s_axi_aruser    (s_axi_aruser),
    .s_axi_arvalid   (s_axi_arvalid),
    .s_axi_arready   (s_axi_arready),

    .s_axi_rid       (s_axi_rid),
    .s_axi_rdata     (s_axi_rdata),
    .s_axi_rresp     (s_axi_rresp),
    .s_axi_rlast     (s_axi_rlast),
    .s_axi_ruser     (s_axi_ruser),
    .s_axi_rvalid    (s_axi_rvalid),
    .s_axi_rready    (s_axi_rready),

    // Write packet interfaces
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt),

    // Read packet interfaces
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    .fub_axi_rvalid  (tb_r_valid),
```

```
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Parse incoming write address
localparam int AWSize = 8+32+8+3+2+1+4+3+4+4+4;  // match your
instance widths
wire [7:0]  aw_id     = tb_aw_pkt[AWSize-1:AWSize-8];
wire [31:0] aw_addr   = tb_aw_pkt[AWSize-9:AWSize-40];
wire [7:0]  aw_len    = tb_aw_pkt[AWSize-41:AWSize-48];
// ... additional fields

// Parse incoming write data
localparam int WSize = 64+8+1+4;  // match your instance widths
wire [63:0] w_data = tb_w_pkt[WSize-1:WSize-64];
wire [7:0]  w_strb = tb_w_pkt[WSize-65:WSize-72];
wire        w_last = tb_w_pkt[WSize-73];

// Generate write response (OKAY)
assign tb_b_pkt = {
    aw_id,    // bid (match request ID)
    2'b00,    // bresp (OKAY)
    4'h0      // buser
};

// Parse incoming read address
localparam int ARSize = 8+32+8+3+2+1+4+3+4+4+4;  // match your
instance widths
wire [7:0]  ar_id   = tb_ar_pkt[ARSize-1:ARSize-8];
wire [31:0] ar_addr = tb_ar_pkt[ARSize-9:ARSize-40];
wire [7:0]  ar_len  = tb_ar_pkt[ARSize-41:ARSize-48];
// ... additional fields

// Generate read data response
reg [63:0] read_data;  // From memory model
assign tb_r_pkt = {
    ar_id,      // rid (match request ID)
    read_data,  // rdata
    2'b00,      // rresp (OKAY)
    1'b1,       // rlast (single beat example)
    4'h0        // ruser
};
```

20.7 Design Notes

20.7.1 Internal Architecture

The stub instantiates two sub-modules: - **`axi4_slave_wr_stub`** - Handles AW, W, and B channels - **`axi4_slave_rd_stub`** - Handles AR and R channels

That hierarchy buys you a few things: it reuses the proven read/write stub modules, keeps the channel separation clean, simplifies verification and testing, and lets reads and writes operate independently.

20.7.2 Read/Write Independence

Read and write channels are completely independent: - Can overlap in time (simultaneous read and write) - Each has independent skid buffers - No ordering constraints between reads and writes - Testbench can respond at different rates

20.7.3 Skid Buffer Depths

Recommended configurations:

Low latency (fast response):

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(2), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(2)
```

Typical system:

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(4), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(4)
```

High throughput bursts:

```
.SKID_DEPTH_AW(4), .SKID_DEPTH_W(8), .SKID_DEPTH_B(4),
.SKID_DEPTH_AR(4), .SKID_DEPTH_R(8)
```

20.7.4 Memory Model Integration

The slave stub pairs naturally with a memory model – capture addresses and data on the write side, serve reads out of the array on the other:

```
// Simple memory model
reg [63:0] memory [0:1023];

// Handle write requests
always @(posedge aclk) begin
    if (tb_aw_valid && tb_aw_ready) begin
        // Capture write address
    end
    if (tb_w_valid && tb_w_ready) begin
```

```
        // Write data to memory
        memory[write_addr] <= w_data;
    end
end

// Handle read requests
always @(posedge aclk) begin
    if (tb_ar_valid && tb_ar_ready) begin
        // Generate read data from memory
        read_data <= memory[ar_addr];
    end
end
```

20.8 Related Modules

- **AXI4 Slave Read Stub** - Read-only stub (instantiated internally)
- **AXI4 Slave Write Stub** - Write-only stub (instantiated internally)
- **AXI4 Master Stub** - Corresponding combined master stub
- **AXI4 Slave Read** - Full AXI4 slave read module
- **AXI4 Slave Write** - Full AXI4 slave write module

20.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

20.10 Navigation

- **<- Back to AXI4 Index**
- **<- Back to rtl-amba Index**
- **<- Back to Main Documentation Index**

21 AXI4 Slave Write

Module: `axi4_slave_wr.sv` **Location:** `rtl/amba/axi4/` **Status:** Production Ready

21.1 Overview

The AXI4 Slave Write module is a buffered AXI4 slave write interface with configurable-depth skid buffers on all three write channels (AW, W, B). Think of it as an elastic buffer sitting between the AXI4 interconnect and your memory or backend processing element: it decouples the timing on the two sides and absorbs backpressure, so a sluggish backend doesn't stall the interconnect and an eager master doesn't overrun the backend.

21.1.1 Key Features

- **Full AXI4 Write Support:** Complete AW, W, and B channel implementation
- **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- **Elastic Buffering:** Decouples interconnect and backend timing domains
- **Burst Support:** Full burst transaction handling with WLAST tracking
- **User Signal Support:** Carries AWUSER, WUSER, and BUSER signals
- **Clock Gating Support:** Busy signal for dynamic power management

21.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	Depth of write address (AW) channel skid buffer
`SKID_DEPTH_W`	int	4	Depth of write data (W) channel skid buffer
`SKID_DEPTH_B`	int	2	Depth of write response (B) channel skid buffer
`AXI_ID_WIDTH`	int	8	Width of transaction ID signals (AWID, BID)
`AXI_ADDR_WIDTH`	int	32	Width of address bus (AWADDR)
`AXI_DATA_WIDTH`	int	32	Width of data bus (WDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024

Parameter	Type	Default	Description
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AXI_USER_WIDTH	int	1	Width of user-defined signals (AWUSER, WUSER, BUSER)

21.3 Ports

Here's the full module declaration – the tables below it break the ports down by group.

```
module axi4_slave_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Derived shorthands -- override the eight above, not these
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int SW       = AXI_WSTRB_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int AWSize   = IW+AW+8+3+2+1+4+3+4+4+UW,
    parameter int WSize    = DW+SW+1+UW,
    parameter int BSize    = IW+2+UW
) (
    // Clock and Reset
    input  logic                       aclk,
    input  logic                       aresetn,

    // Slave AXI Interface (s_axi_*)
    // Write address channel (AW)
    input  logic [AXI_ID_WIDTH-1:0]   s_axi_awid,
    input  logic [AXI_ADDR_WIDTH-1:0] s_axi_awaddr,
    input  logic [7:0]                 s_axi_awlen,
    input  logic [2:0]                 s_axi_awsize,
    input  logic [1:0]                 s_axi_awburst,
    input  logic                       s_axi_awlock,
    input  logic [3:0]                 s_axi_awcache,
    input  logic [2:0]                 s_axi_awprot,
```

```
    input  logic [3:0]                 s_axi_awqos,
    input  logic [3:0]                 s_axi_awregion,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_awuser,
    input  logic                       s_axi_awvalid,
    output logic                       s_axi_awready,

    // Write data channel (W)
    input  logic [AXI_DATA_WIDTH-1:0] s_axi_wdata,
    input  logic [SW-1:0]               s_axi_wstrb,
    input  logic                       s_axi_wlast,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_wuser,
    input  logic                       s_axi_wvalid,
    output logic                       s_axi_wready,

    // Write response channel (B)
    output logic [AXI_ID_WIDTH-1:0]   s_axi_bid,
    output logic [1:0]                 s_axi_bresp,
    output logic [AXI_USER_WIDTH-1:0] s_axi_buser,
    output logic                       s_axi_bvalid,
    input  logic                       s_axi_bready,

    // Backend Interface (fub_axi_* - to memory/backend)
    // Write address channel (AW)
    output logic [AXI_ID_WIDTH-1:0]   fub_axi_awid,
    output logic [AXI_ADDR_WIDTH-1:0] fub_axi_awaddr,
    output logic [7:0]                 fub_axi_awlen,
    output logic [2:0]                 fub_axi_awsize,
    output logic [1:0]                 fub_axi_awburst,
    output logic                       fub_axi_awlock,
    output logic [3:0]                 fub_axi_awcache,
    output logic [2:0]                 fub_axi_awprot,
    output logic [3:0]                 fub_axi_awqos,
    output logic [3:0]                 fub_axi_awregion,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_awuser,
    output logic                       fub_axi_awvalid,
    input  logic                       fub_axi_awready,

    // Write data channel (W)
    output logic [AXI_DATA_WIDTH-1:0] fub_axi_wdata,
    output logic [SW-1:0]               fub_axi_wstrb,
    output logic                       fub_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_wuser,
    output logic                       fub_axi_wvalid,
    input  logic                       fub_axi_wready,

    // Write response channel (B)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_bid,
```

```
    input  logic [1:0]                      fub_axi_bresp,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_buser,
    input  logic                            fub_axi_bvalid,
    output logic                            fub_axi_bready,

    // Status
    output logic                            busy
);
```

21.3.1 Clock and Reset

Port	Direction	Width	Description
`aclk`	Input	1	AXI clock - all signals sampled on rising edge
`aresetn`	Input	1	Active-low asynchronous reset

21.3.2 Slave AXI Interface (s_axi_*)

21.3.2.1 Write Address Channel (AW)

Port	Direction	Width	Description
`s_axi_awid`	Input	`AXI_ID_WIDTH`	Write transaction ID
`s_axi_awaddr`	Input	`AXI_ADDR_WIDTH`	Write address
`s_axi_awlen`	Input	8	Burst length, AXI-encoded: beats - 1 (`0x00` = 1 beat, `0xFF` = 256)
`s_axi_awsize`	Input	3	Burst size (bytes per beat)
`s_axi_awburst`	Input	2	Burst type (FIXED, INCR, WRAP)
`s_axi_awlock`	Input	1	Lock type (atomic access support)
`s_axi_awcache`	Input	4	Cache attributes
`s_axi_awprot`	Input	3	Protection attributes

Port	Direction	Width	Description
s_axi_awqos	Input	4	Quality of Service identifier
s_axi_awregion	Input	4	Region identifier
s_axi_awuser	Input	AXI_USER_WIDTH	User-defined signal
s_axi_awvalid	Input	1	Write address valid
s_axi_awready	Output	1	Write address ready

21.3.2.2 Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	Input	AXI_DATA_WIDTH	Write data
s_axi_wstrb	Input	AXI_DATA_WIDTH/8	Write strobes (byte enables)
s_axi_wlast	Input	1	Last beat of burst indicator
s_axi_wuser	Input	AXI_USER_WIDTH	User-defined signal
s_axi_wvalid	Input	1	Write data valid
s_axi_wready	Output	1	Write data ready

21.3.2.3 Write Response Channel (B)

Port	Direction	Width	Description
s_axi_bid	Output	AXI_ID_WIDTH	Response transaction ID
s_axi_bresp	Output	2	Write response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_buser	Output	AXI_USER_WIDTH	User-defined signal
s_axi_bvalid	Output	1	Write response valid
s_axi_bready	Input	1	Write response ready

21.3.3 Backend Interface (fub_axi_*)

Mirrors the slave interface but in the opposite direction (output on AW/W, input on B).

21.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions in buffers (for clock gating)

21.4 Functional Description

21.4.1 Architecture

Three independent `gaxi_skid_buffer` instances do the heavy lifting, one per write channel:

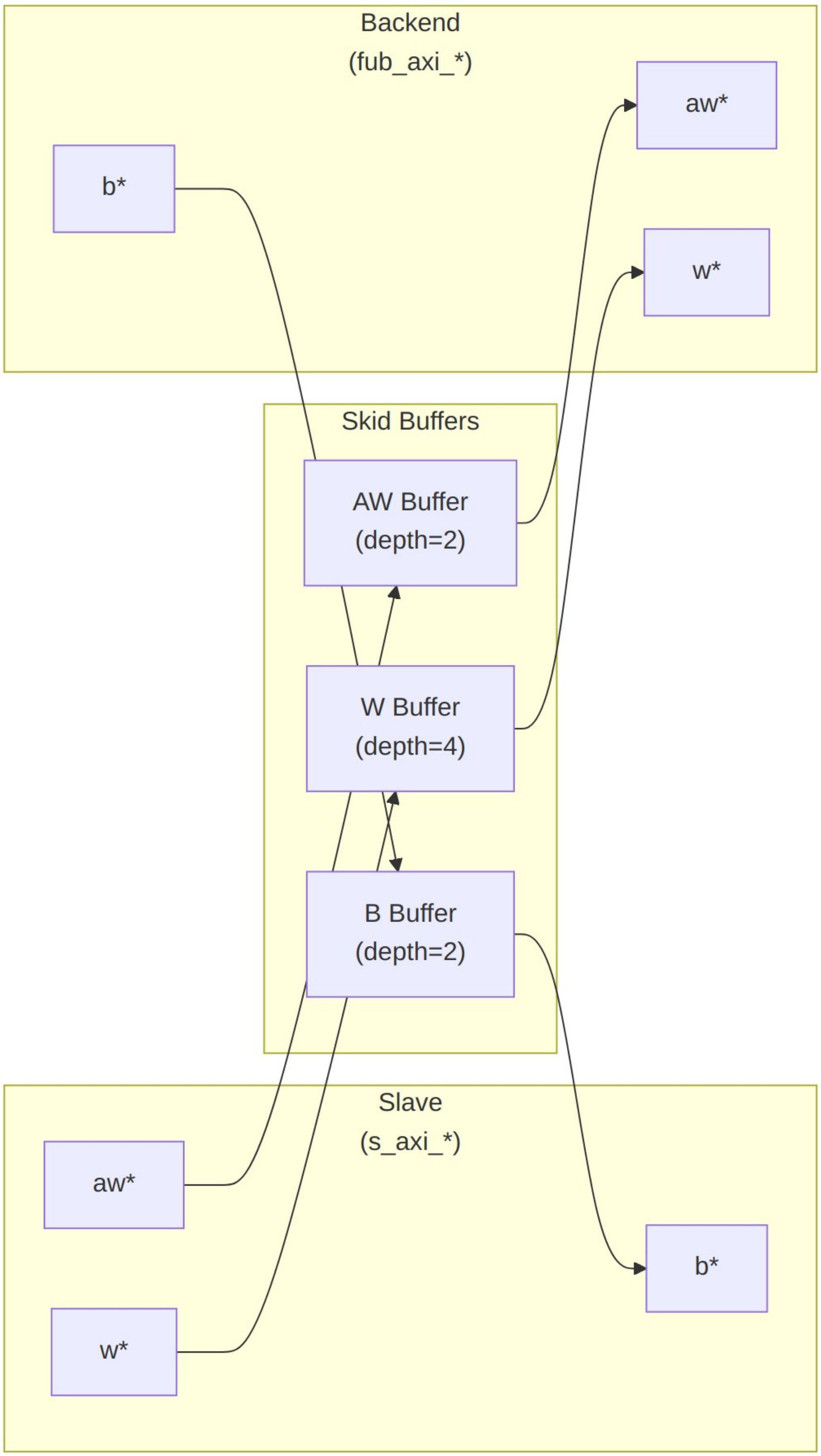

Backend
(fub_axi_*)
b*
aw*
w*
Skid Buffers
AW Buffer
(depth=2)
W Buffer
(depth=4)
B Buffer
(depth=2)
Slave
(s_axi_*)
aw*
w*
b*

Diagram 27

21.4.2 Channel Operations

21.4.2.1 Write Address (AW) Channel

1. Master presents write address and attributes via interconnect
2. AW skid buffer accepts when space available (`s_axi_awready` high)
3. Buffered address presented to backend when ready
4. Configurable depth (`SKID_DEPTH_AW`) smooths timing variations

21.4.2.2 Write Data (W) Channel

1. Master presents write data with strobes via interconnect
2. W skid buffer provides deeper buffering (default depth=4)
3. WLAST signal preserved to indicate burst boundaries
4. Independent from AW channel (AXI4 allows W before AW)

21.4.2.3 Write Response (B) Channel

1. Backend returns write response with transaction ID
2. B skid buffer captures response when master busy
3. Master receives response via interconnect when ready to accept
4. ID matching handled by AXI interconnect (not this module)

21.4.3 Busy Signal

The `busy` output indicates active transactions:

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count
> 0) ||
                s_axi_awvalid || s_axi_wvalid || fub_axi_bvalid;
```

Use cases: - **Clock gating:** Disable clock when `busy` is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

21.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the**

unstalled case – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

21.6 Usage Examples

21.6.1 Basic Integration with Memory

```
// Instantiate AXI4 slave write buffer
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_slave_wr (
    .aclk                   (axi_aclk),
    .aresetn                (axi_aresetn),

    // Slave interface (from AXI interconnect)
    .s_axi_awid             (s_axi_awid),
    .s_axi_awaddr           (s_axi_awaddr),
    .s_axi_awlen            (s_axi_awlen),
    .s_axi_awsize           (s_axi_awsize),
    .s_axi_awburst          (s_axi_awburst),
    .s_axi_awlock           (s_axi_awlock),
    .s_axi_awcache          (s_axi_awcache),
    .s_axi_awprot           (s_axi_awprot),
    .s_axi_awqos            (s_axi_awqos),
    .s_axi_awregion         (s_axi_awregion),
    .s_axi_awuser           (s_axi_awuser),
    .s_axi_awvalid          (s_axi_awvalid),
    .s_axi_awready          (s_axi_awready),

    .s_axi_wdata            (s_axi_wdata),
    .s_axi_wstrb            (s_axi_wstrb),
    .s_axi_wlast            (s_axi_wlast),
    .s_axi_wuser            (s_axi_wuser),
    .s_axi_wvalid           (s_axi_wvalid),
```

```
    .s_axi_wready       (s_axi_wready),

    .s_axi_bid          (s_axi_bid),
    .s_axi_bresp        (s_axi_bresp),
    .s_axi_buser        (s_axi_buser),
    .s_axi_bvalid       (s_axi_bvalid),
    .s_axi_bready       (s_axi_bready),

    // Backend interface (to memory controller)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    .fub_axi_awlen      (mem_awlen),
    .fub_axi_awsize     (mem_awsize),
    .fub_axi_awburst    (mem_awburst),
    .fub_axi_awlock     (mem_awlock),
    .fub_axi_awcache    (mem_awcache),
    .fub_axi_awprot     (mem_awprot),
    .fub_axi_awqos      (mem_awqos),
    .fub_axi_awregion   (mem_awregion),
    .fub_axi_awuser     (mem_awuser),
    .fub_axi_awvalid    (mem_awvalid),
    .fub_axi_awready    (mem_awready),

    .fub_axi_wdata      (mem_wdata),
    .fub_axi_wstrb      (mem_wstrb),
    .fub_axi_wlast      (mem_wlast),
    .fub_axi_wuser      (mem_wuser),
    .fub_axi_wvalid     (mem_wvalid),
    .fub_axi_wready     (mem_wready),

    .fub_axi_bid        (mem_bid),
    .fub_axi_bresp      (mem_bresp),
    .fub_axi_buser      (mem_buser),
    .fub_axi_bvalid     (mem_bvalid),
    .fub_axi_bready     (mem_bready),

    // Status
    .busy               (wr_slave_busy)
);

// Memory controller backend
memory_controller u_mem_ctrl (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_awid       (mem_awid),
    .axi_awaddr     (mem_awaddr),
```

```
    // ... rest of signals
);
```

21.6.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .clk_in            (axi_clk),
    .aresetn           (axi_resetn),
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),
    .wakeup            (wr_slave_busy),
    .clk_out           (axi_clk_gated),
    .gating            ()
);

// Connect module to gated clock
axi4_slave_wr #( ... ) u_slave_wr (
    .aclk (axi_clk_gated),
    ...
);
```

21.7 Design Notes

21.7.1 Buffer Depth Selection

`SKID_DEPTH_*` is an entry count, not a log2 exponent. The underlying `gaxi_skid_buffer` allocates one register slot per entry and tracks occupancy with a 4-bit counter, so legal values are 2..8 inclusive (any integer). Values greater than 8 overflow the occupancy counter and are not supported.

Write Address (SKID_DEPTH_AW): - Default: 2 (sufficient for most cases) - Increase if: - High-latency backend address decode - Frequent address channel backpressure - Multiple outstanding bursts needed

Write Data (SKID_DEPTH_W): - Default: 4 (deeper than AW for burst data) - Increase if: - Large burst sizes (AWLEN > 4) - High-bandwidth streaming writes - Variable backend write latency

Write Response (SKID_DEPTH_B): - Default: 2 (responses are typically single-beat) - Increase if: - Master slow to accept responses - Many concurrent outstanding writes - Response channel backpressure observed

21.7.2 Recommended Configurations

Low-Latency Memory (single outstanding write):

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_slave_wr ( ... );
```

High-Throughput Streaming (burst writes):

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(4),
    .SKID_DEPTH_W(8),     // Deep for burst data
    .SKID_DEPTH_B(4)
) u_slave_wr ( ... );
```

Multiple Outstanding Writes:

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(8),
    .SKID_DEPTH_W(8),
    .SKID_DEPTH_B(8)
) u_slave_wr ( ... );
```

21.7.3 Buffer Independence

The three skid buffers operate independently: - AW and W channels can progress at different rates - AXI4 allows W data before AW address - B responses return asynchronously based on backend timing

21.7.4 Packet Preservation

All signal groups packed and unpacked atomically: - AW channel: `{awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser}` - W channel: `{wdata, wstrb, wlast, wuser}` - B channel: `{bid, bresp, buser}`

21.7.5 Backpressure Handling

Ready signals propagate backpressure: - Interconnect sees `awready`/`wready` low when buffers full - Backend backpressure captured in buffers - Prevents data loss while decoupling timing

21.7.6 ID and Burst Handling

This module is a pass-through buffer, and it's honest about it: - Does NOT track transaction IDs - Does NOT enforce burst ordering - Relies on backend to: - Match BID to AWID - Generate appropriate BRESP

21.7.7 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

21.8 Related Modules

21.8.1 Companion Modules

- **axi4_slave_rd** - AXI4 slave read with buffering
- **axi4_slave_wr_cg** - Clock-gated variant with additional CG logic
- **axi4_slave_wr_mon** - Write transaction monitor for verification

21.8.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

21.8.3 Related Infrastructure

- **axi4_master_wr** - Corresponding AXI4 write master module
- **axi4_interconnect** - Multi-master/multi-slave crossbar

21.9 Testing

The unit testbench lives at `val/amba/test_axi4_slave_wr.py`, built on the shared AXI4 test framework in `bin/TBClasses/components/axi4/`.

21.10 References

21.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

21.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_wr.sv`

21.10.3 Documentation

- Architecture: rtl-amba Overview

- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

21.11 Navigation

- **<- Back to AXI4 Index**
- **<- Back to rtl-amba Index**
- **<- Back to Main Documentation Index**

22 AXI4 Slave Write Interface (Clock-Gated)

Module: `axi4_slave_wr_cg.sv` **Base Module:** axi4_slave_wr **Location:** `rtl/amba/axi4/` **Status:** Production Ready

---

22.1 Overview

This is the **clock-gated variant** of axi4_slave_wr. The base page owns the functional story; this page only covers what changes when you put the module behind a clock gate.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the Clock-Gated Variants Guide.

The `axi4_slave_wr_cg` module adds power optimization to `axi4_slave_wr` through activity-based clock gating:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable at runtime:** `cfg_cg_enable` / `cfg_cg_idle_count` inputs
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate

22.2 Parameters

In addition to all axi4_slave_wr parameters:

Parameter	Default	Description
`CG_IDLE_COUNT_WIDTH`	4	Width of the idle countdown, sizing `cfg_cg_idle_count`
`SKID_DEPTH_AW`	2	Skid-buffer depth on the AW channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_B`	2	Skid-buffer depth on the B channel. Legal range 2..8 inclusive; odd depths are legal.
`SKID_DEPTH_W`	4	Skid-buffer depth on the W channel. Legal range 2..8 inclusive; odd depths are legal.

The gating controls are RUNTIME INPUTS, not parameters: `cfg_cg_enable` and `cfg_cg_idle_count`; status outputs `cg_gating` / `cg_idle`. One `amba_clock_gate_ctrl` gates the whole module – no per-domain gates. The base module's `busy` output is NOT re-exported (consumed internally as a wake term). (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

22.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`AW`	`AXI_ADDR_WIDTH`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`
`AWSize`	`IW+AW+8+3+2+1+4+3+4+4+UW`
`WSize`	`DW+SW+1+UW`

Derived parameter	Default expression
BSize	IW+2+UW

22.3 Ports

Port	Dir	Width	Description
aclk	In	1	
aresetn	In	1	
cfg_cg_enable	In	1	
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axi_awid	In	[IW-1:0]	
s_axi_awaddr	In	[AW-1:0]	
s_axi_awlen	In	[7:0]	
s_axi_awsize	In	[2:0]	
s_axi_awburst	In	[1:0]	
s_axi_awlock	In	1	
s_axi_awcache	In	[3:0]	
s_axi_awprot	In	[2:0]	
s_axi_awqos	In	[3:0]	
s_axi_awregion	In	[3:0]	
s_axi_awuser	In	[UW-1:0]	
s_axi_awvalid	In	1	
s_axi_awready	Out	1	
s_axi_wdata	In	[DW-1:0]	
s_axi_wstrb	In	[SW-1:0]	
s_axi_wlast	In	1	
s_axi_wuser	In	[UW-1:0]	
s_axi_wvalid	In	1	
s_axi_wready	Out	1	
s_axi_bid	Out	[IW-1:0]	
s_axi_bresp	Out	[1:0]	

Port	Dir	Width	Description
s_axi_buser	Out	[UW-1:0]	
s_axi_bvalid	Out	1	
s_axi_bready	In	1	
fub_axi_awid	Out	[IW-1:0]	
fub_axi_awaddr	Out	[AW-1:0]	
fub_axi_awlen	Out	[7:0]	
fub_axi_awsize	Out	[2:0]	
fub_axi_awburst	Out	[1:0]	
fub_axi_awlock	Out	1	
fub_axi_awcache	Out	[3:0]	
fub_axi_awprot	Out	[2:0]	
fub_axi_awqos	Out	[3:0]	
fub_axi_awregion	Out	[3:0]	
fub_axi_awuser	Out	[UW-1:0]	
fub_axi_awvalid	Out	1	
fub_axi_awready	In	1	
fub_axi_wdata	Out	[DW-1:0]	
fub_axi_wstrb	Out	[SW-1:0]	
fub_axi_wlast	Out	1	
fub_axi_wuser	Out	[UW-1:0]	
fub_axi_wvalid	Out	1	
fub_axi_wready	In	1	
fub_axi_bid	In	[IW-1:0]	
fub_axi_bresp	In	[1:0]	

Port	Dir	Width	Description
`fub_axi_buser`	In	`[UW-1:0]`	
`fub_axi_bvalid`	In	1	
`fub_axi_bready`	Out	1	
`cg_gating`	Out	1	
`cg_idle`	Out	1	

22.4 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

22.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

22.6 Usage Examples

```
axi4_slave_wr_cg #(
    // Base module parameters (see axi4_slave_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_slave_wr (except busy)
);
```

22.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

22.8 Related Modules

- **Base Module Functionality:** axi4_slave_wr.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

22.9 Testing

`val/amba/test_axi4_slave_wr_cg.py` exercises this module. It collects 1 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi4_slave_wr_cg.py -v
```

22.10 Navigation

- **<- Back to AXI4 Index**
- **<- Back to rtl-amba Index**
- **<- Back to Main Documentation Index**

23 AXI4 Slave Write Monitor

Module: `axi4_slave_wr_mon.sv` **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

23.1 Overview

The AXI4 Slave Write Monitor bolts a full transaction monitor onto a working AXI4 slave write interface. You get the buffered slave from `axi4_slave_wr` plus real-time protocol checking, error detection, performance metrics, and configurable packet filtering on slave-side write transactions – everything a verification environment needs to know what the bus actually did, not just what it was supposed to do.

23.1.1 Key Features

- **Integrated Monitoring:** Combines `axi4_slave_wr` with `axi_monitor_filtered`
- **2-Level Filtering:** Packet-type drop masks, individual event masking
- **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions
- **Timeout Monitoring:** Configurable timeout detection for stuck transactions
- **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
- **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
- **Configuration Validation:** Detects conflicting configuration settings
- **Clock Gating Support:** Busy signal for power management

23.2 Parameters

23.2.1 AXI4 Slave Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth
`SKID_DEPTH_W`	int	4	W channel skid buffer depth
`SKID_DEPTH_B`	int	2	B channel skid buffer depth
`AXI_ID_WIDTH`	int	8	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	Address bus width
`AXI_DATA_WIDTH`	int	32	Data bus width
`AXI_WSTRB_WIDTH`	int	AXI_DATA_WIDTH/8	Write strobe width (transport only – the monitor does not inspect strobes)
`AXI_USER_WIDTH`	int	1	User signal width

23.2.2 Monitor Parameters

Parameter	Type	Default	Description
`UNIT_ID`	logic [7:0]	8'h02	8-bit unit identifier in monitor packets
`AGENT_ID`	logic [15:0]	16'h0015	16-bit agent identifier in monitor packets
`MAX_TRANSACTIONS`	int	16	Maximum concurrent outstanding transactions
`ACLK_MHZ`	int	100	Clock frequency in MHz – keeps the 1 us tick exact off-100MHz
`USE_WDATA_ORDER_Q`	bit	0	Write-data ordering queue
`NUM_BANKS`	int	1	Banked transaction tables. >**1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
`CFI_MIN_FREQ_MHZ` / `CFI_MAX_FREQ_MHZ`	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
`ACTIVE_TRANS_THRESHOLD`	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when

Parameter	Type	Default	Description
			`cfg_threshold_enable=` 1. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
`ENABLE_FILTERING`	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
`ADD_PIPELINE_STAGE`	bit	0	Add register stage for timing closure
`USE_MONITOR`	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
`N_ADDR_RANGES`	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
`ADDR_RANGE_IS_ERROR`	logic [N_ADDR_RANGES-1:0]	`'0`	Per-range flavor: bit i = 0 -> DEBUG range (hit -> AddrMatch), 1 -> ERROR range (allowlist miss -> Error/ADDR_RANGE).

Parameter	Type	Default	Description
			Default all-0 (feature inert).

23.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a `generate`-if inside `axi_monitor_reporter` – the area saving is real.

Parameter	Type	Default	Effect when 0
`ENABLE_ERROR_LOGIC`	bit	1	Drop the error-detection cone
`ENABLE_TIMEOUT_LOGIC`	bit	1	Drop the timeout cone **and** the `axi_monitor_timeout` instance
`ENABLE_COMPL_LOGIC`	bit	1	Drop the completion cone
`ENABLE_THRESHOLD_LOGIC`	bit	1	Drop the threshold cone
`ENABLE_PERF_LOGIC`	bit	1	Gates only the reporter's legacy perf-packet cone and the two lifetime counters – the window FSM and meters are unconditional
`ENABLE_DEBUG_LOGIC`	bit	0	Drop the debug (trace) cone – the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (the reporter's legacy perf cone is built; `ENABLE_PERF_LOGIC` gates THAT cone only, never the measurement window) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

23.2.4 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

23.3 Ports

23.3.1 AXI4 Write Channels

Slave Interface (s_axi_*): - AW channel: `awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser, awvalid, awready` - W channel: `wdata, wstrb, wlast, wuser, wvalid, wready` - B channel: `bid, bresp, buser, bvalid, bready`

Backend Interface (fub_axi_*): - Same signals as slave, mirrored direction (to memory/backend)

23.3.2 Monitor Configuration

Configuration ports are identical to other AXI4 monitors: - Basic enables: `cfg_monitor_enable` (master runtime gate: 0 = monitor inert, CAM held clear, never stalls the datapath), `cfg_error_enable`, `cfg_timeout_enable`, `cfg_perf_enable`, `cfg_compl_enable` (completion packets), `cfg_threshold_enable` (threshold-crossed packets), `cfg_debug_enable` (debug/trace cone – the 6th reporter sub-block) - Clear: `cam_clear` (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: `cfg_timeout_cycles` (unified coarse timeout, a MICROSECOND count at full 16-bit width: 1..65535 us per phase, 0 = 16'hFFFF ~ effectively never; drives all three phase counts), `cfg_latency_threshold` - Filtering: 8 mask signals (`cfg_axi_*_mask`: pkt, error, timeout, compl, thresh, perf, addr, debug) plus `cfg_axi_err_select` - Performance window control: `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close` (see Performance Monitoring)

> The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) are fixed inside the wrapper; `cfg_active_trans_threshold` is driven by the `ACTIVE_TRANS_THRESHOLD` parameter (default MAX_TRANSACTIONS/2), not hardwired to 8, and all three are **not** top-level ports on this module.

23.3.3 Monitor Bus Output

Port	Direction	Width	Description
`monbus_va`	Output	1	Monitor packet valid

Port	Direction	Width	Description
`lid`			
`monbus_ready`	Input	1	Downstream ready to accept packet
`monbus_packet`	Output	128	`monitor_packet_t` (see format below)
`monbus_timestamp`	Output	64	`monbus_timestamp_t` paired atomically with `monbus_packet`
`debug_block_ready`	Output	1	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)
`i_mon_time`	Input	64	Free-running counter from `monbus_group_core`, sampled at packet emission

23.3.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions (for clock gating)
`active_transactions`	Output	8	Current number of outstanding transactions
`error_count`	Output	16	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`transaction_count`	Output	32	Lifetime count of completion packets actually emitted (zero-

Port	Direction	Width	Description
			extended 16-bit reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
`cfg_conflict_error`	Output	1	Configuration conflict detected

23.3.5 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

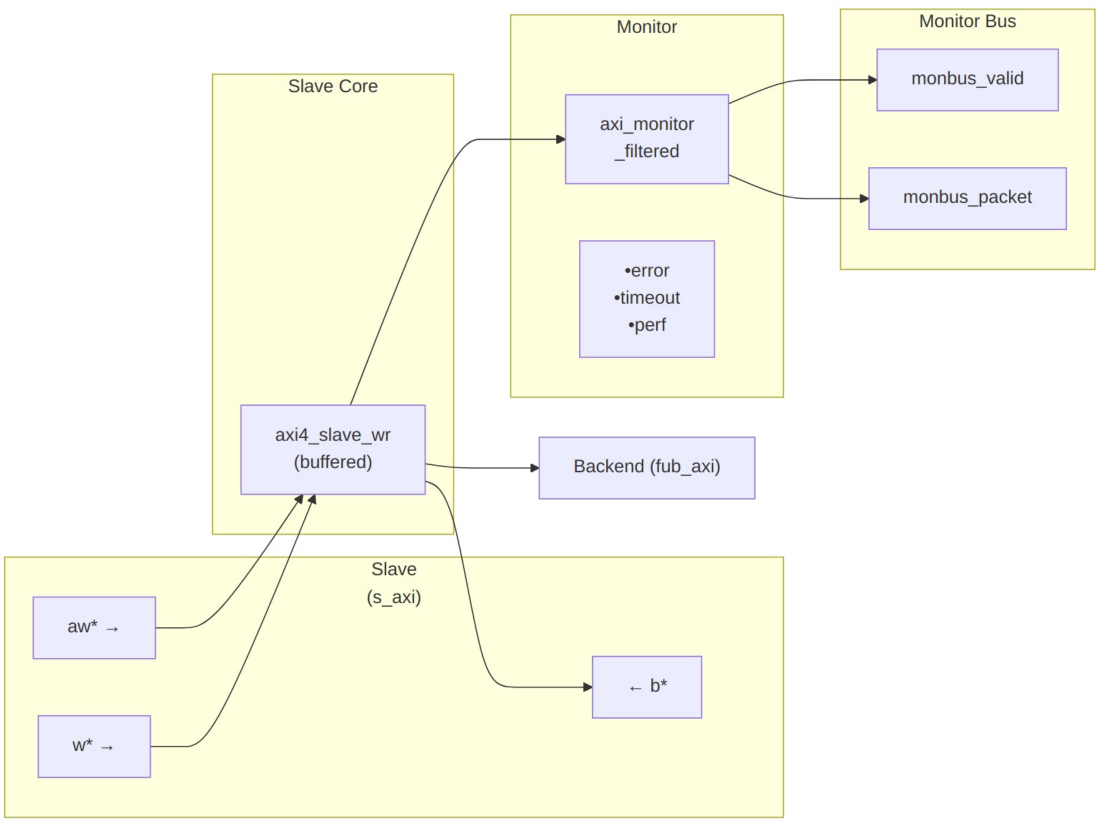
Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
`cfg_id_filter_enable`	1	High: use the window below. Low: use the parameters, bit-identical

Port	Width	Description
		to a build without this feature
`cfg_id_match_base`	`ID_WIDTH`	First ID owned
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

23.4 Functional Description



Architecture

The module instantiates two sub-modules: 1. **axi4_slave_wr** - Core AXI4 slave write functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)

23.4.1 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `s_axi_awready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `s_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (in-RTL formal property `ap_disabled_never_stalls`).

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `monitor_common_pkg`), and `block_ready` re-asserts at occupancy < `MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

23.4.2 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** – a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.

- **ERROR range** – the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper exposes **`ADDR_RANGE_IS_ERROR`** (per-range) as a parameter, so ranges can be assigned to either flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` – master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` – per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` – inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

23.4.3 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`. The **measurement-window state machine** and its counters are unconditional `always_ff` blocks in `axi_monitor_base` – outside every generate – so they are present and running in EVERY build, including `ENABLE_PERF_LOGIC = 0`. That parameter reaches one consumer: the `g_perf` generate in `axi_monitor_reporter` holding the legacy perf-packet cone and the two 16-bit lifetime counters behind `perf_completed_count` / `perf_error_count`. Setting it to 0 saves that cone and nothing else. `USE_MONITOR = 0` is what ties the perfmon outputs off. The wrapper instantiates plus a bank of W-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

23.4.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

23.4.3.2 *Utilization Counters (W Data Channel)*

Every cycle inside the window is classified by the W channel's `wvalid` / `wready` into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	wvalid && wready	productive beat transferred
`perf_bp_cycles`	32	wvalid && !wready	back-pressure (data offered, sink not ready)
`perf_starv_cycles`	32	!wvalid && wready	starvation (sink ready, no data)
`perf_idle_cycles`	32	!wvalid && !wready	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); the one-count skew is negligible for long windows.

23.4.3.3 *Throughput Counters*

Output	Width	Meaning
`perf_beat_count`	32	W data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats x (1 << AWSIZE), using the AWSIZE captured at the most recent AW address phase (upper bound: counts full-width beats and does not subtract bytes masked off by `WSTRB` on unaligned or partial beats)
`perf_burst_count`	32	AW address-phase handshakes

The integrator computes average burst length as `perf_beat_count / perf_burst_count`.

23.4.3.4 Performance Monitoring Ports

Port	Direction	Width	Description
`cfg_perf_enable`	Input	1	Enable performance-metric packet generation
`cfg_start_event_sel`	Input	3	Window **start** event source select
`cfg_end_event_sel`	Input	3	Window **end** event source select
`cfg_start_trigger`	Input	1	Pulse: open the measurement window
`cfg_end_trigger`	Input	1	Pulse: close the measurement window
`cfg_window_force_close`	Input	1	Software override: force the window closed
`window_active`	Output	1	High while a measurement window is open
`window_cycles`	Output	32	Cycles elapsed in the current window
`perf_prod_cycles`	Output	32	wvalid && wready cycles
`perf_bp_cycles`	Output	32	wvalid && !wready cycles (back-pressure)
`perf_starv_cycles`	Output	32	!wvalid && wready cycles (starvation)
`perf_idle_cycles`	Output	32	!wvalid && !wready cycles
`perf_beat_count`	Output	32	W data beats transferred
`perf_byte_count`	Output	64	bytes transferred
`perf_burst_count`	Output	32	AW address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

23.4.4 Monitor Packet Format

Identical 128-bit format (with 64-bit side-band timestamp) as other AXI4 monitors. See axi4_master_rd_mon for complete specification.

23.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

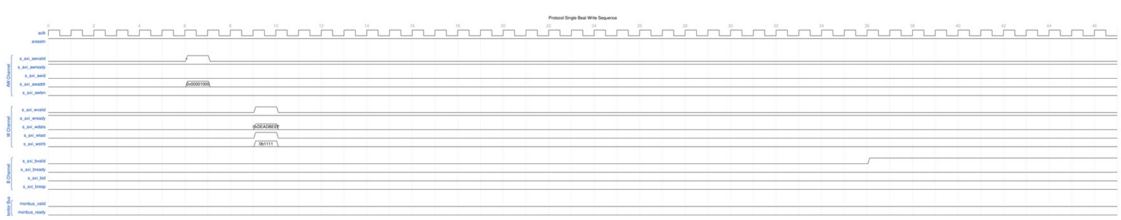
No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

23.6 Waveforms

The following waveforms show AXI4 slave write monitor behavior from the slave perspective:

23.6.1 Scenario 1: Single-Beat Write (Slave View)

AXI4 write transaction from slave interface perspective:



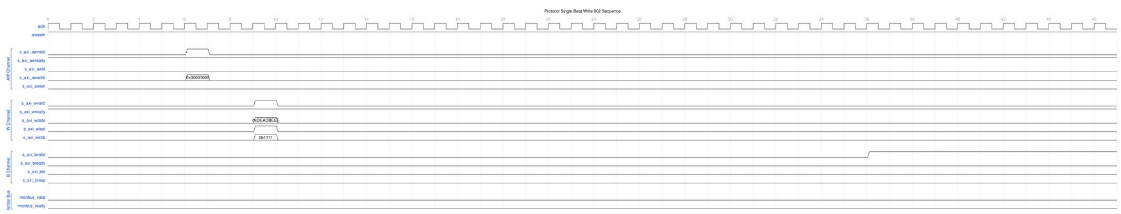
Single Beat Write Slave

WaveJSON: single_beat_write_001.json

Key Observations: - Slave-side AW channel (s_axi_aw) - *Slave-side W channel data (s_axi_w)* - Slave-side B channel response (s_axi_b*) - Monitor packet generation from slave perspective - Slave BRESP status monitoring - Transaction tracking in slave context

23.6.2 Scenario 2: Alternative Single-Beat Write (Slave View)

Variant write transaction with different timing from slave:



Single Beat Write Slave Alt

WaveJSON: single_beat_write_002_001.json

Key Observations: - Slave ready signal behavior - AWREADY backpressure effects - WREADY flow control - BVALID generation timing - Slave latency monitoring - Error detection from slave side

---

23.7 Usage Examples

23.7.1 Basic Integration with Memory

```
axi4_slave_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (2),
    .AGENT_ID           (21),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_slave_wr_mon (
    .aclk               (axi_aclk),
    .aresetn            (axi_aresetn),

    // Slave interface (from interconnect)
    .s_axi_awid         (s_axi_awid),
    .s_axi_awaddr       (s_axi_awaddr),
    // ... rest of AW/W/B signals
```

```
    // Backend interface (to memory controller)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    // ... rest of AW/W/B signals

    // Monitor configuration
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd15),    // 15 microseconds per phase
(full 16-bit range)
    .cfg_freq_sel         (4'd0),   // counter_freq_invariant LUT
index; scales the 1 us tick
    .cam_clear            (1'b0),   // hold high one cycle while idle
to clear the CAM -- do NOT leave unconnected
    .cfg_latency_threshold  (32'd1000),

    .cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR,
TIMEOUT
    .cfg_axi_error_mask     (16'h0000),
    .cfg_axi_timeout_mask   (16'h0000),
    .cfg_axi_compl_mask     (16'hFFFF),
    // ... rest of mask signals

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (wr_slave_busy),
    .active_transactions    (wr_active),
    .error_count            (wr_errors),
    .transaction_count      (wr_count),
    .cfg_conflict_error     (cfg_conflict)
);

// Memory controller backend
memory_controller u_mem (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_awid       (mem_awid),
    .axi_awaddr     (mem_awaddr),
```

```
    // ...
);
```

23.8 Design Notes

23.8.1 Configuration Strategies

23.8.1.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch slave-side write errors

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),      // Detect SLVERR/DECERR from
backend
.cfg_timeout_enable     (1'b1),      // Detect backend timeouts
.cfg_perf_enable        (1'b0),      // Disable (reduces traffic)

// Filtering - pass error and timeout only
.cfg_axi_pkt_mask       (16'hFFF6),  // Drop all but ERROR, TIMEOUT
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask   (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask      (16'hFFFF),  // Drop performance

// Timeouts
.cfg_timeout_cycles     (16'd15),    // 15 microseconds per phase:
allow backend write latency
.cfg_latency_threshold  (32'd1000)
```

23.8.1.2 Strategy 2: Performance Analysis

Goal: Analyze slave write performance

```
// Enable configuration
.cfg_monitor_enable     (1'b1),
.cfg_error_enable       (1'b1),
.cfg_timeout_enable     (1'b0),
.cfg_perf_enable        (1'b1),      // Enable performance metrics

// Filtering - pass error and performance only
.cfg_axi_pkt_mask       (16'hFFEE),  // Drop all but ERROR, PERF (set
bit = drop)
.cfg_axi_error_mask     (16'h0000),  // Pass all errors
.cfg_axi_perf_mask      (16'h0000),  // Pass all performance
.cfg_axi_compl_mask     (16'hFFFF),  // Drop completions
```

23.8.2 Slave-Side Monitoring

Key Differences from Master Monitors: - Monitors from slave perspective (interconnect to backend) - Tracks backend write response latency - Detects backend timeout scenarios - Default UNIT_ID=2, AGENT_ID=21 (distinguishes from master)

Slave Write Sequence: 1. AW channel: Master write address arrives at slave 2. Monitor captures: Address, ID, burst parameters 3. AW forwarded to backend (memory/logic) 4. W channel: Master sends data beats 5. Monitor tracks: Write beat count, WLAST 6. B channel: Backend returns write response 7. Monitor tracks: Response latency, BRESP 8. B forwarded back to master via interconnect

Common Slave Errors Detected: - **Backend timeout:** AW accepted but B never returned - **Write data timeout:** AW accepted but W never completed - **SLVERR:** Slave error (access violation, parity error, write protection) - **DECERR:** Decode error (shouldn't occur at slave, but detected) - (Burst-length mismatch is NOT detected – completion logic only) - **ID corruption:** BID doesn't match tracked AWID - (WSTRB violations are NOT detected – no strobe reaches the monitor)

23.8.3 Performance Considerations

Backend Latency Monitoring: - Tracks AW-to-B response latency - Includes W channel completion time - Configurable timeout threshold - Separate from master-side latency

Typical Timeout Values: - SRAM backend: 100-500 cycles - DDR controller: 1000-5000 cycles - Flash/EEPROM: 10000+ cycles (write operations) - PCIe/external: 10000+ cycles

Write-Specific Timing: - AW-W ordering flexible in AXI4 - Backend may wait for WLAST before responding - B response latency includes W channel completion

23.8.4 Buffer Depth Guidelines

Same as axi4_slave_wr: - **SKID_DEPTH_AW:** 2 (default) - handles interconnect backpressure - **SKID_DEPTH_W:** 4 (default) - buffers burst write data - **SKID_DEPTH_B:** 2 (default) - write responses are single-beat - Increase for high-latency backends or large bursts

23.8.5 Write Transaction Characteristics

Burst Write Monitoring: - Monitors AW channel for address capture - Tracks W channel beats until WLAST - Correlates B channel response with AWID - Completes write data on WLAST or the expected count – no burst-length mismatch event exists - (WSTRB is NOT checked – no strobe signal reaches the monitor)

Performance Impact: - Write bursts more efficient than single writes - Backend may buffer/pipeline write data - A B response before W completion is flagged as a protocol error (EVT_PROTOCOL) – the monitor does not model response reordering - Monitor packets generated per transaction, not per beat

---

23.9 Related Modules

23.9.1 Companion Monitors

- **axi4_master_rd_mon** - AXI4 master read with monitoring
- **axi4_master_wr_mon** - AXI4 master write with monitoring
- **axi4_slave_rd_mon** - AXI4 slave read with monitoring

23.9.2 Base Modules

- **axi4_slave_wr** - Functional AXI4 slave write (without monitoring)
- **axi_monitor_filtered** - Monitoring engine with filtering (monitor/)

23.9.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **axi_monitor_base** - Core monitoring logic (monitor/)
- **axi_monitor_trans_mgr** - Transaction tracking (monitor/)

23.10 Testing

The unit testbench lives at `val/amba/test_axi4_slave_wr_mon.py`, built on the shared AXI4 test framework in `bin/TBClasses/components/axi4/`.

23.11 References

23.11.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: monitor_package_spec.md

23.11.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_wr_mon.sv`

23.11.3 Documentation

- Configuration Guide: AXI Monitor Base
- Architecture: rtl-amba Overview
- AXI4 Index: README.md

Last Updated: 2026-07-18

23.12 Navigation

- <- Back to AXI4 Index
- <- Back to rtl-amba Index
- <- Back to Main Documentation Index

24 AXI4 Slave Write Monitor (Clock-Gated)

Module: `axi4_slave_wr_mon_cg.sv` **Base Module:** axi4_slave_wr_mon **Location:** `rtl/amba/axi4/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/monitor/`) **Status:** Production Ready

---

24.1 Overview

This is the **clock-gated variant** of axi4_slave_wr_mon. The base page owns the functional story; this page only covers what changes when the whole monitor sits behind a clock gate.

For complete clock-gating documentation, usage examples, and configuration guidelines, see the Clock-Gated Variants Guide.

The `axi4_slave_wr_mon_cg` module adds power optimization to `axi4_slave_wr_mon` through activity-based clock gating:

- **Same Functionality:** 100% equivalent to base module
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized
- **Configurable at runtime:** `cfg_cg_enable` / `cfg_cg_idle_count` (one gate, no domains)
- **Zero Overhead When Disabled:** `cfg_cg_enable=0` bypasses the gate at runtime

---

24.2 Parameters

MOST axi4_slave_wr_mon parameters pass through. As of 2026-09-01 only ACTIVE_TRANS_THRESHOLD is NOT forwarded, and that one is harmless: its inner default is MAX_TRANSACTIONS/2, computed from the MAX_TRANSACTIONS this wrapper DOES forward.

An earlier version of this page listed nine more as unforwarded, which was accurate when written. ACLK_MHZ, CFI_MIN_FREQ_MHZ, CFI_MAX_FREQ_MHZ, USE_WDATA_ORDER_Q, NUM_BANKS and ADDR_RANGE_IS_ERROR were threaded through every _cg wrapper on 2026-09-01, and ID_FILTER_ENABLE / ID_MATCH_BASE / ID_MATCH_COUNT the day before. Until then a clock-gated build could not state its clock frequency, so the 1 us timer tick was pinned to the 100 MHz default and every microsecond-denominated timeout was miscalibrated on any other clock – silently.

Parameter	Default	Description
CG_IDLE_COUNT_WIDTH	4	Width of the idle countdown, sizing cfg_cg_idle_count
USE_MONITOR	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	0	Number of address-range comparators (forwarded to base module).
ADDR_FILTER_ENABLE	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
ID_FILTER_ENABLE	0	Synthesises the ID report filter (see cfg_id_* for the runtime override).
ID_MATCH_BASE	0	First ID this instance owns.
ID_MATCH_COUNT	0	How many IDs; 0 means ALL, so a zeroed register block does not silently filter everything away.

Parameter	Default	Description
ADD_PIPELINE_STAGE	0	Insert a register stage for timing closure. Costs a cycle of latency. (Add register stage for timing closure)
ENABLE_FILTERING	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing.
SKID_DEPTH_AW	2	Skid-buffer depth on the AW channel. Legal range 2..8 inclusive; odd depths are legal.
SKID_DEPTH_B	2	Skid-buffer depth on the B channel. Legal range 2..8 inclusive; odd depths are legal.
SKID_DEPTH_W	4	Skid-buffer depth on the W channel. Legal range 2..8 inclusive; odd depths are legal.
AGENT_ID	16'h0015	Agent identifier emitted in the `agent_id` field of every monitor packet. Pairs with `UNIT_ID` to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID	8'h02	Unit identifier emitted in the `unit_id` field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

Gating is controlled by RUNTIME inputs `cfg_cg_enable` / `cfg_cg_idle_count` with status outputs `cg_gating` / `cg_idle`; ONE `amba_clock_gate_ctrl` gates the entire inner module. (The ENABLE_CLOCK_GATING / CG_IDLE_CYCLES / CG_GATE_* interface this page once documented never existed.)

Base-module ports are forwarded EXCEPT `debug_block_ready`, which this wrapper ties off (use the base module for the backpressure tap) – including the full performance-monitoring interface (see Performance Monitoring below). The six `ENABLE_*_LOGIC` synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables are also passed straight through.

24.2.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

Neither filter un-filters entries already admitted to the transaction table, which is what makes changing them at runtime safe.

24.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`AXI_WSTRB_WIDTH`	`AXI_DATA_WIDTH / 8`
`DW`	`AXI_DATA_WIDTH`
`IW`	`AXI_ID_WIDTH`
`SW`	`AXI_WSTRB_WIDTH`
`UW`	`AXI_USER_WIDTH`

24.3 Ports

Port	Dir	Width	Description
`aclk`	In	1	
`aresetn`	In	1	
`cam_clear`	In	1	sync clear of the monitor trans CAM
`s_axi_awid`	In	`[IW-1:0]`	
`s_axi_awaddr`	In	`[AW-1:0]`	
`s_axi_awlen`	In	`[7:0]`	
`s_axi_awsize`	In	`[2:0]`	
`s_axi_awburst`	In	`[1:0]`	
`s_axi_awlock`	In	1	
`s_axi_awcache`	In	`[3:0]`	
`s_axi_awprot`	In	`[2:0]`	
`s_axi_awqos`	In	`[3:0]`	
`s_axi_awregion`	In	`[3:0]`	
`s_axi_awuser`	In	`[UW-1:0]`	
`s_axi_awvalid`	In	1	
`s_axi_awready`	Out	1	
`s_axi_wdata`	In	`[DW-1:0]`	

Port	Dir	Width	Description
s_axi_wstrb	In	[SW-1:0]	
s_axi_wlast	In	1	
s_axi_wuser	In	[UW-1:0]	
s_axi_wvalid	In	1	
s_axi_wready	Out	1	
s_axi_bid	Out	[IW-1:0]	
s_axi_bresp	Out	[1:0]	
s_axi_buser	Out	[UW-1:0]	
s_axi_bvalid	Out	1	
s_axi_bready	In	1	
fub_axi_awid	Out	[IW-1:0]	
fub_axi_awaddr	Out	[AW-1:0]	
fub_axi_awlen	Out	[7:0]	
fub_axi_awsize	Out	[2:0]	
fub_axi_awburst	Out	[1:0]	
fub_axi_awlock	Out	1	
fub_axi_awcache	Out	[3:0]	
fub_axi_awprot	Out	[2:0]	
fub_axi_awqos	Out	[3:0]	
fub_axi_awregion	Out	[3:0]	
fub_axi_awuser	Out	[UW-1:0]	
fub_axi_awvalid	Out	1	
fub_axi_awready	In	1	
fub_axi_wdata	Out	[DW-1:0]	
fub_axi_wstrb	Out	[SW-1:0]	
fub_axi_wlast	Out	1	
fub_axi_wuser	Out	[UW-1:0]	
fub_axi_wvalid	Out	1	
fub_axi_wready	In	1	
fub_axi_bid	In	[IW-1:0]	

Port	Dir	Width	Description
fub_axi_bresp	In	[1:0]	
fub_axi_buser	In	[UW-1:0]	
fub_axi_bvalid	In	1	
fub_axi_bready	Out	1	
cfg_monitor_enable	In	1	Enable monitoring
cfg_error_enable	In	1	Enable error detection
cfg_timeout_enable	In	1	Enable timeout detection
cfg_perf_enable	In	1	Enable performance monitoring
cfg_compl_enable	In	1	Enable completion packets
cfg_threshold_enable	In	1	Enable threshold packets
cfg_debug_enable	In	1	Enable debug packets
cfg_timeout_cycles	In	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	In	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	In	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	In	[15:0]	Drop mask for packet types
cfg_axi_err_select	In	[15:0]	Error select for packet types (for

Port	Dir	Width	Description
			future routing)
cfg_axi_error_mask	In	[15:0]	Individual error event mask
cfg_axi_timeout_mask	In	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	In	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	In	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	In	[15:0]	Individual performance event mask
cfg_axi_addr_mask	In	[15:0]	Individual address match event mask
cfg_axi_debug_mask	In	[15:0]	Individual debug event mask
cfg_cg_enable	In	1	Enable clock gating
cfg_cg_idle_count	In	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	In	1	
cfg_addr_range_enable	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_low	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_range_high	In	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES :	

Port	Dir	Width	Description
		1)-1:0]	
cfg_addr_filter_enable	In	1	
cfg_addr_filter_low	In	[AW-1:0]	
cfg_addr_filter_high	In	[AW-1:0]	
cfg_id_filter_enable	In	1	
cfg_id_match_base	In	[IW-1:0]	
cfg_id_match_count	In	[IW:0]	
i_mon_time	In	1	
monbus_valid	Out	1	Monitor bus valid
monbus_ready	In	1	Monitor bus ready
monbus_packet	Out	1	Monitor packet (128-bit)
monbus_timestamp	Out	1	Side-band sampled time
busy	Out	1	
active_transactions	Out	[7:0]	Number of active transactions
error_count	Out	[15:0]	Total error count
transaction_count	Out	[31:0]	Total transaction count
cg_gating	Out	1	Gated clock is stopped
cg_idle	Out	1	No activity observed
cfg_conflict_error	Out	1	Configuration conflict detected
cfg_start_event_sel	In	[2:0]	
cfg_end_event_sel	In	[2:0]	

Port	Dir	Width	Description
cfg_start_trigger	In	1	
cfg_end_trigger	In	1	
cfg_window_force_close	In	1	
window_active	Out	1	
window_cycles	Out	[31:0]	
perf_prod_cycles	Out	[31:0]	
perf_bp_cycles	Out	[31:0]	
perf_starv_cycles	Out	[31:0]	
perf_idle_cycles	Out	[31:0]	
perf_beat_count	Out	[31:0]	
perf_byte_count	Out	[63:0]	
perf_burst_count	Out	[31:0]	

24.4 Functional Description

24.4.1 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_slave_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module – see Performance Monitoring in axi4_slave_wr_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AW (write address) handshakes. **WARNING – gating vs window accounting:** an open measurement window is NOT a wake term, and the entire inner monitor (window state machine and counters included) runs on the gated clock. If the bus idles past `cfg_cg_idle_count` with a window open, the counters FREEZE while wall-clock time passes, and trigger pulses arriving while gated are DROPPED. For exact wall-clock windows or idle-bus triggering, hold `cfg_cg_enable` low around the measurement, or use the base module.

24.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH_AW`	2 entries
`SKID_DEPTH_W`	4 entries
`SKID_DEPTH_B`	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

24.6 Usage Examples

```
axi4_slave_wr_mon_cg #(
    // Base module parameters (see axi4_slave_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd8),
    .cg_gating(), .cg_idle(),
    // ... all other ports same as axi4_slave_wr_mon (except
debug_block_ready)
);
```

24.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

24.8 Related Modules

- **Base Module Functionality:** axi4_slave_wr_mon.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

24.9 Testing

`val/amba/test_axi4_slave_wr_mon_cg.py` exercises this module. It collects 3 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/amba/test_axi4_slave_wr_mon_cg.py -v
```

24.10 Navigation

- <- Back to AXI4 Index
- <- Back to rtl-amba Index
- <- Back to Main Documentation Index

25 AXI4 Slave Write Stub

Module: `axi4_slave_wr_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

25.1 Overview

The AXI4 Slave Write Stub gives you a simplified packed-data interface on the receiving end of AXI4 write transactions. Skid buffers pack and unpack the AW (write address), W (write data), and B (write response) channels into simple packet interfaces, which makes it a natural fit for testbenches and integration scenarios where you want a slave without the protocol ceremony.

25.1.1 Key Features

- Packed packet interface for AW, W, and B channels
- Configurable skid buffer depths for each channel
- Full AXI4 write transaction support
- Burst, ID, strobe, user signal support
- Parameterized data widths

25.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	2	AW channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_W`	int	4	W channel skid buffer depth in entries (2..8 inclusive, any integer)
`SKID_DEPTH_B`	int	2	B channel skid buffer depth in entries (2..8 inclusive, any integer)

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)

25.3 Ports

25.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	AXI reset (active low)

25.3.2 AXI4 Write Address Channel (AW)

Port	Direction	Width	Description
s_axi_awid	Input	IW	Write address

Port	Direction	Width	Description
			ID
s_axi_awaddr	Input	AW	Write address
s_axi_awlen	Input	8	Burst length
s_axi_awsize	Input	3	Burst size
s_axi_awburst	Input	2	Burst type
s_axi_awlock	Input	1	Lock type
s_axi_awcache	Input	4	Cache type
s_axi_awprot	Input	3	Protection type
s_axi_awqos	Input	4	Quality of service
s_axi_awregion	Input	4	Region identifier
s_axi_awuser	Input	UW	User signal
s_axi_awvalid	Input	1	Write address valid
s_axi_awready	Output	1	Write address ready

25.3.3 AXI4 Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	Input	DW	Write data
s_axi_wstrb	Input	SW	Write strobes
s_axi_wlast	Input	1	Write last
s_axi_wuser	Input	UW	User signal
s_axi_wvalid	Input	1	Write data valid
s_axi_wready	Output	1	Write data ready

25.3.4 AXI4 Write Response Channel (B)

Port	Direction	Width	Description
s_axi_bid	Output	IW	Response ID

Port	Direction	Width	Description
`s_axi_bresp`	Output	2	Write response
`s_axi_buser`	Output	UW	User signal
`s_axi_bvalid`	Output	1	Write response valid
`s_axi_bready`	Input	1	Write response ready

25.3.5 AW Packet Interface

Port	Direction	Width	Description
`fub_axi_awvalid`	Output	1	AW packet valid
`fub_axi_awready`	Input	1	Ready to accept AW packet
`fub_axi_aw_count`	Output	4	AW buffer occupancy
`fub_axi_aw_pkt`	Output	AWSize	Packed AW packet data

25.3.6 W Packet Interface

Port	Direction	Width	Description
`fub_axi_wvalid`	Output	1	W packet valid
`fub_axi_wready`	Input	1	Ready to accept W packet
`fub_axi_w_pkt`	Output	WSize	Packed W packet data

25.3.7 B Packet Interface

Port	Direction	Width	Description
`fub_axi_bvalid`	Input	1	B packet valid
`fub_axi_bready`	Output	1	Ready to accept B

Port	Direction	Width	Description
			packet
fub_axi_b_pkt	Input	BSize	Packed B packet data

25.4 Functional Description

25.4.1 Architecture

One skid buffer per channel, packets in and out the other side – that's the whole trick:

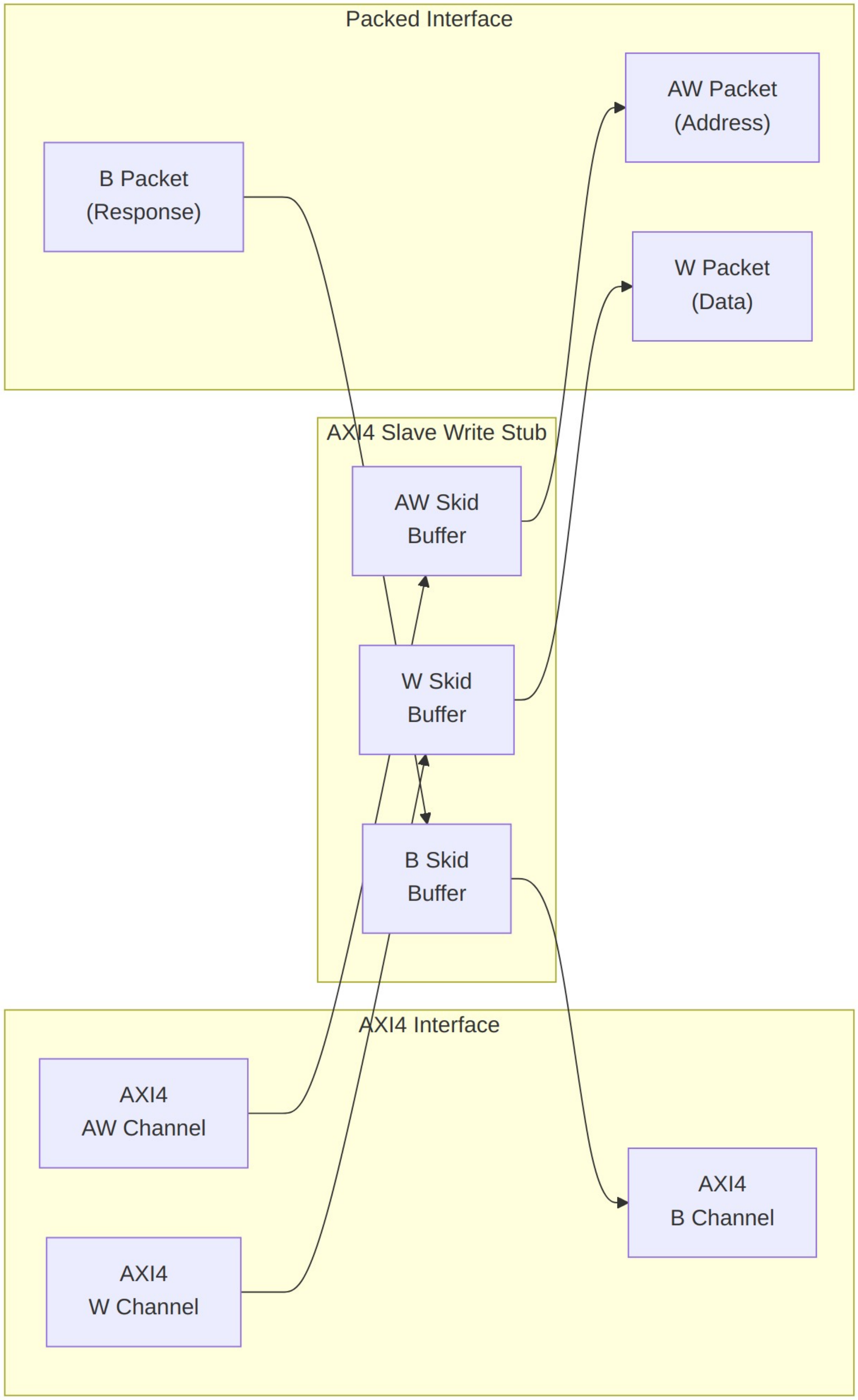
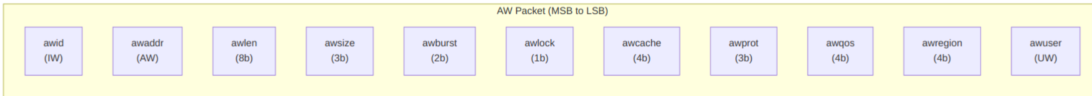

Packed Interface
B Packet
(Response)
AW Packet
(Address)
W Packet
(Data)
AXI4 Slave Write Stub
AW Skid
Buffer
W Skid
Buffer
B Skid
Buffer
AXI4 Interface
AXI4
AW Channel
AXI4
W Channel
AXI4
B Channel

Diagram 29

25.4.2 Packet Formats

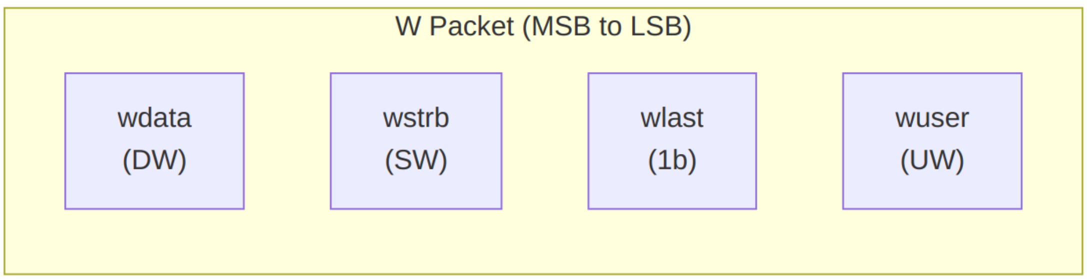


AW Packet Structure (Write Address)

Bit Positions:

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

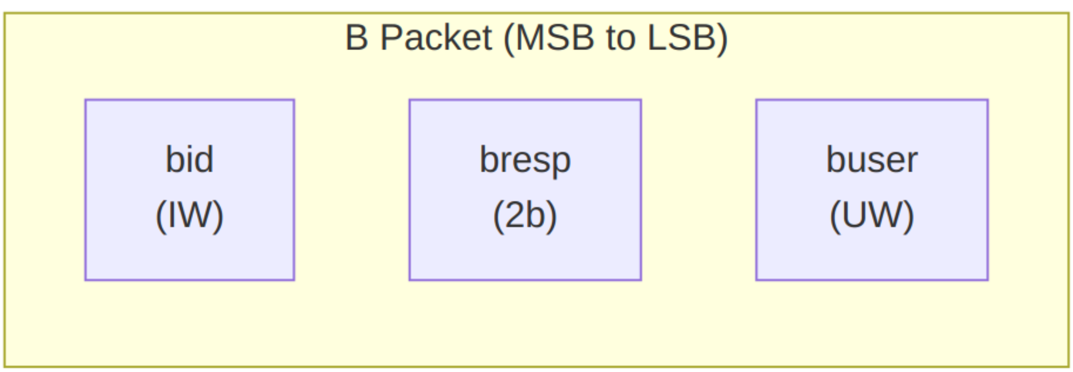


W Packet Structure (Write Data)

Bit Positions:

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}

Width = DW + SW + 1 + UW
```

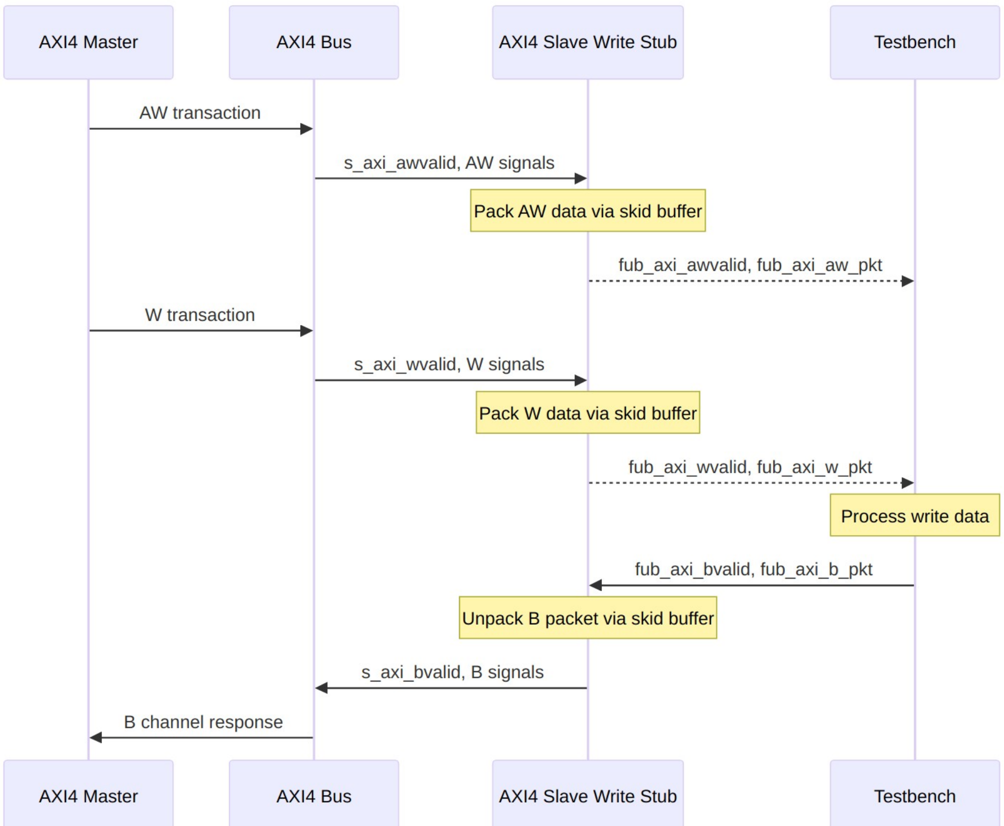


B Packet Structure (Write Response)

Bit Positions:

```
fub_axi_b_pkt = {bid, bresp, buser}
```

```
Width = IW + 2 + UW
```



Transaction Flow

25.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- AXI AW signals (s_axi_awvalid, s_axi_awaddr, s_axi_awlen, etc.)
- fub_axi_awvalid, fub_axi_awready, fub_axi_aw_pkt
- AXI W signals (s_axi_wvalid, s_axi_wdata, s_axi_wstrb, s_axi_wlast, etc.)
- fub_axi_wvalid, fub_axi_wready, fub_axi_w_pkt
- fub_axi_bvalid, fub_axi_bready, fub_axi_b_pkt

- AXI B signals (s_axi_bvalid, s_axi_bid, s_axi_bresp, etc.)
- Packet-to-AXI timing relationship with skid buffer operation

25.6 Usage Examples

Same pattern as the combined stub: instantiate, parse packets MSB-first, and drive responses back as packed words. The localparams have to match your instance widths.

```
axi4_slave_wr_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_wr_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave write interface
    .s_axi_awid      (s_axi_awid),
    .s_axi_awaddr    (s_axi_awaddr),
    .s_axi_awlen     (s_axi_awlen),
    .s_axi_awsize    (s_axi_awsize),
    .s_axi_awburst   (s_axi_awburst),
    .s_axi_awlock    (s_axi_awlock),
    .s_axi_awcache   (s_axi_awcache),
    .s_axi_awprot    (s_axi_awprot),
    .s_axi_awqos     (s_axi_awqos),
    .s_axi_awregion  (s_axi_awregion),
    .s_axi_awuser    (s_axi_awuser),
    .s_axi_awvalid   (s_axi_awvalid),
    .s_axi_awready   (s_axi_awready),

    .s_axi_wdata     (s_axi_wdata),
    .s_axi_wstrb     (s_axi_wstrb),
    .s_axi_wlast     (s_axi_wlast),
    .s_axi_wuser     (s_axi_wuser),
    .s_axi_wvalid    (s_axi_wvalid),
    .s_axi_wready    (s_axi_wready),

    .s_axi_bid       (s_axi_bid),
    .s_axi_bresp     (s_axi_bresp),
    .s_axi_buser     (s_axi_buser),
```

```
    .s_axi_bvalid    (s_axi_bvalid),
    .s_axi_bready    (s_axi_bready),

    // Packed AW interface
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    // Packed W interface
    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    // Packed B interface
    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt)
);

// Parse AW packet
localparam int AWSize = 8+32+8+3+2+1+4+3+4+4+4;  // match your
instance widths
wire [7:0]  aw_id     = tb_aw_pkt[AWSize-1:AWSize-8];
wire [31:0] aw_addr   = tb_aw_pkt[AWSize-9:AWSize-40];
wire [7:0]  aw_len    = tb_aw_pkt[AWSize-41:AWSize-48];
wire [2:0]  aw_size   = tb_aw_pkt[AWSize-49:AWSize-51];
wire [1:0]  aw_burst  = tb_aw_pkt[AWSize-52:AWSize-53];
// ... additional fields as needed

// Parse W packet
localparam int WSize = 64+8+1+4;  // match your instance widths
wire [63:0] w_data = tb_w_pkt[WSize-1:WSize-64];
wire [7:0]  w_strb = tb_w_pkt[WSize-65:WSize-72];
wire        w_last = tb_w_pkt[WSize-73];
wire [3:0]  w_user = tb_w_pkt[3:0];

// Build B packet (OKAY response)
localparam BSize = 8 + 2 + 4;  // Calculate size
assign tb_b_pkt = {
    aw_id,    // bid (match request ID)
    2'b00,    // bresp (OKAY)
    4'h0      // buser
};
```

25.7 Design Notes

25.7.1 Skid Buffer Operation

The stub uses `gaxi_skid_buffer` modules to: - Decouple timing between AXI bus and testbench - Provide configurable buffering depth per channel - Handle backpressure gracefully - Support burst transactions without stalling

Recommended Depths: - **AW Channel:** 2-4 (address transactions) - **W Channel:** 4-8 (data beats for bursts) - **B Channel:** 2-4 (responses)

25.7.2 Channel Independence

The AW, W, and B channels are independent: - AW and W arrive in any order from master - Stub handles proper AXI timing - B responses generated asynchronously by testbench

25.7.3 Packet Packing Order

AW, W, and B packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet parsing - Matches common concatenation order - Efficient for burst transaction handling

25.7.4 Internal Architecture

The stub instantiates three `gaxi_skid_buffer` modules: - **AW Skid Buffer:** Packs AXI AW channel to AW packets - **W Skid Buffer:** Packs AXI W channel to W packets - **B Skid Buffer:** Unpacks B packets to AXI B channel

All AXI protocol handling is done by the skid buffers and upstream modules.

25.8 Related Modules

- **AXI4 Slave Write** - Full AXI4 slave write module (if wrapping one)
- **AXI4 Slave Read Stub** - Corresponding read stub
- **AXI4 Slave Stub** - Combined read/write stub
- **AXI4 Master Write Stub** - Master-side write stub

25.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

25.10 Navigation

- <- Back to AXI4 Index
- <- Back to rtl-amba Index
- <- Back to Main Documentation Index